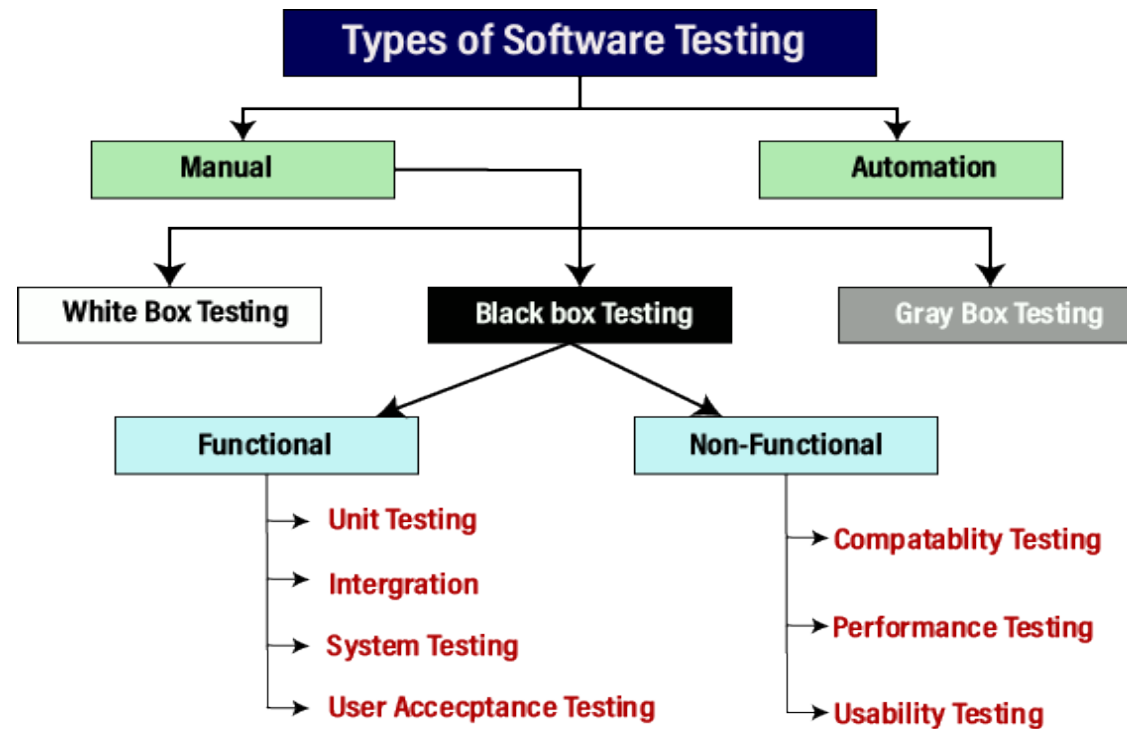


INTRODUCTION

Software testing is a process of identifying the correctness of software by considering its all attributes (Reliability, Scalability, Portability, Re- usability, Usability) and evaluating the execution of software components to find the software bugs or errors or defects. The process of software testing aims not only at finding faults in the existing software but also to improve the software in terms of efficiency, accuracy and usability.

Types of Software Testing



Manual testing

The process of checking the functionality of an application as per the customer needs without taking any help of automation tools is known as manual testing.

While performing the manual testing on any application, we do not need any specific knowledge of any testing tool.

Manual testing can be further divided into three types of testing,

White box testing

Black box testing

Gray box testing

Automation testing

Automation testing is a process of converting any manual test cases into the test scripts with the help of automation tools, or any programming language is known as automation testing.

With the help of automation testing, we can enhance the speed of our test execution.

We do not require any human efforts.

We need to write a test script and execute those scripts.

Functional Testing

- It is a type of software testing that validates the software systems against the functional requirements.
- It is performed to check whether the application is working as per the software's functional requirements or not.
- Various types of functional testing are Unit testing, Integration testing, System testing and so on.

This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.

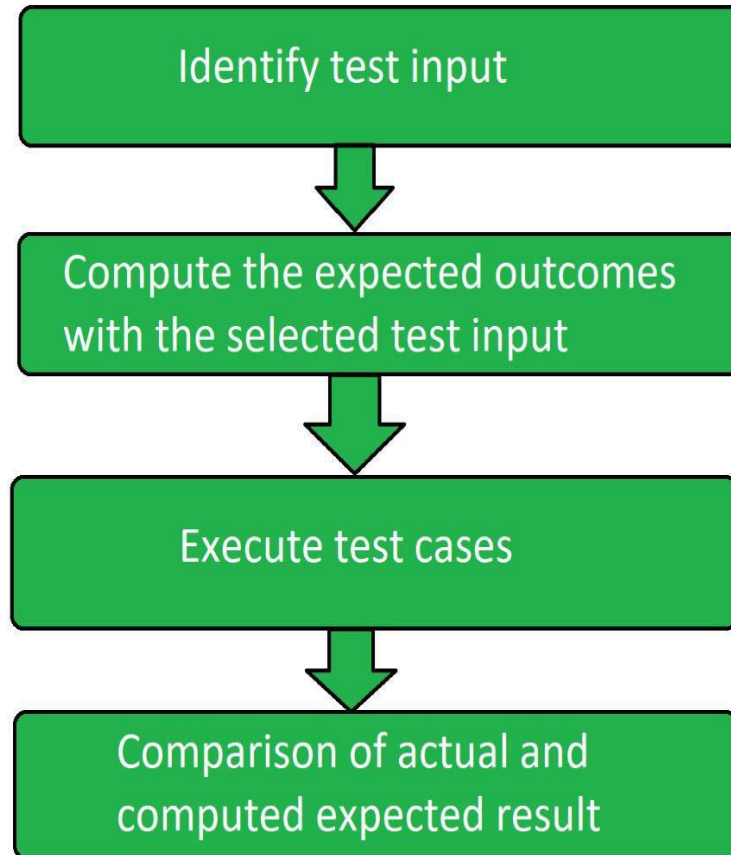
Purpose of Functional Testing

- Functional testing mainly involves black box testing and can be done manually or using automation.
- Test each function of the application: Functional testing tests each function of the application by providing the appropriate input and verifying the output against the functional requirements of the application.
- Test primary entry function: In functional testing, the tester tests each entry function of the application to check all the entry and exit points.
- Test flow of the GUI screen: In functional testing, the flow of the GUI screen is checked so that the user can navigate throughout the application.

Goal of Functional Testing

- Basic Usability: Functional testing involves basic usability testing to check whether the user can freely navigate through the screens without any difficulty.
- Mainline functions: This involves testing the main features and functions of the application.
- Accessibility: This involves testing the accessibility of the system for the user.
- Error Conditions: Functional testing involves checking whether the appropriate error messages are being displayed or not in case of error conditions.

Process of Functional Testing



Types of Functional Testing:

1. Unit Testing: Unit testing is the type of functional testing technique where the individual units or modules of the application are tested. It ensures that each module is working correctly.
—
2. Integration Testing: In Integration testing , combined individual units are tested as a group and expose the faults in the interaction between the integrated units.
3. Smoke Testing: Smoke testing is a type of functional testing technique where the basic functionality or feature of the application is tested as it ensures that the most important function works properly.
4. User Acceptance Testing: User acceptance testing is done by the client to certify that the system meets the requirements and works as intended. It is the final phase of testing before the product release.
5. Interface Testing: Interface testing is a type of software testing technique that checks the proper interaction between two different software systems.
6. System Testing: System testing is a type of software testing that is performed on the complete integrated system to evaluate the compliance of the system with the corresponding requirements.
—
7. Regression Testing: Regression testing is done to make sure that the code changes do not affect the existing functionality and the features of the application. It concentrates on whether all parts are working or not.
—
8. Sanity Testing: Sanity testing is a subset of regression testing and is done to make sure that the code changes introduced are working as expected.
—
9. White box Testing: White box testing is a type of software testing that allows the tester to verify the internal workings of the software system. This includes analyzing the code, infrastructure, and integrations with the external system.
—
10. Black box Testing: Black box testing is a type of software testing where the functionality of the software system is tested without looking at the internal workings or
—

structures of the software system.

11. Database Testing: Database testing is a type of software testing that checks the schema, tables etc., of the database under test.
12. Adhoc Testing: Adhoc testing also known as monkey testing or random testing is a type of software testing where the test cases are selected arbitrarily and outcomes are compared to check whether they are correct.
13. Recovery Testing: Recovery testing is a type of software testing that verifies the software's ability to recover from failures like hardware failures, software failures, crashes, etc.
14. Static Testing: Static testing is a type of software testing that is performed to check the
15. defects in software without actually executing the code of the software application.
16. Grey-box Testing: Grey box testing is a type of software testing that includes black-box and white-box testing.
17. Component Testing: Component testing also known as program testing or module testing, done after the unit testing. In this, the test objects can be tested independently as a component without integrating with other components.

Functional Testing Vs Non-Functional Testing

Parameters	Functional Testing	Non-functional Testing
Definition	Functional testing verifies the operations and actions of an application.	Non-functional verifies the behavior of an application.
Testing based on	It is based on the requirements of the customer.	It is based on the expectations of the customer.
Objective	The objective is to validate software actions.	The objective is to performance of the software system
Requirements	Functional testing is carried out using the functional specification.	Non-functional testing is carried out using the performance specifications.
Functionality	It describes what the product does.	It describes how the product works.
Example	Unit testing. Integration testing. Sanity testing Smoke testing. Regression testing.	Performance testing. Load testing. Stress testing. Volume testing. Usability testing.

Non-functional Testing

It is a type of software testing that checks the application for non- functional requirements like performance, scalability, portability, stress, etc. Various types of non-functional testing are Performance testing, Stress testing, Usability testing and so on.

Objectives of Non-functional Testing

- Increased usability: To increase usability, efficiency, maintainability, and portability of the product.
- Reduction in production risk: To help in the reduction of production risk related to non-functional aspects of the product.
- Cost Reduction: To help in the reduction of costs related to non-functional aspects of the product.
- Optimize installation: To optimize the installation, execution, and monitoring way of the product.
- Collect metrics: To collect and produce measurements and metrics for internal research and development.
- Enhance knowledge of product: To improve and enhance knowledge of the product behavior and technologies in use.

BLACK BOX TESTING

Black box testing is a technique of software testing which examines the functionality of software without peering into its internal structure or coding. The primary source of black box testing is a specification of requirements that is stated by the customer.

- In this method, tester selects a function and gives input value to examine its functionality.
- Checks whether the function is giving expected output or not.
- If the function produces correct output, then it is passed in testing, otherwise failed.
- The test team reports the result to the development team and then tests the next function.
- After completing testing of all functions if there are severe problems, then it is given back to the development team for correction.



Steps of black box testing

- The black box test is based on the specification of requirements, so it is examined in the beginning.
- In the second step, the tester creates a positive test scenario and an adverse test scenario by selecting valid and invalid input values to check that the software is processing them correctly or incorrectly.
- In the third step, the tester develops various test cases such as decision table, all pairs test, equivalent division, error estimation, cause-effect graph, etc.
- The fourth phase includes the execution of all test cases.
- In the fifth step, the tester compares the expected output against the actual output.
- In the sixth and final step, if there is any flaw in the software, then it is cured and tested again.

Test procedure

The test procedure of black box testing is a kind of process in which the tester has specific knowledge about the software's work, and it develops test cases to check the accuracy of the software's functionality.

- It does not require programming knowledge of the software. All test cases are designed by considering the input and output of a particular function.
- A tester knows about the definite output of a particular input, but not about how the result is arising.
- There are various techniques used in black box testing for testing like decision table technique, boundary value analysis technique, state transition, All-pair testing, cause-effect graph technique, equivalence partitioning technique, error guessing technique and use case technique.

Test cases

- Test cases are created considering the specification of the requirements.
- These test cases are generally created from working descriptions of the software including requirements, design parameters, and other specifications.
- The test designer selects both positive test scenario by taking valid input values and adverse test scenario by taking invalid input values to determine the correct output.
- Test cases are mainly designed for functional testing but can also be used for non-functional testing.
- Test cases are designed by the testing team, there is not any involvement of the development team of software.

Techniques Used in Black Box Testing

1. Decision table technique
2. Boundary value technique
3. State transition technique
4. All pair testing technique
5. Cause-Effect technique

6. Equivalence partitioning technique
7. Error guessing technique
8. Use case technique

1. Decision table technique

Decision table technique is one of the widely used case design techniques for black box testing. This is a systematic approach where various input combinations and their respective system behavior are captured in a tabular form.

This technique is used to pick the test cases in a systematic manner; it saves the testing time and gives good coverage to the testing area of the software application.

This technique is related to the correct combination of inputs and determines the result of various combinations of input. To design the test cases by decision table technique, we need to consider conditions as input and actions as output.

Example: Email account

To use an email account, for this you need to enter the email and its associated password. If both email and password are correctly matched, the user will be directed to the email account's homepage; otherwise, it will come back to the login page with an error message specified with "Incorrect Email" or "Incorrect Password."

Email (condition1)	T	T	F	F
Password (condition2)	T	F	T	F
Expected Result (Action)	Account Page	Incorrect password	Incorrect email	Incorrect email

- In this table, there are four conditions or test cases to test the login function. In the first condition if both email and password are correct, then the user should be directed to account's Homepage.

- In the second condition if the email is correct, but the password is incorrect then the function should display Incorrect Password. In the third condition if the email is incorrect, but the password is correct, then it should display Incorrect Email.
- In fourth and last condition both email and password are incorrect then the function should display Incorrect Email.
- The testing team also includes all possible test cases so that upcoming bugs can be cured at testing level.

Number of possible conditions = $2^{\text{Number of Values of the second condition}}$

Number of possible conditions = $2^2 = 4$

- While using the decision table technique, a tester determines the expected output, if the function produces expected output, then it is passed in testing, and if not then it is failed. Failed software is sent back to the development team to fix the defect.

2. Boundary value technique

- Boundary value analysis is one of the widely used case design technique for black box testing. It is used to test boundary values.
- The tester focuses on, while entering boundary value whether the software is producing correct output or not.
- Boundary values are those that contain the upper and lower limit of a variable.
- Assume that, age is a variable of any function, and its minimum value is 18 and the maximum value is 30, both 18 and 30 will be considered as boundary values.
- Test cases are developed for each and every value of the range.
- Testing of boundary values is done by making valid and invalid partitions.
- There is a function that accepts a number between 18 to 30, where 18 is the minimum and 30 is the maximum value of valid partition, the other values of this partition are 19, 20, 21, 22, 23, 24, 25, 26, 27, 28 and 29. The invalid partition consists of the numbers which are less than 18 such as 12, 14, 15, 16 and 17, and more than 30 such as 31, 32, 34, 36 and 40. Tester develops test cases for both valid and invalid partitions to capture the behavior of the system on different input conditions.



Invalid test cases	Valid test cases	Invalid test cases
11, 13, 14, 15, 16, 17	18, 19, 24, 27, 28, 30	31, 32, 36, 37, 38, 39

- The software system will be passed in the test if it accepts a valid number and gives the desired output, if it is not, then it is unsuccessful.
- In another scenario, the software system should not accept invalid numbers, and if the entered number is invalid, then it should display error message.

3. State transition technique

- It is used to capture the behavior of the software application when different input values are given to the same function.

For example: ATMs

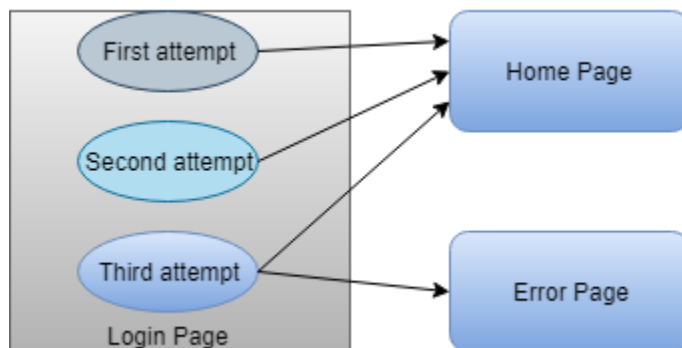
- When we withdraw money from it, it displays account details at last. Now we again do another transaction, then it again displays account details, but the details displayed after the second transaction are different from the first transaction, but both details are displayed by using the same function of the ATM. So the same function was used here but each time the output was different, this is called state transition.
- The login function we use email and password, it gives a specific number of attempts to access the application, after crossing the maximum number of attempts it gets locked with an error message.

Login Here

Email

Password

There is a login function of an application which provides a maximum three number of attempts, and after exceeding three attempts, it will be directed to an error page.



State transition table

State	Login	Validation	Redirected
S1	First Attempt	Invalid	S2
S2	Second Attempt	Invalid	S3
S3	Third Attempt	Invalid	S5

S4	Home Page
S5	Error Page

In the above state transition table, we see that state S1 denotes first login attempt. When the first attempt is invalid, the user will be directed to the second attempt (state S2). If the second attempt is also invalid, then the user will be directed to the third attempt (state S3). Now if the third and last attempt is invalid, then the user will be directed to the error page (state S5).

But if the third attempt is valid, then it will be directed to the homepage (state S4).

Let's see state transition table if third attempt is valid:

State	Login	Validation	Redirected
S1	First Attempt	Invalid	S2
S2	Second Attempt	Invalid	S3
S3	Third Attempt	Valid	S5
S4	Home Page		
S5	Error Page		

Using the above state transition table we can perform testing of any software application.

4. All pair testing technique

- All-pairs testing technique is also known as pairwise testing. It is used to test all the possible discrete combinations of values.
- This combinational method is used for testing the application that uses checkbox input, radio button input, list box, text box, etc.
- A function of a software application for testing, in which there are 10 fields to input the data, so the total number of discrete combinations is 10^4 (100 billion), but testing of all combinations is complicated because it will take a lot of time.

- Assume that there is a function with a list box that contains 10 elements, text box that can accept 1 to 100 characters, radio button, checkbox and OK button.

The input values are given below that can be accepted by the fields of the given function.

1. Check Box - Checked or Unchecked
2. List Box - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
3. Radio Button - On or Off
4. Text Box - Number of alphabets between 1 to 100.
5. OK - Does not accept any value, only redirects to the next page.

Calculation of all the possible combinations:

Check Box = 2

List Box = 10

Radio Button = 2

Text Box = 100

Total number of test cases = $2 \times 10 \times 2 \times 100$
= 4000

The total number of test cases, including negative test cases, is 4000.

- Testing of 4000 positive and negative test cases, is a very long and time-consuming process. Therefore, the task of the testing team is to reduce the number of test cases.
- The testing team considers the list box values in such a way that the first value is 0, and the other value can be any numeric number, neither positive nor negative. Ten values are now converted into 2 values.
- Values of checkbox and radio button cannot be reduced because each has a combination of only 2 values.
- At last, the value of the text box is divided into three input categories valid integer, invalid integer, and alpha-special character.
 $2 \times 2 \times 2 \times 3 = 24$ (Now, we have only 24 test cases, including negative test cases)
- In order to make text box column, put the most common input on the first place that is a valid integer, on the second place put the second most common input that is an invalid

integer, and at the last place put the least common input that is an Alpha Special Character.

- Then start filling the table, the first column is a text box with three values, the next column is a list box that has 2 values, the third column is a checkbox that has 2 values, and the last one is a radio button that also has 2 values.

Text Box	List Box	Check Box	Radio Button
Valid Integer	0	Check	ON
Invalid Integer	Other	Uncheck	OFF
Valid Integer	0	Check	ON
Invalid Integer	Other	Uncheck	OFF
AlphaSpecialCharacter	0	Check	ON
AlphaSpecialCharacter	Other	Uncheck	OFF

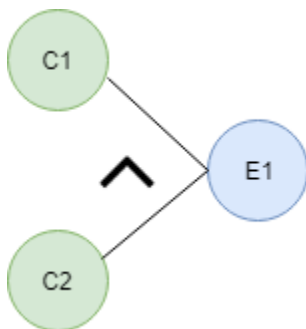
5. Cause-Effect technique

- Cause-Effect Technique underlines the relationship between a given result and all the factors affecting the result.
- Cause-Effect graph technique is based on a collection of requirements and used to determine minimum possible test cases which can cover a maximum test area of the software.
- It is used to write dynamic test cases.
- The dynamic test cases are used when code works dynamically based on user input.
- For example:
- While using email account, on entering valid email, the system accepts it but, when you enter invalid email, it throws an error message.

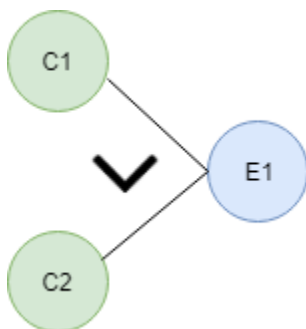
- In this technique, the input conditions are assigned with causes and the result of these input conditions with effects.
- It reduces the time of test execution and cost.
- To reduce the number of test cases but still covers all necessary test cases with maximum coverage to achieve the desired application quality.
- Cause-Effect graph technique converts the requirements specification into a logical relationship between the input and output conditions by using logical operators like AND, OR and NOT.

Notations used in the Cause-Effect Graph

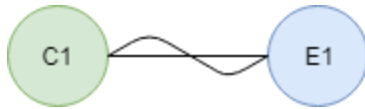
AND - E1 is an effect and C1 and C2 are the causes. If both C1 and C2 are true, then effect E1 will be true.



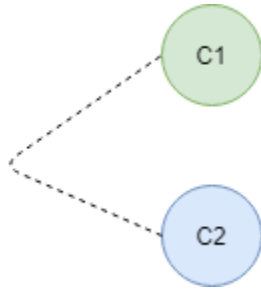
OR - If any cause from C1 and C2 is true, then effect E1 will be true.



NOT - If cause C1 is false, then effect E1 will be true.



Mutually Exclusive - When only one cause is true.



6. Equivalence partitioning technique

- Equivalence partitioning is a technique of software testing in which input data is divided into partitions of valid and invalid values, and it is mandatory that all partitions must exhibit the same behavior.
- If a condition of one partition is true, then the condition of another equal partition must also be true, and if a condition of one partition is false, then the condition of another equal partition must also be false.
- The principle of equivalence partitioning is, test cases should be designed to cover each partition at least once.
- Each value of every equal partition must exhibit the same behavior as other.
- The equivalence partitions are derived from requirements and specifications of the software.
- It helps to reduce the time of testing due to a smaller number of test cases from infinite to finite.
- It is applicable at all levels of the testing process.

Example

An OTP number which contains only six digits, less or more than six digits will not be accepted, and the application will redirect the user to the error page.

Enter OTP Number

Enter Six Digit OTP Number

Back

Submit

INVALID	INVALID	VALID	VALID
1 Test case	2 Test case	3 Test case	
DIGITS >=7	DIGITS<=5	DIGITS = 6	DIGITS = 6
93847262	9845	456234	451483

A function of the software application accepts a 10 digit mobile number.

Enter Mobile Number

Enter Ten Digit Mobile Number

Back

Submit

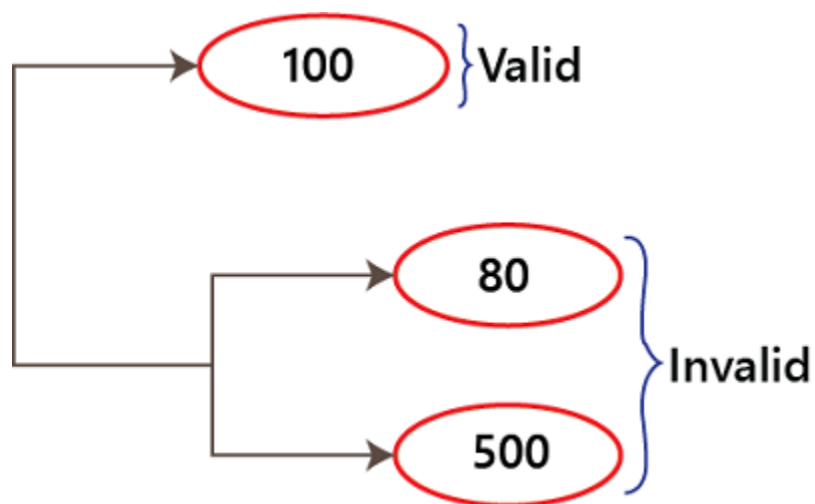
INVALID 1 Test case	INVALID 2 Test case	VALID 3 Test case	VALID
DIGITS >=11	DIGITS<=9	DIGITS = 10	DIGITS =10
93847262219	984543985	9991456234	9893451483

In both examples, we can see that there is a partition of two equally valid and invalid partitions, on applying valid value such as OTP of six digits in the first example and mobile number of 10 digits in the second example, both valid partitions behave same, i.e. redirected to the next page.

Condition1

If the requirement is a **range of values**, then derive the test case for **one valid** and **two invalid** inputs.

For example, the Amount of test field accepts a Range (100-400) of values:



Example 2

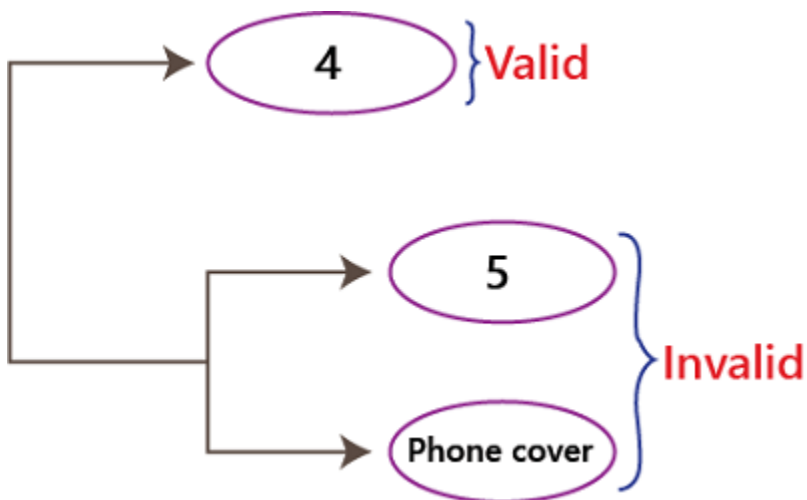
If we are doing online shopping, mobile phone product, and the different **Product ID -1,4,7,9**



The image shows a light blue rectangular box representing an online shopping interface. At the top, the text "ONLINE SHOPPING" is centered and underlined. Below this, on the left, is the label "Product ID". To the right of the label is a white rectangular input field. At the bottom of the box, there are two buttons: a green button with the word "SUBMIT" in green text, and a red button with the word "CANCEL" in red text.

Here, 1 → **phone covers** 4 → **earphones** 7 → **charger** 9 → **Screen guard**

And if we give the product id as **4**, it will be accepted, and it is one valid value, and if we provide the product id as **5 and phone cover**, it will not be accepted as per the requirement, and these are the two invalid values.



7. Error Guessing Technique

- Error guessing is a technique in which there is no specific method for identifying the error. It is based on the experience of the test analyst, where the tester uses the experience to guess the problematic areas of the software.
- It is a type of black box testing technique which does not have any defined structure to find the error.
- In this approach, every test engineer will derive the values or inputs based on their understanding or assumption of the requirements, and we do not follow any kind of rules to perform error guessing technique.
- The error guessing technique is dependent on the ability and product knowledge of the tester because a good test engineer knows where the bugs are most likely to be, which helps to save lots of time.

How does the error guessing technique be implemented?

- The implementation of this technique depends on the experience of the tester or analyst having prior experience with similar applications.
- It requires only well-experienced testers with quick error guessing technique.
- This technique is used to find errors that may not be easily captured by formal black box testing techniques, and that is the reason, it is done after all formal techniques.
- The scope of the error guessing technique entirely depends on the tester and type of experience in the previous testing involvements because it does not follow any method and guidelines.
- Test cases are prepared by the analyst to identify conditions.
- The conditions are prepared by identifying most error probable areas and then test cases are designed for them.
- The main purpose of this technique is to identify common errors at any level of testing.
- For Example:
 - o Enter blank space into the text fields.
 - o Null pointer exception.
 - o Enter invalid parameters.

- o Divide by zero.
 - o Use maximum limit of files to be uploaded.
 - o Check buttons without entering values.
-

Purpose of Error guessing

- To deal with all possible errors which cannot be identified informal testing.
- It must contain the all-inclusive sets of test cases without skipping any problematic areas.
- There are some factors that can be used by the examiner while using their experience.

For Example:

- o Historical learning
- o Review checklist
- o Risk reports of the software
- o Previous test results
- o Defects occurred in the past

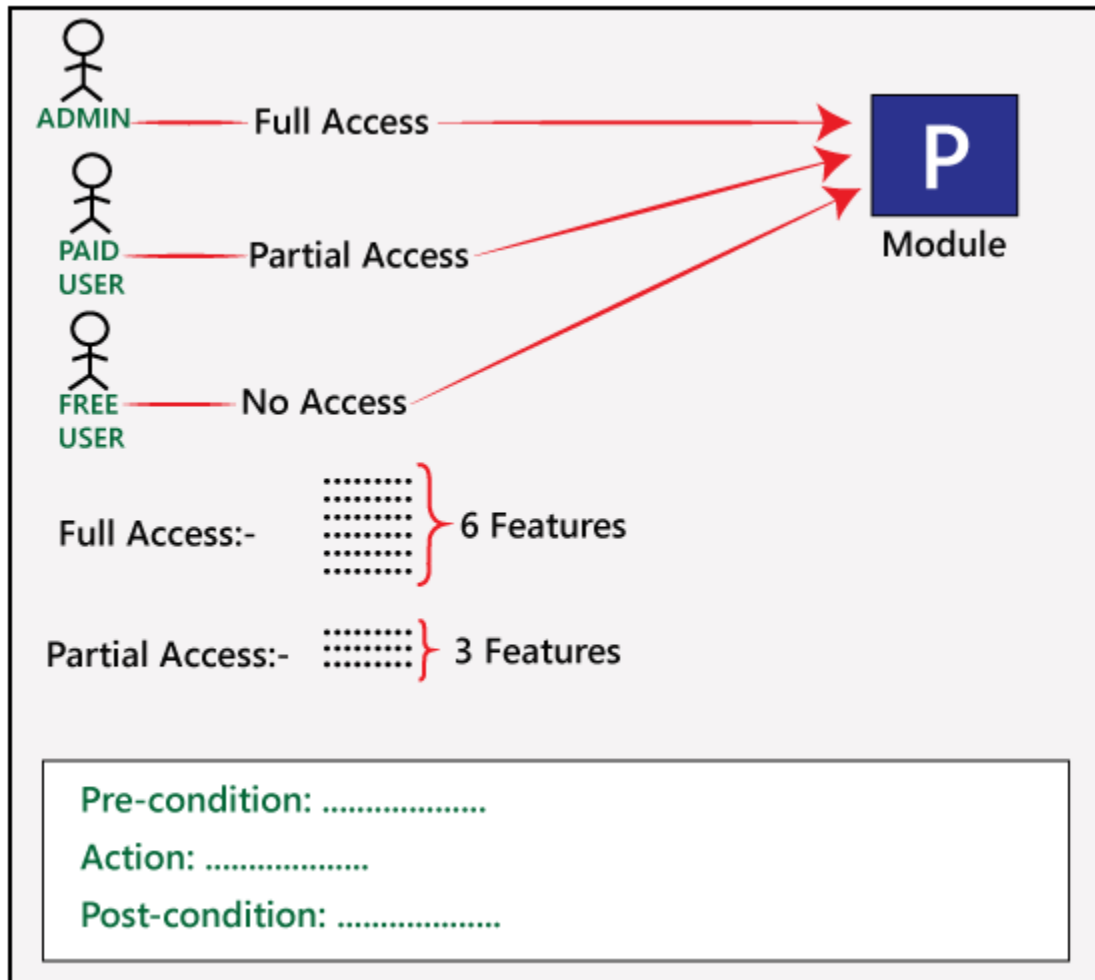
Example1

- A function of the application requires a mobile number which must be of 10 characters. Now, below are the techniques that can be applied to guess error in the mobile number field:
 - o What will be the result, if the entered character is other than a number?
 - o What will be the result, if entered characters are less than 10 digits?
 - o What will be the result, if the mobile field is left blank?
- After implementing these techniques, if the output is similar to the expected result, the function is considered to be bug-free, but if the output is not similar to the expected result, so it is sent to the development team to fix the defects.

8. Use Case Technique

- Use case Technique used to identify the test cases from the beginning to the end of the system as per the usage of the system.

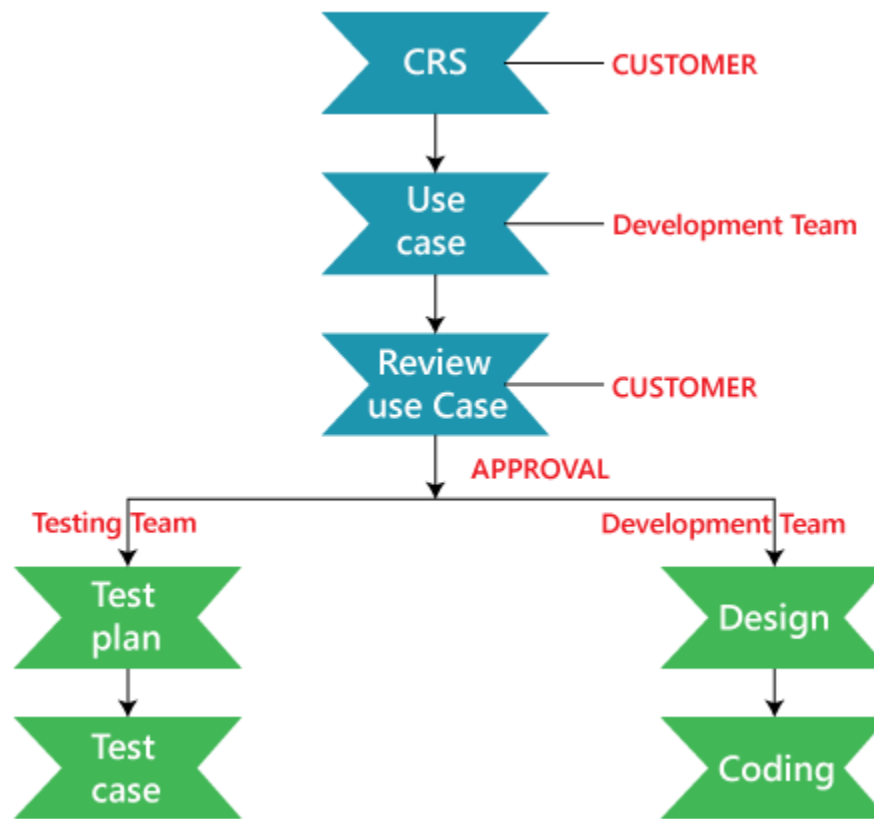
- The test team creates a test scenario that can exercise the entire software based on the functionality of each function from start to end.
- It is a graphic demonstration of business needs, which describe how the end-user will cooperate with the software or the application.
- The use cases provide us all the possible techniques of how the end-user uses the application.



- In the above image, a sample of a use case where we have a requirement related to the customer requirement specification (CRS).
- For module P of the software, we have six different features.
- Admin has access to all the six features, the Paid user has access to the three features and for the free user, there is no access provided to any of the features.

Who writes the use case?

The client provides the customer requirement specification for the application, then the development team will write the **use case** according to the CRS, and the use case is sent to the customer for their review.



If the client approves it, then the approved **use case** is sent to the development team for further design and coding process and these approved use case is also sent to the testing team, so they can start writing the test plan and start writing the test cases for the different features of the software.

Advantages

- o It is used to take the functional needs of the system.
- o Describe the connections between the user and its actions.

- o It starts from an elementary view where the system is created first and primarily used for its users.
 - o It is used to determine the complete analyses, which help us to achieve the complication
-

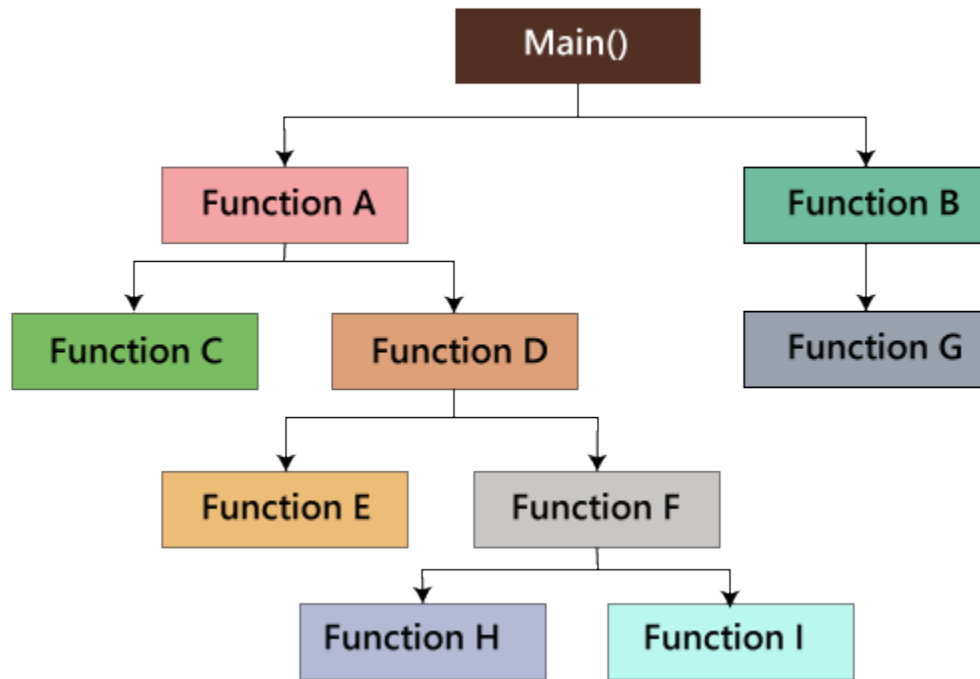
WHITE BOX TESTING

- The white box testing also known as glass box is testing, structural testing, clear box testing, open box testing and transparent box testing.
- It tests internal coding and infrastructure of a software focus on checking of predefined inputs against expected and desired outputs.
- It is based on inner workings of an application.
- In this type of testing programming skills are required to design test cases.
- The primary goal of white box testing is to focus on the flow of inputs and outputs through the software and strengthening the security of the software.
- The clear box or white box or transparent box name denote the ability to see through the software's outer shell into its inner workings.
- The developer will test every line of the code of the program.
- The developers perform the White-box testing and then send the application or the software to the testing team.
- They will perform the testing and verify the application along with the requirements and identify the bugs and sends it to the developer.
- The developer fixes the bugs and sends it to the testing team.
- The white box testing contains various tests, which are as follows:
 1. Path testing
 2. Loop testing
 3. Condition testing
 4. Testing based on the memory perspective
 5. Test performance of the program

1. Path Testing

- In the path testing, we will write the flow graphs and test all independent paths.

- Here writing the flow graph implies that flow graphs are representing the flow of the program and also show how every program is added with one another.



- Test all the independent paths, a path from main() to function G, first set the parameters and test if the program is correct in that particular path, and in the same way test all other paths and fix the bugs.

2. Loop Testing

- In the loop testing, we will test the loops such as while, for, and do-while, etc. and also check for ending condition if working correctly and if the size of the conditions is enough.

For example: we have one program where the developers have given about 50,000 loops.

```

{
while(50,000)
.....
.....
}

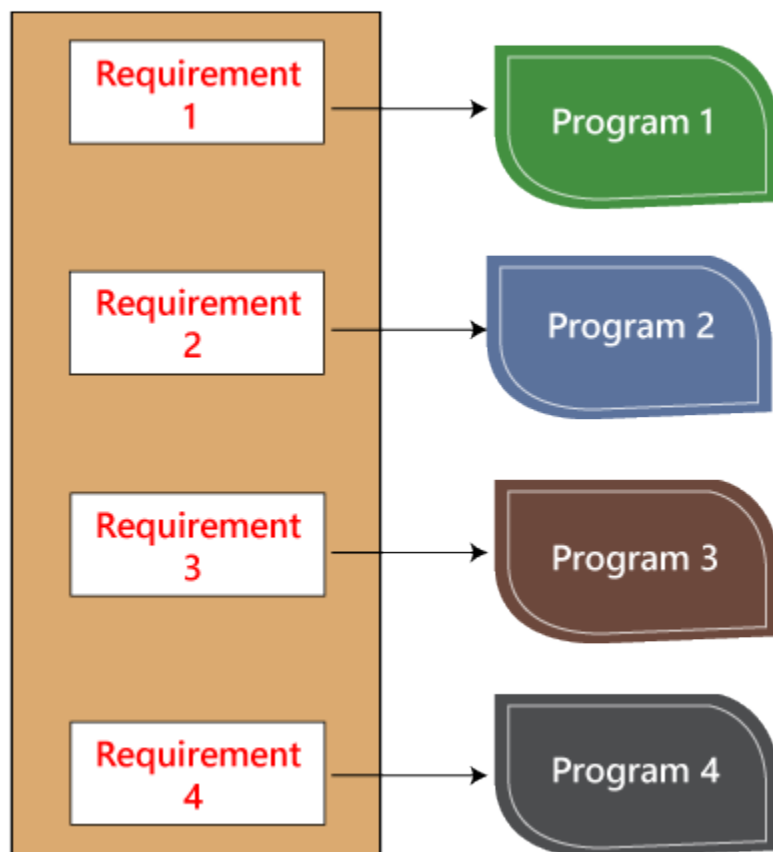
```

- We cannot test this program manually for all the 50,000 loops cycle.
- So we write a small program that helps for all 50,000 cycles that test P is written in the similar language as the source code program, and this is known as a Unit test. And it is written by the developers only.

Test P

```
{  
.....  
..... }
```

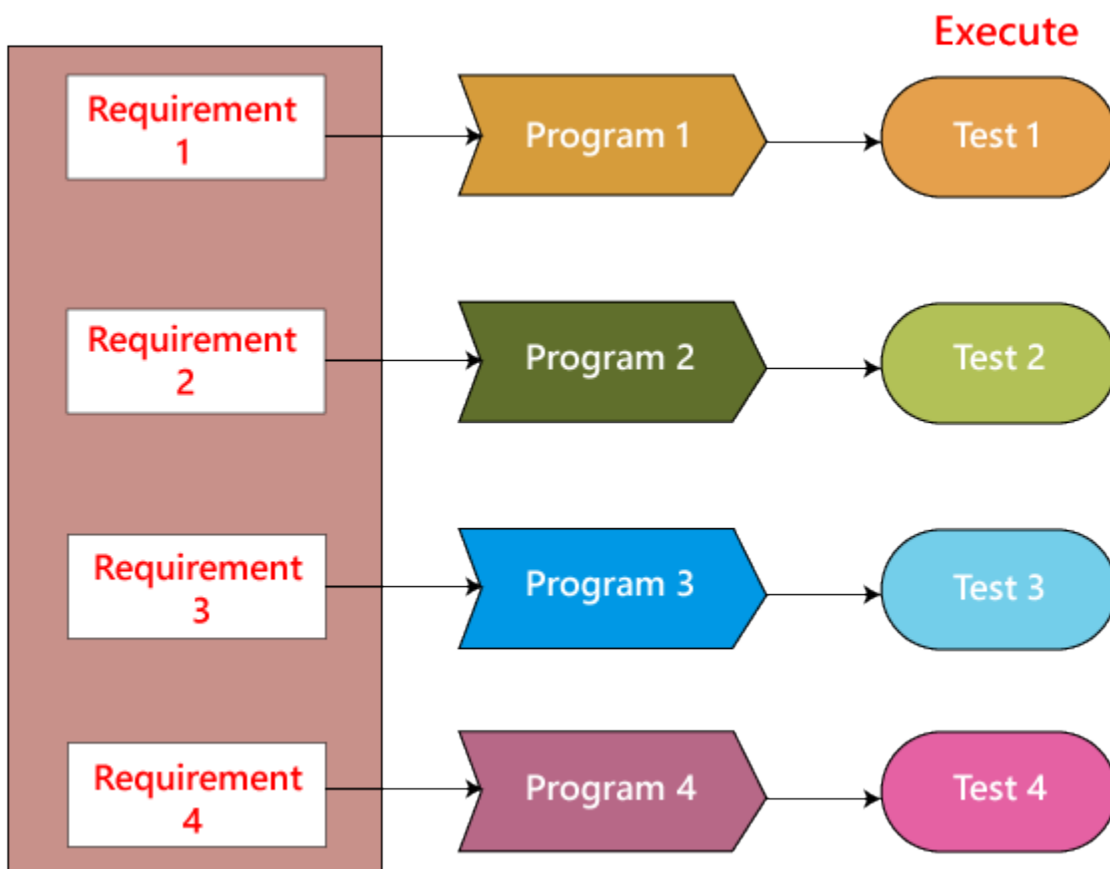
- We have various requirements such as 1, 2, 3, and 4. , then, the developer writes the programs such as program 1,2,3,4 for the parallel conditions. Here the application contains the 100s line of codes.



- The developer will do the white box testing, and they will test all the programs line by line of code to find the bug.
- If they found any bug in any of the programs, they will correct it.
- Again have to test the system then this process contains lots of time and effort and slows down the product release time.
- The another case, where the clients want to modify the requirements, then the developer will do the required changes and test all four program again, which take lots of time and efforts.

These issues can be resolved in the following ways:

- In this, we will write test for a similar program where the developer writes these test code in the related language as the source code.
- Then they execute these test code, which is also known as unit test programs.
- These test programs linked to the main program and implemented as programs.



- Therefore, if there is any requirement of modification or bug in the code, then the developer makes the adjustment both in the main program and the test program and then executes the test program.

3. Condition Testing

- In this, we will test all logical conditions for both **true** and **false** values; that is, we will verify for both **if** and **else** condition.

For example:

```

if(condition) - true
{
.....
.....
.....
}
else - false
{
.....
.....
.....
}

```

- The above program will work fine for both the conditions, which means that if the condition is accurate.

4. Testing based on the memory (size) perspective

The size of the code is increasing for the following reasons:

- **The reuse of code is not there:** let us take one example, where we have four programs of the same application, and the first ten lines of the program are similar. We can write these

ten lines as a discrete function, and it should be accessible by the above four programs as well. And also, if any bug is there, we can modify the line of code in the function rather than the entire code.

- **The developers use the logic** that might be modified. If one programmer writes code and the file size is up to 250kb, then another programmer could write a similar code using the different logic, and the file size is up to 100kb.
- **The developer declares so many functions and variables** that might never be used in any portion of the code. Therefore, the size of the program will increase.

For example,

```
int a=15;
int b=20;
String S= "Welcome";
....
.....
int p=b;
Create user()
{
.....
.....
200's line of code
}
```

- In the above code, we can see that the integer a has never been called anywhere in the program, and also the function Create user has never been called anywhere in the code. Therefore, it leads us to memory consumption.

5. Test the performance (Speed, response time) of the program

The application could be slow for the following reasons:

- When logic is used.
- For the conditional cases, we will use **or** & **and** adequately.
- Switch case, which means we cannot use **nested if**, instead of using a switch case.

The developer is performing white box testing, they understand that the code is running slow, or the performance of the program is also getting deliberate. And the developer cannot go manually over the program and verify which line of the code is slowing the program.

- To recover with this condition, we have a tool called **Rational Quantify**, which resolves these kinds of issues automatically.
- Once the entire code is ready, the rational quantify tool will go through the code and execute it.
-

TECHNIQUES USED IN WHITE BOX TESTING

1. Data Flow Testing
2. Control Flow Testing
3. Branch Testing
4. Statement Testing
5. Decision Testing

1. Data Flow Testing

- Data flow testing is used to analyze the flow of data in the program.
- It is the process of collecting information about how the variables flow the data in the program.
- Data flow testing is a group of testing strategies to examine the control flow of programs in order to explore the sequence of variables according to the sequence of events.
- It mainly focuses on the points at which values assigned to the variables and the point at which these values are used by concentrating on both points, data flow can be tested.
- Data flow testing uses the control flow graph to detect illogical things that can interrupt the flow of data.
- Anomalies in the flow of data are detected at the time of associations between values and variables due to:
 - If the variables are used without initialization.
 - If the initialized variables are not used at least once.

Example

1. read x;	
2. If(x>0)	(1, (2, t), x), (1, (2, f), x)
3. a= x+1	(1, 3, x)
4. if (x<=0) {	(1, (4, t), x), (1, (4, f), x)
5. if (x<1)	(1, (5, t), x), (1, (5, f), x)
6. x=x+1; (go to 5)	(1, 6, x)
else	
7. a=x+1	(1, 7, x)
8. print a;	(6, (5, f)x), (6, (5, t)x)
	(6, 6, x)
	(3, 8, a), (7, 8, a).

- In this code, we have a total 8 statements, and we will choose a path which covers all the 8 statements.
- We cannot cover all the statements in a single path because if statement 2 is true then statements 4, 5, 6, 7 not covered, and if statement 4 is true then statement 2 and 3 are not covered.
- So, we are taking two paths to cover all the statements.

1. x= 1

Path - 1, 2, 3, 8

- When we set value of x as 1 first it come on step 1 to read and assign the value of x (we took 1 in path) then come on statement 2 (x>0 (we took 2 in path)) which is true and it comes on statement 3 (a= x+1 (we took 3 in path)) at last it comes on statement 8 to print the value of x (output is 2).

2. Set x= -1

Path = 1, 2, 4, 5, 6, 5, 6, 5, 7, 8

- When we set the value of x as -1 then first, it comes on step 1 to read and assign the value of x , then come on step number 2 which is false because x is not greater than 0.

- Due to false condition, it will not come on statement 3 and directly jump on statement 4 and 4 is , then come on statement 5 , which is also true so it will come on statement 6 ($x=x+1$ and here x is incremented by 1.

$x=-1+1$
 $x=0$

- There is value of x become 0. Now it goes to statement 5 with value 0 and 0 is less than 1 so, it is true. Come on statement 6.

$x=x+1$
 $x= 0+1$
 $x=1$

- There x has become 1 and again goes to statement and now 1 is not less than 1 so, condition is false and it will come to else part and assign the value to a .
- At last, it come on statement 8 and print the value (Output is 2).

Associations

In associations all the definitions with all of its uses.

How to make associations in data flow testing?

1 read x;	
2 If($x>0$)	(1, (2, t), x), (1, (2, f), x)
3 a= x+1	(1, 3, x)
4 if ($x\leq 0$) {	(1, (4, t), x), (1, (4, f), x)
5 if ($x<1$)	(1, (5, t), x), (1, (5, f), x)
6 $x=x+1$; (go to 5)	(1, 6, x)
else	
7 a=x+1	(1, 7, x)
8 print a;	(6,(5, f)x), (6,(5,t)x)
	(6, 6, x)
	(3, 8, a), (7, 8, a).

- (1, (2, t), x), (1, (2, f), x)- This association is made with statement 1 (read x;) and statement 2 (If(x>0)) where x is defined at line number 1, and it is used at line number 2 so, x is the variable.
- Statement 2 is logical, and it can be true or false that's why the association is defined in two ways; one is (1, (2, t), x) for true and another is (1, (2, f), x) for false.
- (1, 3, x)- This association is made with statement 1 (read x;) and statement 3 (a= x+1) where x is defined in statement 1 and used in statement 3. It is a computation use.

2. Control Flow Testing

- Control flow testing is a testing technique that comes under white box testing.
- To determine the execution order of statements or instructions of the program through a control structure.
- The control structure of a program is used to develop a test case for the program.
- In this technique, a particular part of a large program is selected by the tester to set the testing path.
- It is mostly used in unit testing. Test cases represented by the control graph of the program.
- Control Flow Graph is formed from the node, edge, decision node, junction node to specify all possible execution path.
- Notations used for Control Flow Graph
 1. Node
 2. Edge
 3. Decision Node
 4. Junction node

1. Node

- Nodes in the control flow graph are used to create a path of procedures.
- It represents the sequence of procedures.
- The tester can determine the sequence of occurrence of procedures.

2. Edge

- Edge in control flow graph is used to link the direction of nodes.

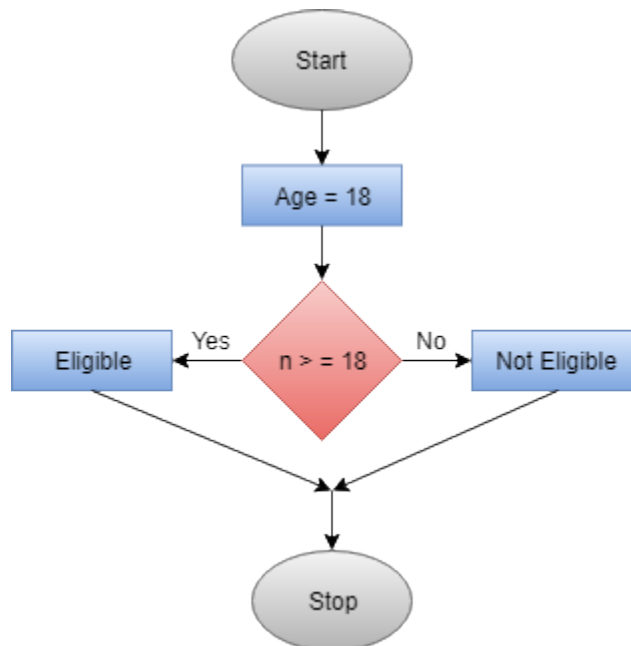
3. Decision node

- Decision node in the control flow graph is used to decide next node of procedure as per the value.

4. Junction node

- Junction node in control flow graph is the point where at least three links meet.

Diagram - control flow graph



- Test cases are designed through the flow graph of the programs to determine the execution path is correct or not.
- All nodes, junction, edges, and decision are the essential parts to design test cases.

3. Branch Coverage Testing

- Branch coverage technique is used to cover all branches of the control flow graph.
- It covers all the possible outcomes (true and false) of each condition of decision point at least once.
- It ensures that every branch of each decision point must be executed.

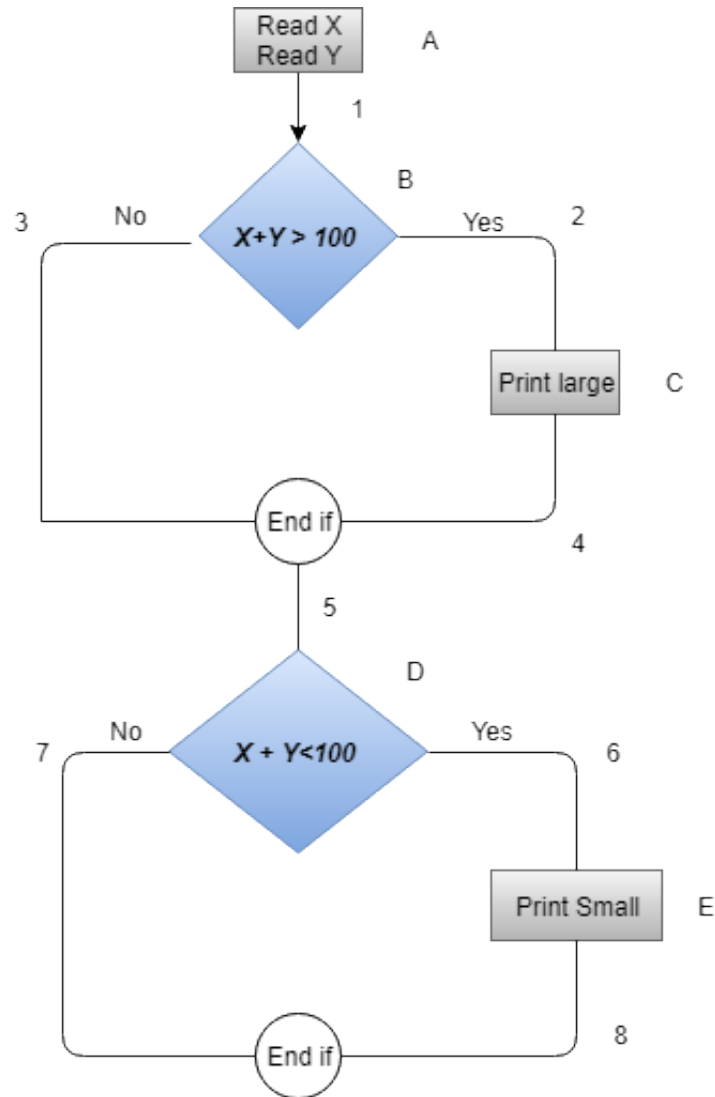
How to calculate Branch coverage?

- There are several methods to calculate Branch coverage, but path finding is the most common method.
- The number of paths of executed branches is used to calculate Branch coverage.
- Branch coverage technique can be used as the alternative of decision coverage.

Example

1. Read X
2. Read Y
3. IF $X+Y > 100$ THEN
4. Print "Large"
5. ENDIF
6. If $X + Y < 100$ THEN
7. Print "Small"
8. ENDIF

Control flow graph of code structure



- In the first case traversing through "Yes" decision, the path is **A1-B2-C4-D6-E8**, and the number of covered edges is 1, 2, 4, 5, 6 and 8 but edges 3 and 7 are not covered in this path.
- To cover these edges, to traverse through "No" decision.
- In the case of "No" decision the path is **A1-B3-5-D7**, and the number of covered edges is 3 and 7.

Path 1 - A1-B2-C4-D6-E8

Path 2 - A1-B3-5-D7

Branch Coverage (BC) = Number of paths

=2

Software Testing Life Cycle (STLC)

- The Software Testing Life Cycle (STLC) is a systematic approach to testing a software application to ensure that it meets the requirements and is free of defects.
- It is a process that follows a series of steps or phases, and each phase has specific objectives and deliverables.
- The STLC is used to ensure that the software is of high quality, reliable, and meets the needs of the end-users.
- The main goal of the STLC is to identify and document any defects or issues in the software application as early as possible in the development process.
- This allows for issues to be addressed and resolved before the software is released to the public.
- Stages of the STLC include Test Planning, Test Analysis, Test Design, Test Environment Setup, Test Execution, Test Closure, and Defect Retesting.
- Each of these stages includes specific activities and deliverables that help to ensure that the software is thoroughly tested.

Phases of STLC

1. Requirement Analysis: Requirement Analysis is the first step of the Software Testing Life Cycle (STLC). In this phase quality assurance team understands the requirements like what is to be tested. If anything is missing or not understandable then the quality assurance team meets with the stakeholders to better understand the detailed knowledge of requirements.

The activities that take place during the Requirement Analysis stage include:

- Reviewing the software requirements document (SRD) and other related documents
- Interviewing stakeholders to gather additional information
- Identifying any ambiguities or inconsistencies in the requirements
- Identifying any missing or incomplete requirements
- Identifying any potential risks or issues that may impact the testing process

Creating a requirement traceability matrix (RTM) to map requirements to test cases.

The testing team should have a clear understanding of the software requirements and should have identified any potential issues that may impact the testing process.

This will help to ensure that the testing process is focused on the most important areas of the software and that the testing team is able to deliver high-quality results.

2. Test Planning: Test Planning is the most efficient phase of the software testing life cycle where all testing plans are defined. In this phase manager of the testing, team calculates the estimated effort and cost for the testing work. This phase gets started once the requirement-gathering phase is completed.

The activities that take place during the Test Planning stage include:

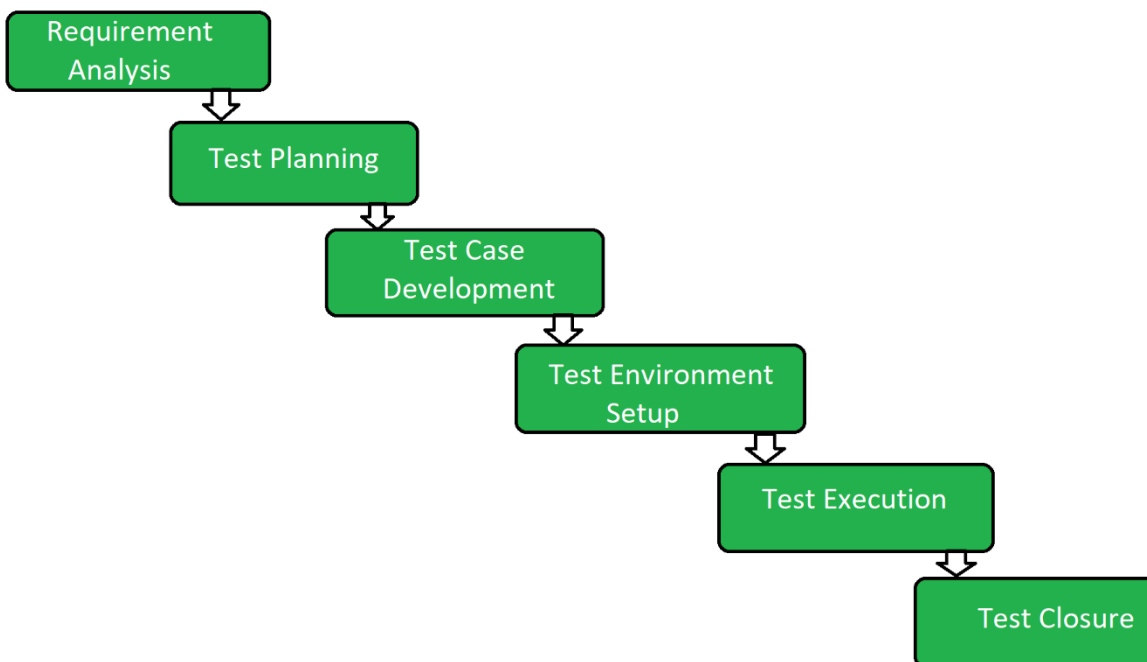
- Identifying the testing objectives and scope
- Developing a test strategy: selecting the testing methods and techniques that will be used
- Identifying the testing environment and resources needed
- Identifying the test cases that will be executed and the test data that will be used
- Estimating the time and cost required for testing
- Identifying the test deliverables and milestones
- Assigning roles and responsibilities to the testing team
- Reviewing and approving the test plan

3. Test Case Development: The test case development phase gets started once the test planning phase is completed. In this phase testing team notes down the detailed test cases. The testing team also prepares the required test data for the testing. When the test cases are prepared then they are reviewed by the quality assurance team.

The activities that take place during the Test Case Development stage include:

- Identifying the test cases that will be developed
- Writing test cases that are clear, concise, and easy to understand
- Creating test data and test scenarios that will be used in the test cases
- Identifying the expected results for each test case
- Reviewing and validating the test cases
- Updating the requirement traceability matrix (RTM) to map requirements to test cases

Software Testing Lifecycle



4. Test Environment Setup:

Test environment setup is a vital part of the STLC.

Basically, the test environment decides the conditions on which software is tested.

In this process, the testing team is not involved either the developer or the customer creates the testing environment.

5. Test Execution:

After the test case development and test environment setup test execution phase gets started.

In this phase testing team starts executing test cases based on prepared test cases in the earlier step.

The activities that take place during the test execution stage of the Software Testing Life Cycle (STLC) include:

- **Test execution:** The test cases and scripts created in the test design stage are run against the software application to identify any defects or issues.
- **Defect logging:** Any defects or issues that are found during test execution are logged in a defect tracking system, along with details such as the severity, priority, and description of the issue.
- **Test data preparation:** Test data is prepared and loaded into the system for test execution
- **Test environment setup:** The necessary hardware, software, and network configurations are set up for test execution
- **Test execution:** The test cases and scripts are run, and the results are collected and analyzed.
- **Test result analysis:** The results of the test execution are analyzed to determine the software's performance and identify any defects or issues.
- **Defect retesting:** Any defects that are identified during test execution are retested

to ensure that they have been fixed correctly.

- **Test Reporting:** Test results are documented and reported to the relevant stakeholders.

It is important to note that test execution is an iterative process and may need to be repeated multiple times until all identified defects are fixed and the software is deemed fit for release.

6. Test Closure: Test closure is the final stage of the Software Testing Life Cycle (STLC) where all testing-related activities are completed and documented. The main objective of the test closure stage is to ensure that all testing-related activities have been completed and that the software is ready for release.

At the end of the test closure stage, the testing team should have a clear understanding of the software's quality and reliability, and any defects or issues that were identified during testing should have been resolved.

The test closure stage also includes documenting the testing process and any lessons learned so that they can be used to improve future testing processes.

Test closure is the final stage of the Software Testing Life Cycle (STLC) where all testing-related activities are completed and documented.

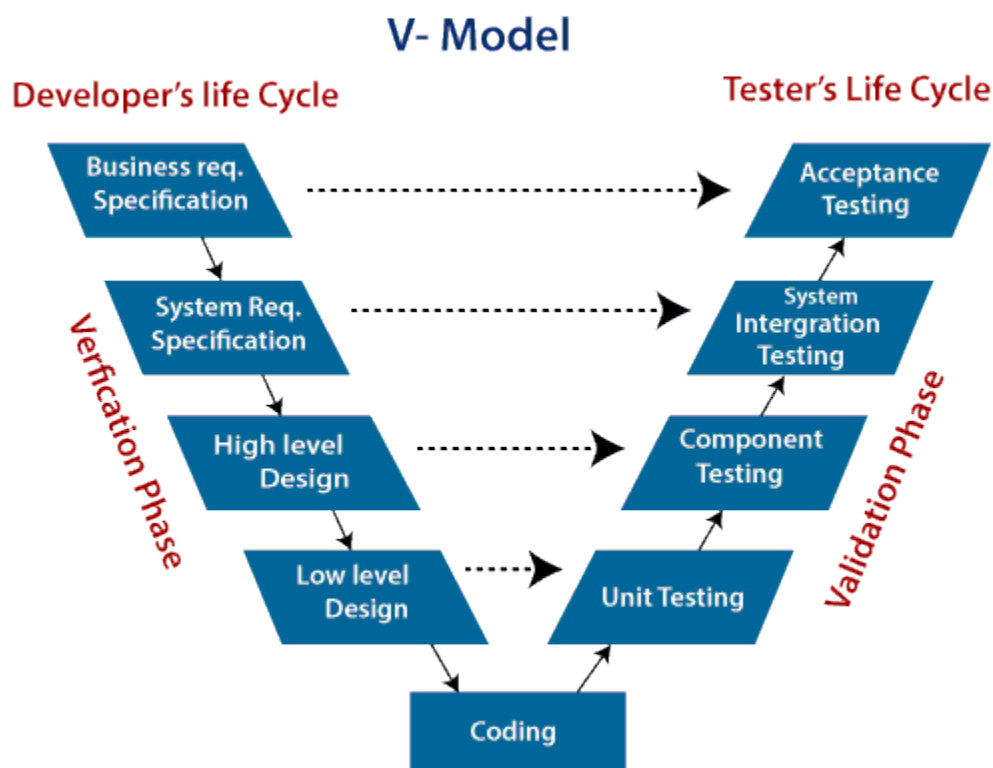
The main activities that take place during the test closure stage include:

- **Test summary report:** A report is created that summarizes the overall testing process, including the number of test cases executed, the number of defects found, and the overall pass/fail rate.
- **Defect tracking:** All defects that were identified during testing are tracked and managed until they are resolved.
- **Test environment clean-up:** The test environment is cleaned up, and all test data and test artifacts are archived.
- **Test closure report:** A report is created that documents all the testing-related activities that took place, including the testing objectives, scope, schedule, and resources used.

- **Knowledge transfer:** Knowledge about the software and testing process is shared with the rest of the team and any stakeholders who may need to maintain or support the software in the future.
- **Feedback and improvements:** Feedback from the testing process is collected and used to improve future testing processes

V-Model

V-Model also referred to as the Verification and Validation Model. In this, each phase of SDLC must complete before the next phase starts. It follows a sequential design process same as the waterfall model. Testing of the device is planned in parallel with a corresponding stage of development.



Verification:

It involves a static analysis method (review) done without executing code. It is the

process of evaluation of the product development process to find whether specified requirements meet.

Validation:

It involves dynamic analysis method (functional, non-functional), testing is done by executing code. Validation is the process to classify the software after the completion of the development process to determine whether the software meets the customer expectations and requirements.

There are the various phases of Verification Phase of V-model:

1. **Business requirement analysis:** This is the first step where product requirements understood from the customer's side. This phase contains detailed communication to understand customer's expectations and exact requirements.
2. **System Design:** In this stage system engineers analyze and interpret the business of the proposed system by studying the user requirements document.
3. **Architecture Design:** The baseline in selecting the architecture is that it should understand all which typically consists of the list of modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, technology detail, etc.
4. **Module Design:** In the module design phase, the system breaks down into small modules. The detailed design of the modules is specified, which is known as Low-Level Design
5. **Coding Phase:** After designing, the coding phase is started. Based on the requirements, a suitable programming language is decided.

There are the various phases of Validation Phase of V-model:

1. **Unit Testing:** In the V-Model, Unit Test Plans (UTPs) are developed during the module design phase. These UTPs are executed to eliminate errors at code level

or unit level. Unit testing verifies that the smallest entity can function correctly when isolated from the rest of the codes/ units.

2. **Integration Testing:** Integration Test Plans are developed during the Architectural Design Phase. These tests verify that groups created and tested independently can coexist and communicate among themselves.
3. **System Testing:** System Tests Plans are developed during System Design Phase. System Test ensures that expectations from an application developer are met.
4. **Acceptance Testing:** Acceptance testing is related to the business requirement analysis part. It includes testing the software product in user level.

Advantage:

1. Easy to Understand.
2. Testing Methods like planning, test designing happens well before coding.
3. Saves a lot of time.
4. Avoids the downward flow of the defects.
5. Works well for small plans where requirements are easily understood.

Disadvantage:

1. Very rigid and least flexible.
2. Not a good for a complex project.
3. Software is developed during the implementation stage, so no early prototypes of the software are produced.
4. If any changes happen in the midway, then the test documents along with the required documents, has to be updated.

Program Correctness

The best practices to ensure software correctness are listed below –

- The problem statement or the use case for the software should be clear and unambiguous.
- Before actually implementing the code, there should be a complete architecture, and design for the software.
- Testing should be done in parallel to the development of the software.

Software Reliability & Safety

Reliability focuses on a system's ability to perform its intended function consistently over time, while safety focuses on preventing potential harm or damage caused by the software's operation.

Software Failures

- To analyze software reliability and safety it is important to understand the meaning of a failure due to software.
- Software does not exhibit the random or wear out related failure behavior.
- Software will always function in the same way as long as the same input and computer states are present.
- Software can cause system failures either because of design errors or implementation errors.
- Design errors are often caused by wrong assumptions about system operation.

Example:

Input A is always followed by input B. Typical implementation errors are caused by confusing symbols, such as 'g' instead of 'G'. Software faults can only cause failures if the fault is encountered during usage. Therefore faults existing in frequently used code will cause failures more often than faults residing in rarely used code but the later can be equally serious. In mission or safety critical applications it is particularly important to

review and test rarely used code.

Software Reliability

- Software reliability involves much more than analyzing test results, estimating remaining faults, and modeling future failure probabilities.
- Although in most organizations software test is no longer an afterthought, management is almost always surprised by the cost and schedule requirements of the test program, and it is often downgraded in favor of design activities.
- Often adding a new feature will seem more beneficial than performing a complete test on existing features.

A good software reliability program, introduced early in the development cycle, will mitigate these problems by:

- Preparing program management in advance for the testing effort and allowing them to plan both schedule and budget to cover the required testing.
- Continuous review of requirements throughout the life cycle, particularly for handling of exception conditions. If requirements are incomplete there will be no testing of the exception conditions.
- Offering management a quantitative assessment of the dependence of reliability metrics (software/system availability; software/system outages per day, etc.) on the effort (time and cost) allotted to testing.
- Providing the most efficient test plan targeted to bringing the product to market in the shortest time subject to the reliability requirements imposed by the customer or market expectations.
- Performing continuous quantitative assessment of software/system reliability and the effort/cost required to improve them by a specified amount.

Software Safety

As systems and products become more and more dependent on software components it is no longer realistic to develop a system safety program that does not include the software

elements.

Safety-critical software

Safety-critical software is very different from both non-critical software and safety-critical hardware. The difference lies in the massive testing program that such software undergoes.

What are "software failure modes"?

- Software, especially in critical systems, tends to fail where least expected.
- We are usually extremely good at setting up test plans for the main line code of the program, and these sections usually do run flawlessly.
- Software does not "break" but it must be able to deal with "broken" input and conditions, which often cause the "software failures".
- The task of dealing with abnormal/anomalous conditions and inputs is handled by the exception code dispersed throughout the program.
- Setting up a test plan and exhaustive test cases for the exception code is by definition difficult and somewhat subjective.

Software Testing – Bug vs Defect vs Error vs Fault vs Failure

- Software Testing defines a set of procedures and methods that check whether the actual software product matches with expected requirements, thereby ensuring that the product is Defect free.
- There are a set of procedures that needs to be in mind while testing the software manually or by using automated procedures.
- The main purpose of software testing is to identify errors, deficiencies, or missing requirements with respect to actual requirements.
- Software Testing is Important because if there are any bugs or errors in the software, they can be identified early and can be solved before the delivery of the software product.

What is a Bug?

A bug refers to defects which means that the software product or the application is not working as per the adhered requirements set. When we have any type of logical error, it causes our code to break, which results in a bug. It is now that the Automation/ Manual Test Engineers describe this situation as a bug.

What is a Defect?

A defect refers to a situation when the application is not working as per the requirement and the actual and expected result of the application or software are not in sync with each other.

What is an Error?

Error is a situation that happens when the Development team or the developer fails to understand a requirement definition and hence that misunderstanding gets translated into buggy code. This situation is referred to as an Error and is mainly a term coined by the developers.

What is a Fault?

Fault occurs in software which means that the logic was not incorporated to handle the errors in the application. This is an undesirable situation, but it mainly happens due to invalid documented steps or a lack of data definitions.

What is a Failure?

Failure is the accumulation of several defects that ultimately lead to Software failure and results in the loss of information in critical modules thereby making the system unresponsive. Generally, such situations happen very rarely because before releasing a product all possible scenarios and test cases for the code are simulated. Failure is detected by end-users once they face a particular issue in the software.

- Failure can happen due to human errors or can also be caused intentionally in the system by an individual.
- It is a term that comes after the production stage of the software.
- It can be identified in the application when the defective part is executed.

A simple diagram depicting Bug vs Defect vs Fault vs Failure:



Bug vs Defect vs Error vs Fault vs Failure

Basis	Bug	Defect	Fault	Error	Failure
Definit ion	A bug refers to defects which means that the software product or the	A Defect is a deviation between the actual and expected output	A Fault is a state that causes the software to fail and therefore it does not achieve its necessary	An Error is a mistake made in the code due to which compilation or execution fails,	Failure is the accumulation of several defects that ultimately lead to Software

	application is not working as per the adhered requirements set		function.		failure and results in the loss of information in critical modules thereby making the system unresponsive.
Raised by	Test Engineers	The defect is identified by The Testers And is resolved by developers in the development phase of SDLC.	Human mistakes lead to fault.	Developers and automation test engineers	The failure is found by the test engineer during the develop ment cycle of SDLC
Differ ent types	- Logical bugs - Algorithmic bugs - Resource	Defects are classified as follows: Based on Priority: High	- Business Logic Faults - Functional and	- Syntactic Error - UI screen error - Error handling	NA

	e bugs	• Medium • Low Based on Severity: • Critical • Major • Minor • Trivial	Logical Faults • Graphical User Interface (GUI) Faults • Performance Faults • Security Faults • Hardware Faults	error • Flow control error • Calculation error • Hardware error	
Reasons behind	• Missing Logic • Erroneous Logic • Redundant codes	• Receiving & providing incorrect input • Coding/Logical Error leading to the	• Wrong design of the data definition on processes. • An irregular	• Error in code. • Inability to compile/execute a program • Ambiguity in code logic • Misunderst	• Environment variables • System Errors • Human Error

		breakdown of software	arity in Logic or gaps in the software leads to the non-functioning of the software.	anding of requirements - Faulty design and architecture - Logical error	
Way to prevent the reasons	- Implementing Test-driven development. - Adjusting enhanced development practices and evaluation	- Implementing Out-of-the-box programming methods. - Proper usage of primary and correct software coding practices.	- Peer review of the Test documents and requirements. - Verifying the correctness of software	- Conduct peer reviews and code-reviews - Need for validation of bug fixes and enhancing the overall quality of the software.	- Confirmation of Re-testing the process end to end, - Carefully review the requirements as well

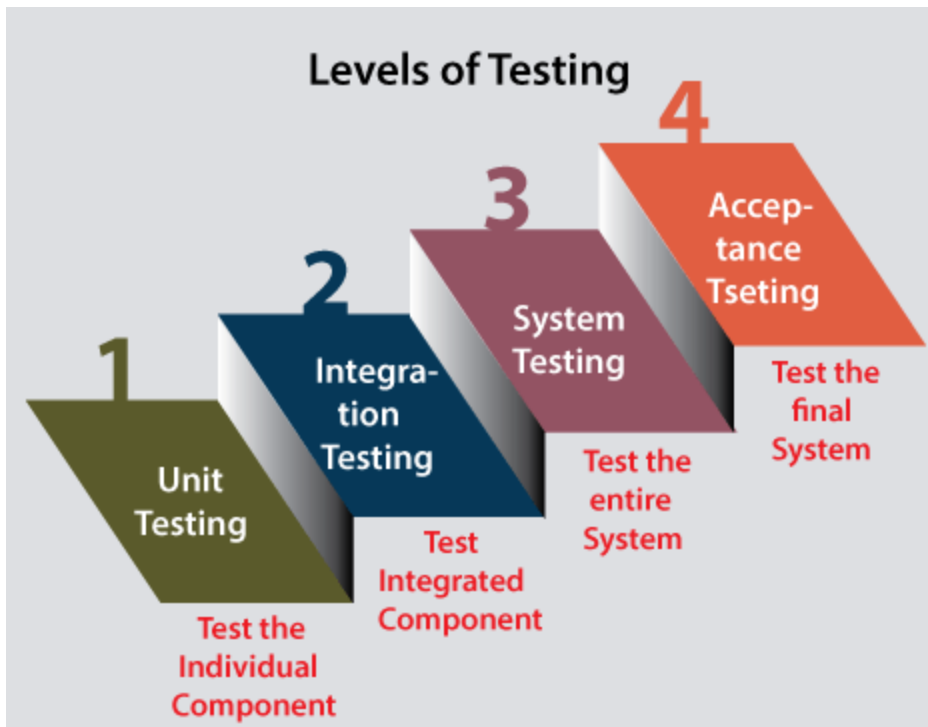
	on of cleanlin ess of the code.		design and coding.		as the specific ations. • Categor izing and evaluati ng the errors and issues.
--	---	--	--------------------------	--	---

Stages of Software Testing

The levels of software testing involve the different methodologies, which can be used while we are performing the software testing.

In software testing, we have four different levels of testing, which are as discussed below:

1. **Unit Testing**
2. **Integration Testing**
3. **System Testing**
4. **Acceptance Testing**



Unit Testing

Unit testing is the first level of software testing, which is used to test if software modules are satisfying the given requirement or not.

The first level of testing involves **analyzing each unit or an individual component** of the software application.

Unit testing is also the first level of **functional testing**. The primary purpose of executing unit testing is to validate unit components with their performance.

A unit component is an individual function or regulation of the application, or we can say that it is the smallest testable part of the software. The reason of performing the unit testing is to test the correctness of inaccessible code.

Unit testing will help the test engineer and developers in order to understand the base of

code that makes them able to change defect causing code quickly. The developers implement the unit.

Some crucial reasons are listed below:

- o Unit testing helps tester and developers to understand the base of code that makes them able to change defect causing code quickly.
 - o Unit testing helps in the documentation.
 - o Unit testing fixes defects very early in the development phase that's why there is a possibility to occur a smaller number of defects in upcoming testing levels.
 - o It helps with code reusability by migrating code and test cases.
-

Example of Unit testing

Amount transfer

Below are the application access details, which is given by the customer

- o URL → login Page
- o Username/password/OK → home page
- o To reach Amount transfer module follow the below

Loans → sales → Amount transfer

While performing unit testing, we should follow some rules, which are as follows:

- o To start unit testing, at least we should have one module.
 - o Test for positive values
 - o Test for negative values
 - o No over testing
 - o No assumption required
-

When we feel that the **maximum test coverage** is achieved, we will stop the testing.

Now, we will start performing the unit testing on the different components such as

- o **From account number(FAN)**
- o **To account number(TAN)**
- o **Amount**
- o **Transfer**
- o **Cancel**

Values	Description
1234	accept
4311	Error message→ account valid or not
blank	Error message→ enter some values
5 digit/ 3 digit	Error message→ accept only 4 digit
Alphanumeric	Error message → accept only digit
Blocked account no	Error message
Copy and paste the value	Error message→ type the value
Same as FAN and TAN	Error message

For the TAN component

- o Provide the values just like we did in **From account number** (FAN) components

For Amount component

- o Provide the values just like we did in FAN and TAN components.

For Transfer component

- o Enter valid FAN value
- o Enter valid TAN value
- o Enter the correct value of Amount
- o Click on the Transfer button → amount transfer successfully(confirmation message)

For Cancel Component

- o Enter the values of FAN, TAN, and amount.
 - o Click on the Cancel button → all data should be cleared.
-

Unit Testing Tools

We have various types of unit testing tools available in the market, which are as follows:

- o NUnit
 - o JUnit
 - o PHPunit
 - o Parasoft Jtest
 - o EMMA
-

Unit Testing Techniques:

Unit testing uses all white box testing techniques as it uses the code of software application:

- o Data flow Testing
 - o Control Flow Testing
 - o Branch Coverage Testing
 - o Statement Coverage Testing
 - o Decision Coverage Testing
-

How to achieve the best result via Unit testing?

Unit testing can give best results without getting confused and increase complexity by following the steps listed below:

- o Test cases must be independent because if there is any change or enhancement in requirement, the test cases will not be affected.
 - o Naming conventions for unit test cases must be clear and consistent.
 - o During unit testing, the identified bugs must be fixed before jump on next phase of the SDLC.
 - o Only one code should be tested at one time.
 - o Adopt test cases with the writing of the code, if not doing so, the number of execution paths will be increased.
 - o If there are changes in the code of any module, ensure the corresponding unit test is available or not for that module.
-

Integration Testing

The second level of software testing is the **integration testing**. The integration testing

process comes after **unit testing**.

It is mainly used to test the **data flow from one module or component to other modules**.

In integration testing, the **test engineer** tests the units or separate components or modules of the software in a group.

The primary purpose of executing the integration testing is to identify the defects at the interaction between integrated components or units.

When each component or module works separately, we need to check the data flow between the dependent modules, and this process is known as **integration testing**.

Integration testing aims to evaluate the accuracy of communication among all the modules.

Example of integration testing

Let us assume that we have a **Gmail** application where we perform the integration testing.

First, we will do **functional testing on the login page**, which includes the various components such as **username, password, submit, and cancel** button. Then only we can perform integration testing.

The different integration scenarios are as follows:

Compose Mail

Inbox
Compose mail
Sent Items
Trash
Spam
Contact
Folders
Logout

To

From

Subject

☐ Save To Draft
☐ Add To Contact

SEND CANCEL

Scenario 1:

- o First, we login as **P** users and click on the **Compose** mail and performing the functional testing for the specific components.
- o Now we click on the **Send** and also check for **Save Drafts**.
- o After that, we send a **mail** to **Q** and verify in the **Sent Items** folder of P to check if the send mail is there.
- o Now, we will **log out** as P and login as Q and move to the **Inbox** and verify that if the mail has reached.

Scenario 2:

We also perform the integration testing on **Spam** folders. If the particular contact has been marked as spam, then any mail sent by that user should go to the spam folder and not in the inbox.

As we can see in the below image, we will perform the **functional testing** for all the **text**

fields and every feature. Then we will perform **integration testing** for the related functions. We first test the **add user, list of users, delete user, edit user,** and then **search user.**

The screenshot shows a web application interface with a light green background. On the left is a white sidebar menu with the following items: 'Add User', 'Delete User', 'List User', 'Edit User', 'Product Sales', 'Product Purchases', 'Search Users', and 'Help'. The main content area is titled 'Add Users' in bold black text. It contains several form fields with red labels: 'User Name', 'Password', 'Designation' (a dropdown menu showing 'Team Lead' and 'Manager'), 'Email', 'Telephone', and 'Address'. At the bottom of the form are two buttons: 'Submit' and 'Cancel'.

Integration Testing Techniques

Any testing technique (Blackbox, Whitebox, and Greybox) can be used for Integration Testing; some are listed below:

Black Box Testing

- o State Transition technique
 - o Decision Table Technique
 - o Boundary Value Analysis
 - o All-pairs Testing
 - o Cause and Effect Graph
 - o Equivalence Partitioning
 - o Error Guessing
-

White Box Testing

- o Data flow testing
 - o Control Flow Testing
 - o Branch Coverage Testing
 - o Decision Coverage Testing
-

System Testing

The third level of software testing is **system testing**, which is used to test the software's functional and non-functional requirements.

It is **end-to-end testing** where the testing environment is parallel to the production environment. In the third level of software testing, **we will test the application as a whole system.**

To check the end-to-end flow of an application or the software as a user is known as **System testing.**

In system testing, we will go through all the necessary modules of an application and test if the end features or the end business works fine, and test the product as a complete system.

System testing is a sequence of different types of tests to implement and examine the entire working of an integrated software computer system against requirements.

Example of System testing

Suppose we open an application, let say **www.rediff.com**, and there we can see that an advertisement is displayed on the top of the homepage, and it remains there for a few seconds before it disappears. These types of Ads are done by the Advertisement

Management System (AMS). Now, we will perform system testing for this type of field.

The below application works in the following manner:

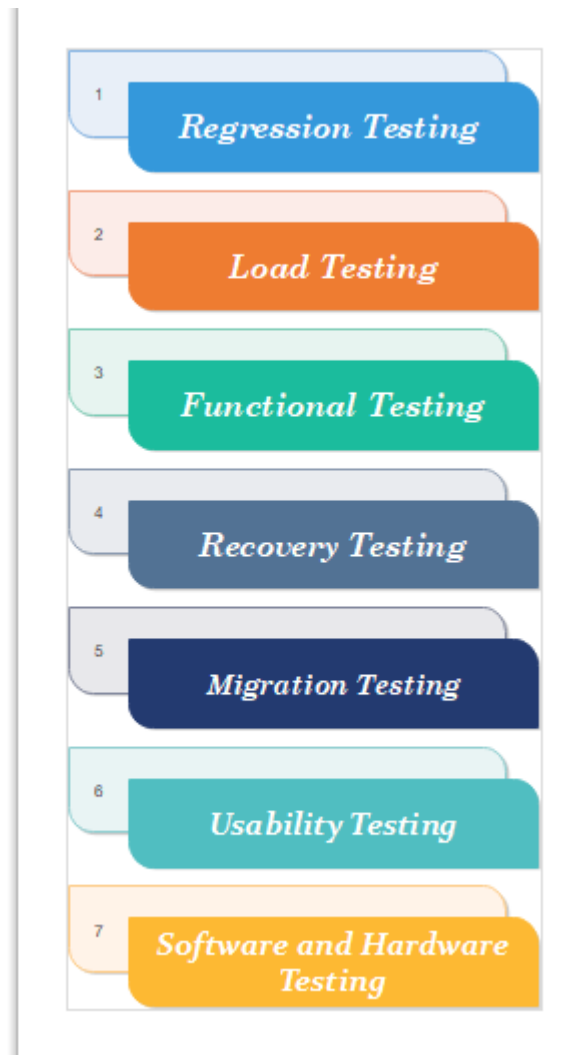
- o Let's say that Amazon wants to display a promotion ad on January 26 at precisely 10:00 AM on the Rediff's home page for the country India.
- o Then, the sales manager logs into the website and creates a request for an advertisement dated for the above day.
- o He/she attaches a file that likely an image files or the video file of the AD and applies.
- o The next day, the AMS manager of Rediffmail login into the application and verifies the awaiting Ad request.
- o The AMS manager will check those Amazons ad requests are pending, and then he/she will check if the space is available for the particular date and time.
- o If space is there, then he/she evaluate the cost of putting up the Ad at 15\$ per second, and the overall Ad cost for 10 seconds is approximate 150\$.
- o The AMS manager clicks on the payment request and sends the estimated value along with the request for payment to the Amazon manager.
- o Then the amazon manager login into the Ad status and confirms the payment request, and he/she makes the payment as per all the details and clicks on the **Submit** and **Pay**
- o As soon as Rediff's AMs manager gets the amount, he/she will set up the Advertisement for the specific date and time on the Rediffmail's home page.
- o **Scenario1:** The first test is the general scenario, as we discussed above. The test engineer will do the system testing for the underlying situation where the Amazon manager creates a request for the Ad and that Ad is used at a particular date and time.
- o **Scenario2:** Suppose the Amazon manager feels that the AD space is too expensive and cancels the request. At the same time, the Flipkart requests the Ad space on January 26 at 10:00 AM. Then the request of Amazon has been canceled. Therefore, Flipkart's promotion ad must be arranged on January 26 at

10 AM.

- o After all, the request and payment have been made. Now, if Amazon changes their mind and they feel that they are ready to make payment for January 26 at 10 AM, which should be given because Flipkart has already used that space. Hence, another calendar must open up for Amazon to make their booking.
- o **Scenario3:** in this, first, we login as AMS manger, then click on Set Price page and set the price for AD space on logout page to 10\$ per second.
- o Then login as Amazon manager and select the date and time to put up and Ad on the logout page. And the payment should be 100\$ for 10 seconds of an Ad on Rediffmail logout page.

Types of System Testing

System testing is divided into more than 50 types, but software testing companies typically use some of them. These are listed below:



Acceptance Testing

The **last and fourth level** of software testing is **acceptance testing**, which is used to

evaluate whether a specification or the requirements are met as per its delivery.

The software has passed through three testing levels (**Unit Testing, Integration Testing, and System Testing**). Some minor errors can still be identified when the end-user uses the system in the actual scenario.

Acceptance testing is the **squeezing of all the testing processes that are previously done**. The acceptance testing is also known as **User acceptance testing (UAT)** and is done by the customer before accepting the final product.

TEST PLAN – SOFTWARE TESTING

Software testing is important to make sure applications work properly and meet user needs. A clear and detailed test plan is the foundation of successful testing, guiding everything from creating test cases to fixing issues.

What is a Test Plan?

A test plan is a document that consists of all future testing-related activities. It is prepared at the project level and in general, it defines work products to be tested, how they will be tested, and test type distribution among the testers. Before starting testing there will be a test manager who will be preparing a test plan.

The test plan serves as the blueprint that changes according to the progressions in the project and stays current at all times.

It serves as a base for conducting testing activities and coordinating activities among a QA team.

It is shared with Business Analysts, Project Managers, and anyone associated with the project.

Types of Test Plan

1. Master Test Plan

A Master Test Plan is a high-level document that outlines the overall testing strategy, scope, and phases for a project. It provides a roadmap for the entire testing process and shows how different levels of testing connect with each other. This plan ensures that all testing activities are aligned and helps keep the project on track.

2. Specific Test Plan

A Specific Test Plan focuses on particular types of testing, like load testing, performance testing, or security testing. It provides detailed instructions and guidelines for carrying out these specialized tests, ensuring that each area is tested thoroughly.

3. Analytical Test Strategies

Analytical Test Strategies are based on specific factors like project requirements, specifications, or risks. For example:

User Acceptance Testing is based on user requirements to make sure the software meets the needs of the end-users.

System Testing is based on the technical specifications to check if each part of the system functions correctly.

How to create a Test Plan?

1. Analyze the product

This phase focuses on analyzing the product, Interviewing clients, designers, and developers, and performing a product walkthrough. This stage focuses on answering the following questions:

What is the primary objective of the product?

Who will use the product?

What are the hardware and software specifications of the product?

How does the product work?

2. Design the test strategy

The test strategy document is prepared by the manager and details the following information:

Scope of testing which means the components that will be tested and the ones that will be skipped.

Type of testing which means different types of tests that will be used in the project.

Risks and issues that will list all the possible risks that may occur during testing.

Test logistics mentions the names of the testers and the tests that will be run by them.

3. Define test objectives

This phase defines the objectives and expected results of the test execution. Objectives include:

A list of software features like functionality, GUI, performance standards, etc.

The ideal expected outcome for every aspect of the software that needs testing.

4. Define test criteria

Two main testing criteria determine all the activities in the testing project:

Suspension criteria: Suspension criteria define the benchmarks for suspending all the tests.

Exit criteria: Exit criteria define the benchmarks that signify the successful completion of the test phase or project. These are expected results and must match before moving to the next stage of development.

5. Resource planning

This phase aims to create a detailed list of all the resources required for project completion. For example, human effort, hardware and software requirements, all infrastructure needed, etc.

6. Plan test environment

This phase is very important as the test environment is where the QAs run their tests. The test environments must be real devices, installed with real browsers and operating systems so that testers can monitor software behavior in real user conditions.

7. Schedule and Estimation

Break down the project into smaller tasks and allocate time and effort for each task. This helps in efficient time estimation. Create a schedule to complete these tasks in the designated time with a specific amount of effort.

8. Determine test deliverables

Test deliverables refer to the list of documents, tools, and other equipment that must be created, provided, and maintained to support testing activities in the project.

Why is Test Plan creation important?

Defines Objectives: A test plan clearly outlines the testing objectives and the scope of testing activities, ensuring that all team members understand what needs to be achieved.

Structured Approach : It provides a systematic approach to testing, detailing the steps and processes involved, which helps in organizing the testing effort.

Avoids Scope Creep : By defining what will and will not be tested, the test plan helps manage the scope of testing activities, preventing unnecessary work and focusing on irrelevant areas.

Resource Allocation : Helps in identifying the necessary resources, including personnel, tools, and environments, ensuring they are available when needed.

Identifies Risks : A test plan identifies potential risks and outlines mitigation strategies, helping to address issues proactively rather than reactively.

Contingency Plans : These include contingency plans for dealing with unexpected events or issues that may arise during testing.

Stakeholder Alignment : Facilitates communication among stakeholders, including developers, testers, project managers, and clients, ensuring everyone is aligned on the testing objectives, approach, and schedule.

Documentation : Serves as a comprehensive document that can be referred to by all team members, aiding in knowledge sharing and transparency.

Resource Optimization : Helps in efficiently utilizing available resources, including time and personnel, by providing a clear plan of action.

Focus on Priorities : Ensures that testing efforts are focused on high-priority areas that are critical to the success of the project.

Objectives of the Test Plan

Overview of testing activities: The test plan provides an overview of the testing activities and where to start and stop the work.

Provides timeline: The test plan helps to create the timeline for the testing activities based on the number of hours and the workers needed.

Helps to estimate resources: The test plan helps to create an estimate of the number of resources needed to finish the work.

Serves as a blueprint: The test plan serves as a blueprint for all the testing activities, it has every detail from beginning to end.

Helps to identify solutions: A test plan helps the team members They consider the project's challenges and identify the solutions.

Serves as a rulebook: The test plan serves as a rulebook for following rules when the project is completed phase by phase.

Components and Attributes of Test Plan

1. Objective: It describes the aim of the test plan, whatever the good process and procedure they are going to follow to give quality software to customers. The overall objective of the test is to find as many defects as possible and to make software bug-free. The test objective must be broken into components and sub-components. In every component following activities should be performed.

List all the functionality and performance to be tested.

Make goals and targets based on the application feature.

2. Scope: It consists of information that needs to be tested concerning an application. The scope can be divided into two parts:

In-Scope: The modules that are to be tested rigorously.

Out Scope: The modules that are not to be tested rigorously.

Example: In an application A, B, C, and D features have to be developed, but the B feature has already been designed by other companies. So the development team will purchase B from that company and perform only integrated testing with A, B, and C.

3. Testing Methodology: The methods that are going to be used for testing depend on application to application. The testing methodology is decided based on the feature and application requirements. Since the testing terms are not standard, one should define what kind of testing will be used in the testing methodology. So that everyone can understand it.

4. Approach: The approach of testing different software is different. It deals with the flow of applications for future reference. It has two aspects:

High-Level Scenarios: For testing critical features high-level scenarios are written. For Example, login to a website, and book from a website.

The Flow Graph: It is used when one wants to make benefits such as converging and merging easy.

5. Assumption: In this phase, certain assumptions will be made.

Example:

The testing team will get proper support from the development team.

The tester will get proper knowledge transfer from the development team.

Proper resource allocation will be given by the company to the testing department.

6. Risk: All the risks that can happen if the assumption is broken. For Example, in the case of wrong budget estimation, the cost may overrun. Some reason that may lead to risk is:

Test Manager has poor management skills.

Hard to complete the project on time.

Lack of cooperation.

7. Mitigation Plan: If any risk is involved then the company must have a backup plan, the purpose is to avoid errors. Some points to resolve/avoid risk:

Test priority is to be set for each test activity.

Managers should have leadership skills.

Training course for the testers.

8. Roles and Responsibilities: All the responsibilities and role of every member of a particular testing team has to be recorded.

Example:

Test Manager: Manages the project, takes appropriate resources, and gives project direction.

Tester: Identify the testing technique, verify the test approach, and save project costs.

9. Schedule: Under this, it will record the start and end date of every testing-related activity. For Example, writing the test case date and ending the test case date.

10. Defect Tracking: It is an important process in software engineering as lots of issue arises when you develop a critical system for business. If there is any defect found while testing that defect must be given to the developer team. There are the following methods for the process of defect tracking:

Information Capture: In this, we take basic information to begin the process.

Prioritize: The task is prioritized based on severity and importance.

Communication: Communication between the identifier of the bug and the fixer of the bug.

Environment: Test the application based on hardware and software.

Example: The bug can be identified using bug-tracking tools such as Jira, Mantis, and Trac.

11. Test Environments: It is the environment that the testing team will use i.e. the list of hardware and software, while testing the application, the things that are said to be tested will be written under this section. The installation of software is also checked under this.

Example:

Software configuration on different operating systems, such as Windows, Linux, Mac, etc.

Hardware Configuration depends on RAM, ROM, etc.

12. Entry and Exit Criteria: The set of conditions that should be met to start any new type of testing or to end any kind of testing.

Entry Condition:

Necessary resources must be ready.

The application must be prepared.

Test data should be ready.

Exit Condition:

There should not be any major bugs.

Most test cases should be passed.

When all test cases are executed.

13. Test Automation: It consists of the features that are to be automated and which features are not to be automated.

If the feature has lots of bugs then it is categorized as Manual Testing.

If the feature is frequently tested then it can be automated.

14. Effort Estimation: This involves planning the effort that needs to be applied by every team member.

15. Test Deliverables: It is the outcome from the testing team that is to be given to the customers at the end of the project.

Creating an effective Test Plan

Creating an effective test plan is essential for ensuring a comprehensive and systematic approach to software testing. Here are some best practices to consider when developing a test plan:

1. Understand the Project Requirements:

Gather Information: Ensure a thorough understanding of both functional and non-functional requirements.

Stakeholder Input: Involve stakeholders to gather expectations and address specific concerns.

2. Define Clear Objectives and Scope:

Purpose of Testing: Clearly state the objectives and what you aim to achieve.

In-Scope and Out-of-Scope: Define what will be tested and what will not, to manage expectations and focus resources.

3. Develop a Comprehensive Test Strategy:

Approach: Outline the types of testing to be performed (e.g., functional, regression, performance).

Techniques and Tools: Specify testing techniques (e.g., black-box, white-box) and tools (e.g., Selenium, JIRA) to be used.

4. Create Detailed Test Cases:

Test Case Design: Develop detailed test cases covering all scenarios, including positive, negative, edge, and boundary cases.

Traceability: Ensure each test case is traceable to specific requirements to verify comprehensive coverage.

5. Establish a Test Environment:

Setup Requirements: Define hardware, software, network configurations, and tools required for testing.

Environment Management: Ensure the test environment closely mirrors the production environment to identify environment-specific issues.

6. Plan for Test Data and Reporting Mechanisms:

Data Requirements: Identify and manage realistic, consistent test data securely, especially if it includes sensitive information.

Status Reporting: Establish processes for regular status updates on testing progress, issues, and results, and use defect tracking systems effectively.

TEST STRATEGY

Test strategy is a document that which the **Testing** of any application, which is organized in a well-documented way with proper goals.

Test strategy is a plan for defining an approach to the Software Testing Life Cycle (STLC).

In addition, the test strategy provides the following details, which are required while writing the test document:

What technique must be used in addition to this?

Which of the modules will be examined?

What criteria apply for entry and exit?

What kind of testing is necessary?

Components of a Test Strategy

- The test effort, test domain, test setups, and test tools used to verify and validate a set of functions are all outlined in a Test Strategy.

- It also includes schedules, resource allocations, and employee utilization information. This data is essential for the test team (Test) to be as structured and efficient as possible.
- A Test Strategy differs from a Test Plan, which is a document that gathers and organizes test cases by functional areas and/or types of testing in a format that can be presented to other teams and/or customers.
- Both are critical components of the Quality Assurance process since they aid in communicating the breadth of the test method and ensuring test coverage while increasing the testing effort's efficiency.

Below diagram shows the components of the test strategy:



1. Scope and Overview

An overview of the project, as well as information on who should utilize this page.

Include information such as who will evaluate and approve the document.

Define the testing activities and phases that will be performed, as well as the timetables that will be followed in relation to the overall project timelines stated in the test plan.

2. Testing Methodology

Testing methodology is used to specify the degrees of testing, testing procedures, roles, and duties of all team members. The change management process, which includes the modification request submission, pattern to be utilized, and activity to manage the request, is also included in the testing strategy.

Define the testing process, testing level, roles, and duties of each team member.

Describe why each test type is defined in the test plan (for example, unit, integration, system, regression, installation/uninstallation, usability, load, performance, and security testing) should be performed.

3. Testing Environment Specifications

The testing environment specification in the test strategy document includes clear instructions on how to produce test data.

The information about the number of environments and the needed configuration for each environment should be included in the test environment setup.

For example, the functional test team might have one test environment and the UAT team might have another.

Define the number of users supported in each environment, as well as each user's access roles and software and hardware requirements, such as the operating system, RAM, free disc space, and the number of systems.

It's just as crucial to define the test data needs.

Give specific instructions on how to generate test data (either generate data or use production data by masking fields for privacy).

Define a backup and restoration strategy for test data.

Due to unhandled circumstances in the code, the test environment database may encounter issues.

The backup and restoration method should state who will take backups when backups should be taken, what should be included in backups, when the

database should be restored, who will restore it, and what data masking procedures should be implemented if the database is restored.

4. Testing Tools

Testing tools are an important part of the test strategy document since it contains all of the information on the test management and automation tools that are required for test execution.

Define the tools for test management and automation that will be utilized to execute the tests.

Describe the test approach and tools needed for performance, load, and security testing.

Mention whether the product is open-source or commercial, as well as the number of individuals it can accommodate, and make suitable planning.

5. Release Control

Release Control is a crucial component of the test strategy document. It's used to make sure that test execution and release management strategies are established in a systematic way. It specifies the following information-

Different software versions in test and UAT environments can occur from unplanned release cycles.

All adjustments in that release will be tested using the release management strategy, which includes a proper version history.

Set up a build management process that answers questions like where the new build should be made available, where it should be deployed when to receive the new build, where to acquire the production build, who will give the go signal for the production release, and so on.

6. Risk Analysis

Risk Analysis is the next section of the test strategy paper. All potential hazards associated with the project are described in the test strategy document and can become an issue during test execution.

7. Review and Approval

Review and Approval is the last section of the Testing strategy paper.

When all of the testing activities are stated in the test strategy document, it is evaluated by the persons that are involved, such as:

System Administration Team.

Project Management Team.

Development Team.

Business Team.

RESOURCE REQUIREMENTS

- Resource requirement is a detailed summary of all types of resources required to complete project task.
- Resource could be human, equipment and materials needed to complete a project.
- The resource requirement and planning is important factor of the test planning because helps in determining the number of resources (employee, equipment...) to be used for the project.
- Therefore, the Test Manager can make the correct schedule & estimation for the project.

Some of the following factors need to be considered:

- Machine configuration (RAM,processor,disk)needed to run the product under test.
- Overheads required by test automation tools, if any
- Supporting tools such as compilers, test data generators, configuration management tools.
- The different configurations of the supporting software(e.g. OS)that must be present
- Special requirements for running machine-intensive tests such as load tests and performance tests.

- Appropriate number of licenses of all the software

Human Resource & System Resource:

For testing, a web application, you should plan the resources.

1 Server

Install the web application under test. This includes a separate web server, database server, and application server if applicable

2 Test tool

The testing tool is to automate the testing, simulate the user operation, and generate the test results. There are tons of test tools you can use for this project such as Selenium, QTP...etc.

3. Network

You need a Network include LAN and Internet to simulate the real business and user environment.

TESTER ASSIGNMENTS

- They may be involved in or even be the primary people identifying test conditions and creating test designs, test cases, test procedure specifications and test data, and may automate or help to automate the tests.
- Tester assignments typically refer to the process of assigning individuals or teams to test specific components, features, or aspects of a product or system.
- Testing is an essential part of software development, quality assurance, and product validation. Assigning testers to different tasks helps ensure thorough and efficient testing coverage.

Here are some considerations for tester assignments:

Test plan and strategy: Begin by creating a test plan and strategy that outlines the testing objectives, scope and approach. This will help identify the different types of tests required and the specific areas that need to be covered.

Test case creation: Testers can be assigned to create test cases based on the test plan and requirements. Test cases should cover different scenarios and use cases to ensure comprehensive testing.

Testing types: Identify the different types of testing needed, such as functional testing, performance testing, security testing, usability testing, etc. Assign testers with the relevant expertise and skills for each type of testing.

Test environment: Assign testers to set up and configure the test environment, including any necessary hardware, software, or network configurations. This ensures that the testing environment accurately reflects the production environment.

Test execution: Assign testers to execute the test cases and document the results. Testers should follow the test plan and report any issues or bugs they encounter during the testing process.

Bug reporting and tracking: Assign testers to report identified bugs or issues in a structured manner, including detailed descriptions, steps to reproduce, and any supporting documentation. Testers may also be responsible for tracking the status and resolution of reported issues.

Regression testing: After bug fixes or changes are made, assign testers to perform regression testing to ensure that the fixes or changes did not introduce new issues or break existing functionality.

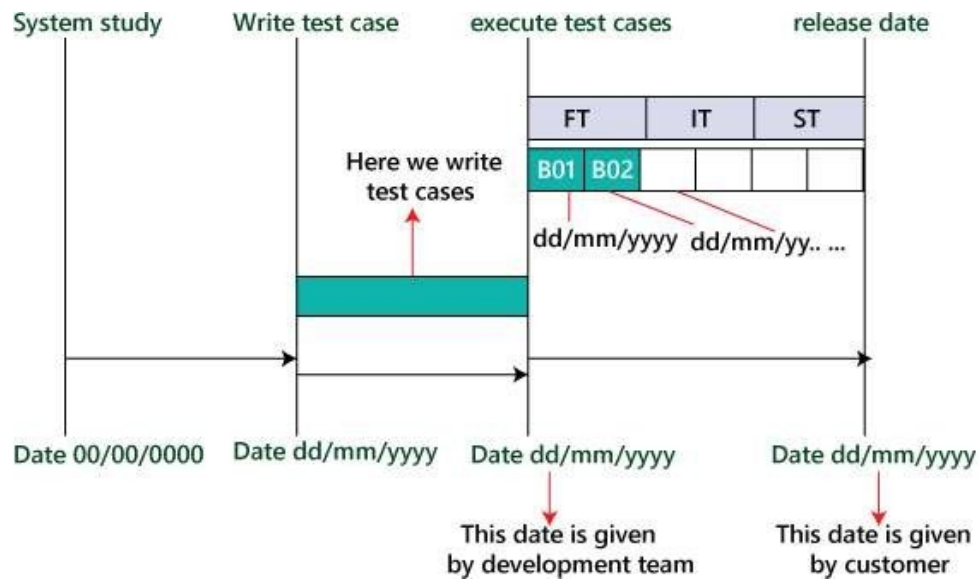
Collaboration and communication: Assign testers to collaborate with developers, product managers, and other stakeholders to ensure clear communication and understanding of testing requirements, priorities, and progress.

Test coverage analysis: Assign testers to analyze the test coverage and identify any gaps or areas that require additional testing. This helps ensure that all critical features and functionalities are adequately tested.

Test automation: Assign testers with automation skills to develop and maintain automated test scripts, which can help increase testing efficiency and coverage.

Test Schedule:

It is used to explain the timing to work, which needs to be done or this attribute covers when exactly each testing activity should start and end? And the exact data is also mentioned for every testing activity for the particular date.



Therefore ,as we can see in the below image that for the particular activity, there will be a starting date and ending date; for each testing to a specific build, there will be the specified date.

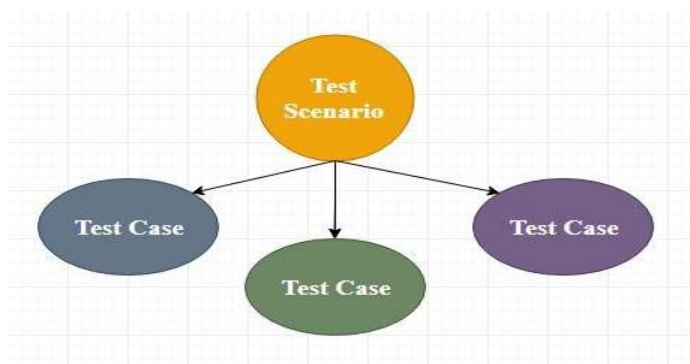
For example

Writing test cases

Execution process

TEST CASES

The test case is defined as a group of conditions under which a tester determines



whether a software application is working as per the customer's requirements or not. Test case designing includes preconditions, case name, input conditions, and expected result. A test case is a first level action and derived from test scenarios.

It is an in-details document that contains all possible inputs (positive as well as negative) and the navigation steps, which are used for the test execution process. Writing of test cases is a one- time attempt that can be used in the future at the time of regression testing.

Test case gives detailed information about testing strategy, testing process, preconditions, and expected output. These are executed during the testing process to check whether the software application is performing the task for that it was developed or not.

Test case helps the tester in defect reporting by linking defect with test case ID.

Detailed test case documentation works as a full proof guard for the testing team because if developer missed something, then it can be caught during execution of these full-proof test cases.

To write the test case, we must have the requirements to derive the inputs, and the test scenarios must be written so that we do not miss out on any features for testing. Then we should have the test case template to maintain the uniformity, or every test engineer follows the same approach to prepare the test document.

Generally, we will write the test case whenever the developer is busy in writing the code.

When do we write a test case?

We will write the test case when we get the following:

When the customer gives the business needs then, the developer starts developing and says that they need 3.5 months to build this product.

And In the meantime, the testing team will start writing the test cases.

Once it is done, it will send it to the Test Lead for the review process.

And when the developers finish developing the product, it is handed over to the testing team.

The test engineers never look at the requirement while testing the product document because testing is constant and does not depend on the mood of the person rather than the quality of the test engineer.

Why we write the test cases?

We will write the test for the following reasons:

- To require consistency in the test case execution
- To make sure a better test coverage

- It depends on the process rather than on a person
- To avoid training for every new test engineer on the product
- To require consistency in the test case execution: we will see the test case and start testing the application.
- To make sure a better test coverage: for this, we should cover all possible scenarios and document it, so that we need not remember all the scenarios again and again.
- It depends on the process rather than on a person: A test engineer has tested an application during the first release, second release, and left the company at the time of third release.
- As the test engineer understood a module and tested the application thoroughly by deriving many values.
- If the person is not there for the third release, it becomes difficult for the new person. Hence all the derived values are documented so that it can be used in the future.
- To avoid giving training for every new test engineer on the product: When the test engineer leaves, he/she leaves with a lot of knowledge and scenarios.
- Those scenarios should be documented so that the new test engineer can test with the given scenarios and also can write the new scenarios.

Test case template

Header

Test Case Name/ID :- **Release - Version - Application Name - Module**

Test Case Type:-

E.T.C	I.T.C	S.T.C
-------	-------	-------

Requirement Number:-

Module:-

Severity:- **Critical/Major/Minor**

Status:-

Release:-

Version:-

Pre-condition:-

Test Data:-

Summary:-

Body

Step No.	Descri- ption	Inputs	Expected Result	Actual Reasult	Status	Comments
...	...	...	...	...	...	...
...	...	...	...	...	...	...

Footer

Author:-

Reviewd By:-

Date:-

Approved By:-

The primary purpose of writing a test case is to achieve the efficiency of the application.

As we know, the actual result is written after the test case execution, and most of the time, it would be same as the expected result. But if the test step will fail, it will be different. So, the actual result field can be skipped, and in the Comments section, we can write about the bugs.

And also, the Input field can be removed, and this information can be added to the description field.

The above template we discuss above is not the standard one because it can be different for each company and also with each application, which is based on the test engineer and the test lead. But for testing one application, all the test engineers should follow a usual template, which is formulated.

The test case should be written in simple language so that a new test engineer can also understand and execute the same. In the above sample template, the header contains the following:

Step number

It is also essential because if step number 20 is failing, we can document the bug report and hence prioritize working and also decide if it's a critical bug.

Test case type:

It can be functional, integration or system test cases or positive or negative or positive and negative test cases. Release:

One release can contain many versions of the release.

Pre-condition:

These are the necessary conditions that need to be satisfied by every test engineer before starting the test execution process. Or it is the data configuration or the data setup that needs to be created for the testing.

For example: In an application, we are writing test cases to add users, edit users, and delete users. The per-condition will be seen if user A is added before editing it and removing it.

Test data

These are the values or the input we need to create as per the per-condition.

For example, Username, Password, and account number of the users.

The test lead may be given the test data like username or password to test the application, or the test engineer may themselves generate the username and password.

Severity

The severity can be major, minor, and critical, the severity in the test case talks about the importance of that particular test cases. All the test execution process always depends on the severity of the test cases.

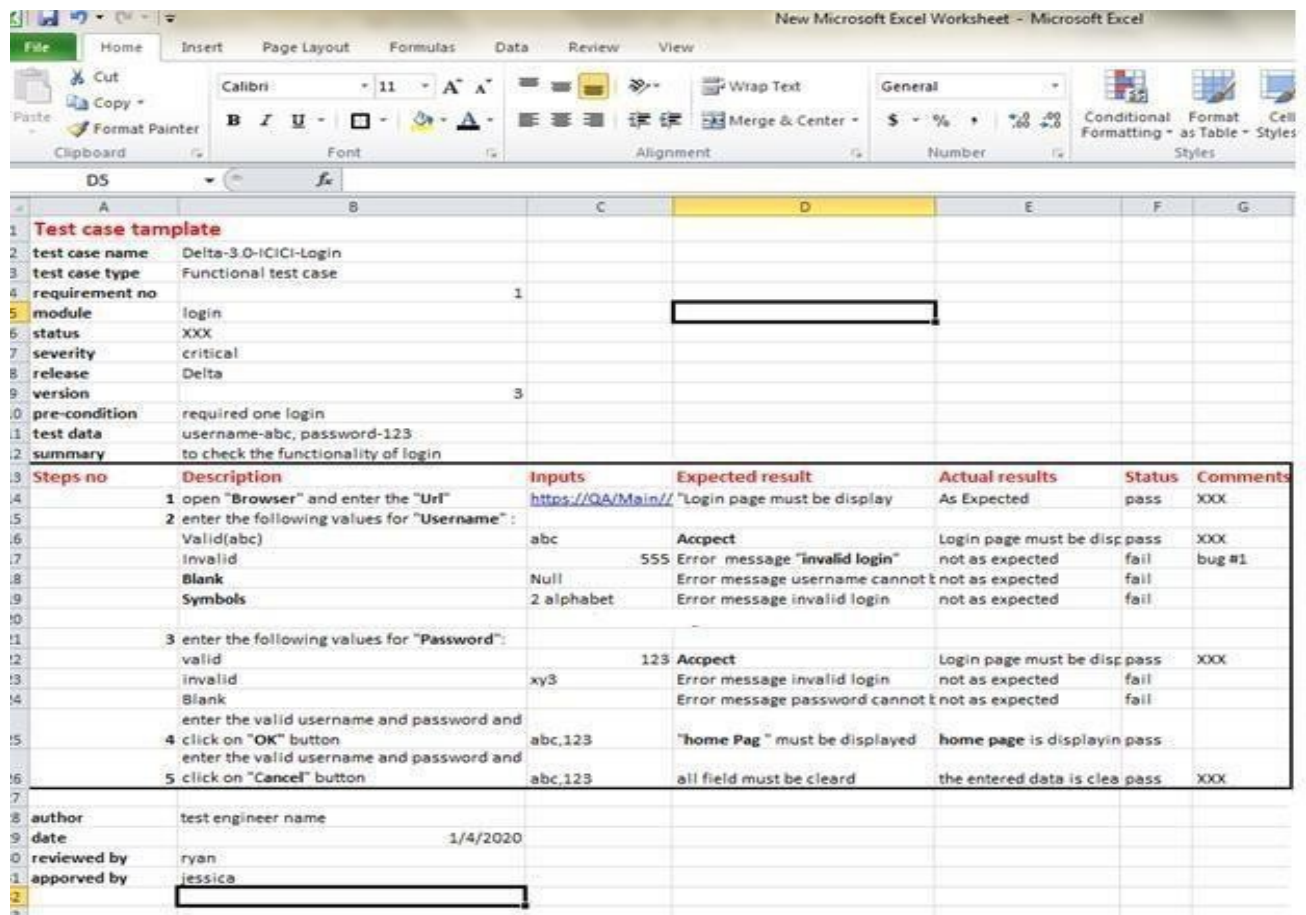
Brief description

The test engineer has written a test case for a particular feature.

If he/she comes and reads the test cases for the moment, he/she will not know for what feature has written it. So, the brief description will help them in which feature test case is written.

Example of a test case template.

Here, we are writing a test case for the ICICI application's Login module:



Steps no	Description	Inputs	Expected result	Actual results	Status	Comments
1	open "Browser" and enter the "Url"	https://QA/Main/	Login page must be display	As Expected	pass	XXX
2	enter the following values for "Username" : Valid(abc) Invalid Blank Symbols	abc Null 2 alphabet	Accept Error message "invalid login" Error message username cannot t Error message invalid login	Login page must be disp not as expected not as expected not as expected	pass fail fail fail	XXX bug #1
3	enter the following values for "Password": valid invalid Blank enter the valid username and password and click on "OK" button enter the valid username and password and click on "Cancel" button	123 xy3 abc,123 abc,123	Accept Error message invalid login Error message password cannot t "home Pag" must be displayed all field must be cleard	Login page must be disp not as expected not as expected home page is displayin the entered data is clea	pass fail fail pass pass	XXX
author	test engineer name					
date	1/4/2020					
reviewed by	ryan					
approved by	jessica					

TYPES OF TEST CASES

We have a different kind of test cases, which are as follows:

Function test cases

Integration test cases

System test cases

FUNCTIONAL TEST CASES

Firstly, we check for which field we will write test cases and then describe accordingly. In functional testing or if the application is data-driven, we require the input column else; it is a bit time-consuming.

Rules to write functional test cases:

- In the expected results column, try to use should be or must be.
- Highlight the Object names.
- We have to describe only those steps which we required the most; otherwise, we do not need to define all the steps.
- To reduce the excess execution time, we will write steps correctly.
- Write a generic test case; do not try to hard code it.
- Let say it is the amount transfer module, so we are writing the functional test cases for it and then also specifies that it is not a login feature.

The image shows a green-themed user interface for an 'AMOUNT TRANSFER' form. The title 'AMOUNT TRANSFER' is at the top in white. Below it, there are three input fields, each preceded by a label and a small icon of four dots. The first field is labeled 'From Account Number (FAN)', the second 'To Account Number (TAN)', and the third 'Amount'. At the bottom, there are two white buttons with black text: 'TRANSFER' and 'CANCEL'.

AMOUNT TRANSFER	
....	From Account Number (FAN)
....	To Account Number (TAN)
....	Amount
TRANSFER CANCEL	

The functional test case for amount transfer module is in the below Excel file:

Functional Test case template						
Test case name	beta-L0-ICIG-amount transfer					
Test case type	Functional test case					
Requirement no	6					
Module	amount transfer					
Status	XXX					
Severity	Critical					
Release	Beta					
Version	1					
Pre-condition	sender login two account number Balance-> exist					
Test data	Username:xyz, Password:1234 1231, 4321 3000-9000					
Summary	to check the functionality of amount transfer					
Steps no	Description	Inputs	Expected result	Actual results	Status	Comments
1	Open "Browser" and enter the "Url"	https://QA/Main/IN	login page" must be display	As Expected	pass	XXX
2	Enter the following values for "Username" and "Password" and click on the "OK" button	xyz, 1234	"Home page" must be displayed	As Expected	pass	XXX
3	Click on the "Amount Transfer"	Null			pass	XXX
4	Enter the following for From Account number (FAN):					
	valid	1234	Accept	As Expected	pass	
	invalid	1124	Error message invalid account		fail	
	blank		Error message FAN value cannot be blank		fail	
			test maximum coverage			
5	Enter the following values for "TO account number" (TAN)					
	valid	4321	Accept	As expected	pass	XXX
	invalid	6655	Error message invalid account		fail	
	Blank		Error message TAN value cannot be blank			
			test maximum coverage			
6	enter the value for "Amount"					
	valid	5000, 5001, 9000, 84	Accept	As expected	pass	XXX
	invalid	4999, 9001	error message amount should be between (5000-9000)		fail	
7	Enter the value for "FAN, TAN, Amount" click on the "Transfer" button					
	FAN	1234				
	TAN	4321	"Confirmation Message" amount transfer successfully must be displayed	As expected	pass	XXX
	Amount	5000				
8	Enter the value for "FAN, TAN, Amount" click on the "Cancel" button					
	FAN	1234				
	TAN	4321	All field must be cleared	As expected	pass	XXX
	Amount	6000				
Author	Semi					
Date	4/1/2020					
Reviewed by	jessica					
Approved by	ryan					

INTEGRATION TEST CASE

In this, we should not write something which we already covered in the functional test cases, and something we have written in the integration test case should not be written in the system test case again.

Rules to write integration test cases

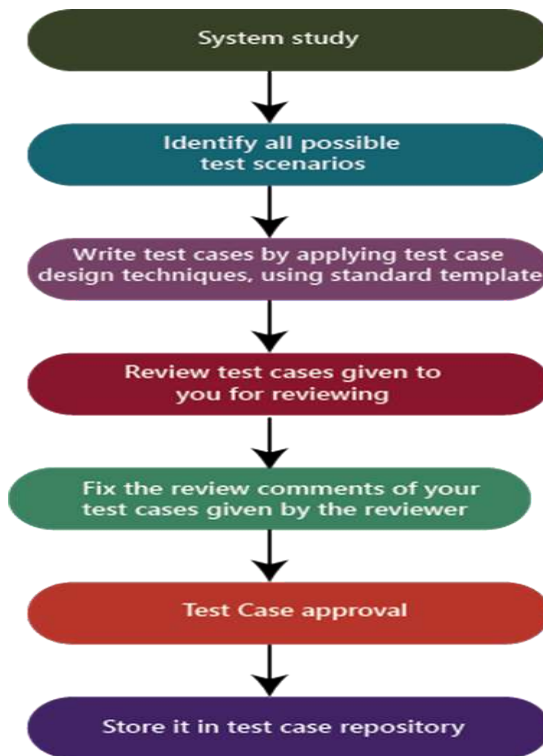
- Firstly, understand the product
- Identify the possible scenarios
- Write the test case based on the priority
- When the test engineer writing the test cases, they may need to consider the following aspects: If the test cases are in details:
 - They will try to achieve maximum test coverage.
 - All test case values or scenarios are correctly described.
 - They will try to think about the execution point of view.
 - The template which is used to write the test case must be unique.

SYSTEM TEST CASES

We will write the system test cases for the end-to-end business flows. And we have the entire modules ready to write the system test cases.

The process to write test cases

The method of writing a test case can be completed into the following steps, which are as below:



System study

In this, we will understand the application by looking at the requirements or the SRS, which is given by the customer.

Identify all scenarios:

When the product is launched, what are the possible ways the end-user may use the software to identify all the possible ways.

I have documented all possible scenarios in a document, which is called test design/high-level design.

The test design is a record having all the possible scenarios.

Write test cases

Convert all the identified scenarios to test claims and group the scenarios related to their features, prioritize the module, and write test cases by applying test case

design techniques and use the standard test case template, which means that the one which is decided for the project.

Review the test cases

Review the test case by giving it to the head of the team and, after that, fix the review feedback given by the reviewer.

Test case approval

After fixing the test case based on the feedback, send it again for the approval.

Store in the test case repository

After the approval of the particular test case, store in the familiar place that is known as the test case repository.

BUG REPORTING

Bug reporting in software testing is a structured process of documenting and communicating issues found in software. It's a crucial part of ensuring the final product meets quality standards and functions as intended. Effective bug reports provide developers with the necessary information to understand, reproduce, and fix issues, ultimately improving software reliability and stability.

Key Aspects of Bug Reporting:

1. Documentation:

Bug reports are written documents that detail software defects or issues.

2. Communication:

They serve as a way for testers to communicate problems to developers and other stakeholders.

3. Reproducibility:

Good bug reports include clear steps to reproduce the issue, allowing developers to easily verify the bug.

4. Context:

They should provide relevant context, such as the test case, environment, and expected vs. actual outcomes.

5. Severity and Priority:

Assessing the severity and priority of the bug helps prioritize fixing efforts.

6. Visual Evidence:

Screenshots, videos, or logs can be included to visually demonstrate the issue and aid in understanding.

7. Bug Tracking:

Bug reporting is often integrated with bug tracking systems to manage, track, and monitor the status of reported bugs.

Key Elements of a Good Bug Report:

Clear Title: A concise and descriptive title summarizing the issue.

Detailed Description: A clear explanation of the issue, including when and how it occurred.

Steps to Reproduce: Step-by-step instructions on how to recreate the bug.

Expected vs. Actual Results: What was expected to happen versus what actually happened.

Environment Details: Information about the environment where the bug was found (e.g., browser, operating system).

Visual Proof: Screenshots, videos, or logs that demonstrate the bug.

Severity and Priority: An assessment of the bug's impact and urgency.

Additional Information: Relevant console logs, network requests, or source URLs can be included.

Benefits of Effective Bug Reporting:

Improved Software Quality:

By identifying and addressing bugs early, the overall quality of the software is enhanced.

Faster Debugging:

Well-written bug reports allow developers to quickly understand and fix issues.

Streamlined Development Process:

Effective bug reporting facilitates better communication and collaboration between testers and developers.

Enhanced User Experience:

By fixing bugs, the user experience is improved, leading to increased user satisfaction.

SOFTWARE TESTING METRICS

The purpose of software testing metrics is to increase the efficiency and effectiveness of the software testing process while also assisting in making better decisions for future testing by providing accurate data about the testing process.

Importance of Metrics in Software Testing:

Test metrics are essential in determining the software's quality and performance. Developers may use the right software testing metrics to improve their productivity.

Early Problem Identification: By measuring metrics such as defect density and defect arrival rate, testing teams can spot trends and patterns early in the development process.

Allocation of Resources: Metrics identify regions where testing efforts are most needed, which helps with resource allocation optimization. By ensuring that testing resources are concentrated on important areas, this enhances the strategy for testing as a whole.

Monitoring Progress: Metrics are useful instruments for monitoring the advancement of testing. They offer insight into the quantity of test cases that have been run, their completion rate, and if the testing effort is proceeding according to plan.

Continuous Improvement: Metrics offer input on the testing procedure, which helps to foster a culture of continuous development.

Types of Software Testing Metrics:

Software testing metrics are divided into three categories:

Process Metrics: A project's characteristics and execution are defined by process metrics. These features are critical to the SDLC process's improvement and maintenance (Software Development Life Cycle).

Product Metrics: A product's size, design, performance, quality, and complexity are defined by product metrics. Developers can improve the quality of their software development by utilizing these features.

Project Metrics: Project Metrics are used to assess a project's overall quality. It is used to estimate a project's resources and deliverables, as well as to determine costs, productivity, and flaws.

Formula for Test Metrics:

To get the percentage execution status of the test cases, the following formula can be used:

Percentage test cases executed = (No of test cases executed / Total no of test cases written) x 100

Similarly, it is possible to calculate for other parameters also such as test cases that were not executed, test cases that were passed, test cases that were failed, test cases that were blocked, and so on.

1. Test Case Effectiveness:

Test Case Effectiveness = (Number of defects detected / Number of test cases run) x 100

2. Passed Test Cases Percentage: Test Cases that Passed Coverage is a metric that indicates the percentage of test cases that pass.

Passed Test Cases Percentage = (Total number of tests ran / Total number of tests executed) x 100

3. Failed Test Cases Percentage: This metric measures the proportion of all failed test cases.

Failed Test Cases Percentage = (Total number of failed test cases / Total number of tests executed) x 100

4. Blocked Test Cases Percentage: During the software testing process, this parameter determines the percentage of test cases that are blocked.

Blocked Test Cases Percentage = (Total number of blocked tests / Total number of tests executed) x 100

5. Fixed Defects Percentage: Using this measure, the team may determine the percentage of defects that have been fixed.

Fixed Defects Percentage = (Total number of flaws fixed / Number of defects reported) x 100

6. Rework Effort Ratio: This measure helps to determine the rework effort ratio.

Rework Effort Ratio = (Actual rework efforts spent in that phase/ Total actual efforts spent in that phase) x 100

7. Accepted Defects Percentage: This measures the percentage of defects that are accepted out of the total accepted defects.

Accepted Defects Percentage = (Defects Accepted as Valid by Dev Team / Total Defects Reported) x 100

8. Defects Deferred Percentage: This measures the percentage of the defects that are deferred for future release.

Defects Deferred Percentage = (Defects deferred for future releases / Total Defects Reported) x 100

Example of Software Test Metrics Calculation:

S No.	Testing Metric	Data retrieved during test case development
1	No. of requirements	5
2	The average number of test cases written per requirement	40
3	Total no. of Test cases written for all requirements	200
4	Total no. of Test cases executed	164
5	No. of Test cases passed	100
6	No. of Test cases failed	60
7	No. of Test cases blocked	4
8	No. of Test cases unexecuted	36
9	Total no. of defects identified	20
10	Defects accepted as valid by the dev team	15
11	Defects deferred for future releases	5
12	Defects fixed	12

1. Percentage test cases executed = (No of test cases executed / Total no of test cases written) x 100

= (164 / 200) x 100

= 82

2. Test Case Effectiveness = (Number of defects detected / Number of test cases run) x 100

= (20 / 164) x 100

= 12.2

3. Failed Test Cases Percentage = (Total number of failed test cases / Total number of tests executed) x 100

= (60 / 164) * 100

= 36.59

4. Blocked Test Cases Percentage = (Total number of blocked tests / Total number of tests executed) x 100

= (4 / 164) * 100

= 2.44

5. Fixed Defects Percentage = (Total number of flaws fixed / Number of defects reported) x 100

= (12 / 20) * 100

= 60

6. Accepted Defects Percentage = (Defects Accepted as Valid by Dev Team / Total Defects Reported) x 100

= (15 / 20) * 100

= 75

7. Defects Deferred Percentage = (Defects deferred for future releases / Total Defects Reported) x 100

= (5 / 20) * 100

= 25

TEST DESIGN AND EXECUTION

Objectives of Testing

Delivering quality products is the ultimate objective of testing. The various objectives of testing:

- Identification of Bugs and Errors
- Delivering Quality Product
- Justification with Requirement
- Increasing Confidence in the Product
- Enhanced Growth

1. Identification of Bugs and Errors

Once the developer finishes coding, the tester starts testing. During testing, QA validates each module under various conditions. Afterward, they collect all the errors & bugs and forward them to the developer to fix. With modern advancements like AI/ML for test case generation, better analysis, and execution, the testing processes are now capable of delivering better products. Other methods include Shift-Left Testing and Robotic Process Automation (RPA) Testing which helps in maximizing test coverage.

2. Delivering Quality Product

The main aim of testing is to maintain the quality of the product. Also, testing has its cycle and in each phase, all focus revolves around quality only. These include using the core testing methods to perform functional testing and non-functional testing. The emergence of DevOps combined with Quality Assurance (QA) to utilize QAOps along with Agile methodologies has certainly boosted the overall productivity to deliver quality products.

3. Justification with Requirement

During testing, the QA team validates whether the application is following the SRS (System Requirement Specification) document or not. Testing for user experience and implementing API testing helps in validating the business requirements of the product.

4. Increasing Confidence in the Product

The testing team constantly checks the features of the software. It must fulfill the business demand and drive confidence. Security, performance, and scalability testing enabled through automation with CI/CD pipelines through DevOps helps in building and increasing confidence. This assures the reliability of the product.

5. Enhanced Growth

A quality delivery increases the potential of a business. And we all know quality comes through testing only. Increased ROI through proper testing helps in the sustained and continuous growth of the product. Business and user goals are properly met with higher revenues and improved cost efficiency.

TEST DESIGN FACTORS

Test design factors in software testing encompass the key elements and considerations that guide the creation of effective test cases and ensure comprehensive testing of a software application. These factors include test objectives, scenarios, data, techniques, and the overall test plan, all working together to verify functionality, usability, security, and other aspects of the software.

Key Factors of Test Design:

- **Test Objectives:**
Defining the specific goals of the testing effort, such as verifying system functionality, ensuring user-friendliness, and assessing security vulnerabilities.
- **Test Scenarios:**
Outlining the different paths a user might take through the application, covering various user interactions and potential scenarios.
- **Test Data:**
Selecting the appropriate data inputs for test execution, including valid, invalid, and boundary value data.
- **Test Design Techniques:**
Employing various methods to generate test cases, such as black-box, white-box, and grey-box techniques.

➤ **Test Plan:**

Creating a comprehensive document that outlines the testing approach, scope, and success criteria.

Types of Test Design Techniques:

Black-box testing:

Focuses on the functionality of the software without considering its internal structure or code.

- **Equivalence Partitioning:** Dividing the input data into classes based on their expected behavior.
- **Boundary Value Analysis:** Testing the boundaries of input and output values to identify potential errors.
- **Decision Table Testing:** Using a table to represent the relationships between conditions and actions.

White-box testing:

Focuses on the internal structure and code of the software.

- **Statement Coverage:** Ensuring that all statements in the code are executed during testing.
- **Branch Coverage:** Ensuring that all branches in the code are executed during testing.

Grey-box testing:

Combines elements of both black-box and white-box testing.

Other Techniques:

- **State Transition Testing:** Testing the transitions between different states in the software.
- **Use Case Testing:** Testing the application based on user stories and scenarios.
- **Error Guessing:** Intentionally introducing errors to identify potential vulnerabilities.

Test Environment:

- A crucial factor in test design is setting up a test environment that mimics the production environment as closely as possible.
- This includes hardware, software, network configurations, and other dependencies needed for accurate test execution.

Test Automation:

- Automating tests can save time and effort, especially for repetitive or complex tests.

- It ensures consistent execution and can be used to identify regressions.

Test Data:

- Test data is essential for ensuring the functionality and performance of the software.
- It should be representative of the range of inputs and conditions the software will encounter in real-world use.

Risk-Based Testing:

- Prioritizing tests based on the potential risk of failure and the impact on the software's functionality.
- This helps to focus testing efforts on the most critical areas.

Continuous Testing:

- Integrating testing throughout the software development lifecycle, from early stages to deployment.
- This helps to catch defects early and improve software quality.

REQUIREMENT IDENTIFICATION

Requirement identification in software testing is the process of clearly defining and documenting the needs and functionalities of a software product to ensure it meets user expectations and business goals. It involves identifying stakeholders, gathering requirements, categorizing them, and ensuring they are complete, consistent, and testable. This process is crucial for effective testing and helps to minimize errors and development costs.

Steps in Requirement Identification:

1. Identify Key Stakeholders and End Users:

Determine who will use the software and their respective roles and needs.

2. Understand the Project Goal:

Clearly define what the software is intended to achieve and its overall objectives.

3. Gather Requirements:

Collect information about user needs, functional requirements, and non-functional requirements (e.g., performance, security).

4. Categorize and Define Requirements:

Organize the gathered requirements into logical categories and provide detailed descriptions for each.

5. Analyze Requirements:

Evaluate the requirements for completeness, consistency, and testability.

6. Model Requirements:

Use diagrams or other visual representations to illustrate the software's functionality and interactions.

7. Document and Review Requirements:

Create a Software Requirements Specification (SRS) document to formally document the requirements and ensure all stakeholders agree on them.

8. Finalize SRS:

Obtain sign-off from stakeholders on the SRS to confirm its completeness and accuracy.

Benefits of Effective Requirement Identification:

➤ Reduced Development Costs:

Early identification and documentation of requirements helps to avoid costly rework later in the development process.

➤ Improved Software Quality:

By ensuring the software meets user needs and business goals, it leads to higher quality and better user satisfaction.

➤ Effective Testing:

Well-defined requirements provide a clear basis for developing test cases and ensuring thorough testing.

➤ Enhanced Communication:

A well-documented SRS facilitates communication between stakeholders, developers, and testers, leading to better alignment and collaboration.

TESTABLE REQUIREMENTS

Testable requirements in software testing are precise and unambiguous descriptions of a system's functionality, allowing for the creation of test cases to verify that the requirements are met. They are clear, measurable, and complete, leaving no room for interpretation and ensuring that test cases can effectively demonstrate whether a requirement is fulfilled.

Characteristics of Testable Requirements:

- **Clarity:**
Requirements should be written in a way that is easy to understand and leaves no room for ambiguity.
- **Measurability:**
Requirements should be expressed in a way that allows for objective assessment, using quantifiable or observable indicators.
- **Completeness:**
Requirements should cover all aspects of the functionality being tested, leaving no gaps or assumptions.
- **Specificity:**
Requirements should be specific and avoid general terms or vague language.
- **Achievability:**
Requirements should be realistic and achievable within the constraints of the project.
- **Relevance:**
Requirements should be relevant to the project and the needs of the stakeholders.
- **Traceability:**
Requirements should be traceable back to the business needs and user stories, ensuring that the development process is aligned with the overall goals.

Benefits of Testable Requirements:

- **Improved Test Case Design:**
Well-defined testable requirements make it easier to design effective test cases that cover all aspects of the functionality being tested.
- **Reduced Misunderstandings:**
Clear and unambiguous requirements reduce the risk of misunderstandings between developers, testers, and stakeholders, leading to fewer defects and rework.

➤ **Early Defect Detection:**

Testable requirements facilitate early detection of defects, which can save time and cost by preventing them from propagating through the development process.

➤ **Increased Confidence in the Software:**

Thoroughly tested software built on testable requirements provides stakeholders with increased confidence in the quality and reliability of the system.

➤ **Examples of Testable Requirements:**

"The login form must display an error message when the user enters an incorrect username or password".

"The system must save user data to the database within 2 seconds".

"The user interface must be responsive to different screen sizes".

How to Write Testable Requirements?

➤ **Break down complex requirements into smaller, more manageable chunks:**

This makes it easier to define clear and measurable criteria for each individual requirement.

➤ **Use a formal requirements specification document:**

This ensures that all requirements are documented in a structured and organized way.

➤ **Use the SMART acronym (Specific, Measurable, Achievable, Relevant, Time-bound) to guide the writing process:**

This helps ensure that requirements are clear, measurable, and achievable.

➤ **Involve stakeholders in the requirements gathering process:**

This ensures that the requirements are aligned with the needs of the stakeholders and that there is a shared understanding of what is required.

➤ **Review and refine requirements iteratively:**

This ensures that the requirements are accurate, complete, and testable throughout the development process.

TEST DESIGN PROCESS

Test design in software testing is the process of creating a strategic plan for identifying test cases, scenarios, and conditions to ensure the software or system meets its requirements and behaves as expected under various conditions. It involves understanding requirements, identifying test

conditions, and designing test cases, selecting or creating test data, developing test scripts if needed, and reviewing and refining the design to ensure comprehensive coverage.

Here's a more detailed breakdown of the test design process:

1. Understanding Requirements:

Thoroughly analyze the software requirements, specifications, and functional descriptions to understand what needs to be tested.

Identify the key functionalities, features, and business rules that need to be validated.

2. Identifying Test Conditions:

Based on the requirements, define the specific conditions under which the software should be tested.

These conditions should cover various scenarios, including normal, edge, and error cases.

3. Designing Test Cases:

Develop test cases that cover the identified test conditions.

Utilize test case design techniques like equivalence partitioning, boundary value analysis, decision table testing, state transition testing, and use case testing.

Test cases should be clear, concise, and well-structured, with clearly defined steps, expected results, and pass/fail criteria.

4. Selecting or Creating Test Data:

Choose or create appropriate test data that will be used to execute the test cases.

Test data should represent a variety of inputs, including valid and invalid data, as well as data that may cause errors.

5. Developing Test Scripts (if needed):

If automating the test execution, develop test scripts using automation testing tools and frameworks.

Test scripts should be well-structured and maintainable, allowing for easy modification and reuse.

TEST RESULTS

Test results in software testing are the outcomes of executing test cases, revealing whether the software behaves as expected. These results are crucial for understanding the quality of the software and informing decisions about its readiness for release.

Key aspects of test results:

- **Expected vs. Actual Results:**
Test results involve comparing the expected outcome of a test case with the actual outcome observed when the test is executed.
- **Passing and Failing Tests:**
Tests are categorized as passing or failing based on whether the actual result matches the expected result.
- **Defect Reporting:**
Failing tests often indicate defects in the software, which are then reported and tracked for resolution.
- **Test Reports:**
Test results are often documented in test reports, which summarize the testing process, results, and any identified defects.
- **Decision-Making:**
Test results help stakeholders make informed decisions about the software's quality, release readiness, and whether further testing or development is needed.

Importance of Test Results:

- **Quality Assurance:**
Test results ensure the software meets the defined requirements and functions correctly, contributing to its overall quality.
- **Defect Management:**
Identifying and reporting defects early in the testing process allows for timely resolution, reducing the cost and effort required later in development.
- **Release Decisions:**
Test results provide the information needed to determine if the software is ready for release, ensuring that only high-quality software is deployed to users.
- **Stakeholder Communication:**

Test reports provide a clear and concise summary of the testing process and results, enabling stakeholders to understand the status of the software and make informed decisions.

- Continuous Improvement:
Analyzing test results can help identify areas for improvement in the software development process, leading to better software quality in future projects.

What is Test Reporting in Software Testing?

Test reporting in software testing is the process of documenting a detailed report that summarizes the testing process, its results, objectives, and the effectiveness of the testing. This report includes details about any defects found, how the software performed, and suggests areas for improvement.

Steps in the Test Reporting Phase

- Collecting and analyzing test results.
- Preparing comprehensive test reports.
- Reviewing test reports and identifying defects.
- Prioritizing and assigning defects to developers for resolution.
- Verifying defect fixes.
- Re-testing defects.
- Tracking and reporting progress on defect resolution.
- Analyzing and reporting on overall test results.
- Preparing final test reports for stakeholders.

Test Design Preparedness Metrics

- Metrics are tracked
 - To know if a test project is progressing according to schedule
 - Plan the next project more accurately
- The following metrics are monitored

- Preparation status of test cases (PST)
- Average time spent (ATS) in test case design
- Number of available test (NAT) cases
- Number of planned test (NPT) cases
- Coverage of a test suite (CTS)

Test Case Design Effectiveness

- The objective of the test design effectiveness metric is to
 - Measure the *defect revealing ability* of the test suite
 - Use the metric to improve the test design process
- A metric commonly used in the industry to measure test case design effectiveness is the Test Case Design Yield (TCDY)

$$\text{TCDY} = \frac{\text{NPT}}{\text{NPT} + \text{Number of TCE}} \times 100\%$$

- TCE stand for Test Case Escaped
- During testing new defects are found for which no test cases had been planned
- For these new defects, new test cases are designed, which is known as Test Case Escaped metric

MODEL-DRIVEN TEST DESIGN

The methodology is also known as model-driven software development (MDSD), model-driven engineering (MDE) and model-driven architecture (MDA). The MDD approach focuses on the construction of a software model. The model is a diagram that specifies how the software system should work before the code is generated.

What is Model-Driven Test Design?

MDTD is built on the idea that designers will become more effective and efficient if they can raise the level of abstraction. This approach breaks down the testing into a series of small tasks that simplify test generation. Then test designers isolate their tasks and work at a higher level of

abstraction by using mathematical engineering structures to design test values independently of the details of the software or design artifacts, test automation, and Test Execution.

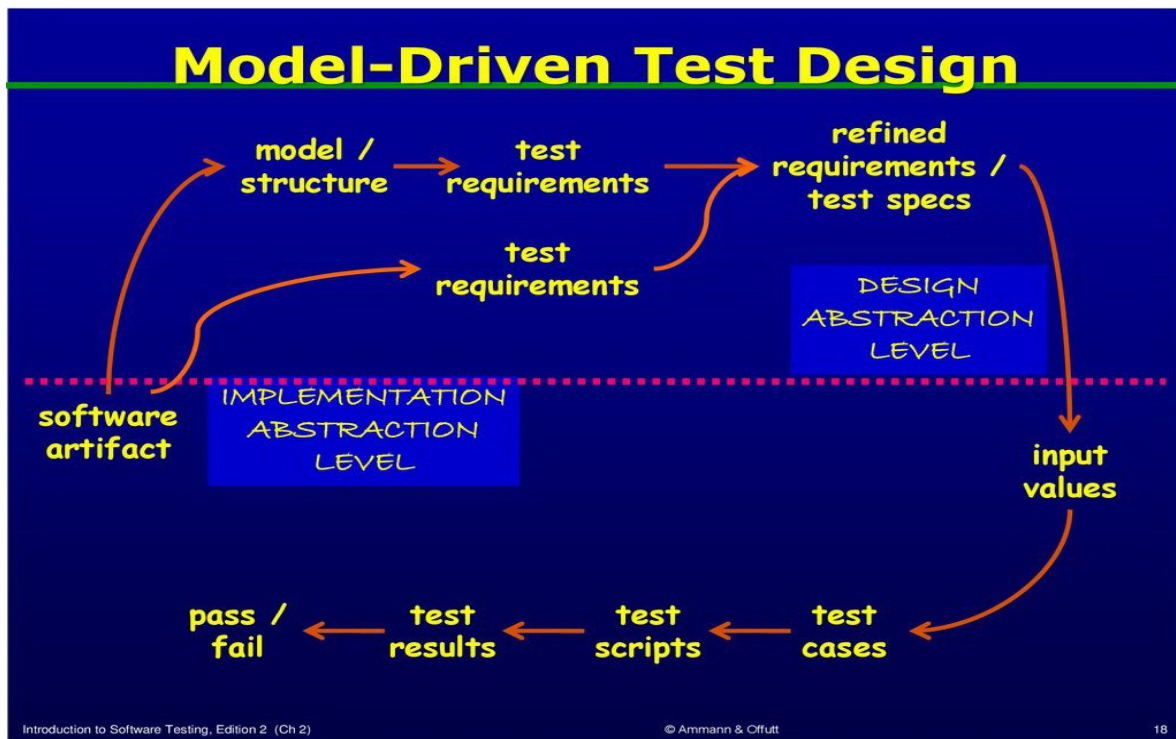
Different phases in MDTD

MDTD can be done in 4 different phases. Each type of activity requires different skills, background knowledge, education, and training. It is better to use different sets of people depend on the situation. 1. Test Design — This can be done in either Criteria-Based where Design test values satisfy coverage criteria or other engineering goals or in Human-Based where Design test values based on domain knowledge of the program and human knowledge of testing which is comparatively harder. This the most technical part of the MDTD process better to use experienced developers in this phase.

2. Test Automation — This involves embedding test values to scripts. Test cases are defined based on the test requirements. Test values are chosen such that we can cover a larger part of the application with fewer test cases. We don't need that much domain knowledge in this phase, however, we need to use technically skilled people.

3. Test Execution — The test engineer will run tests and records the results in this activity. Unlike the previous activities, test execution not required a high skill set such as technical knowledge, logical thinking, or domain knowledge. Since we consider this phase comparatively low risk, we can assign junior intern engineers to execute the process. But we should focus on monitoring, log collecting activities based on automation tools.

4. Test Evaluation — The process of evaluating the results and reporting to developers. This phase is comparatively harder and we expected to have knowledge in the domain, testing, and User interfaces, and psychology



The starting point in Figure 2.4 is a software artifact. This could be program source, a UML diagram, natural language requirements, or even a user manual. A criteria-based test designer uses that artifact to create an abstract model of the software in the form of an input domain, a graph, logic expressions, or a syntax description. Then a coverage criterion is applied to create test requirements. A human-based test designer uses the artifact to consider likely problems in the software, then creates requirements to test for those problems. These requirements are sometimes refined into a more specific form, called the test specification. For example, if edge coverage is being used, a test requirement specifies which edge in a graph must be covered.

TEST CASE ORGANIZATION AND TRACKING

A **Test Case** is a set of actions executed to verify a particular feature or functionality of your software application. A Test Case contains test steps, test data, precondition, postcondition developed for specific test scenario to verify any requirement. The test case includes specific variables or conditions, using which a testing engineer can compare expected and actual results to determine whether a software product is functioning as per the requirements of the customer.

Test Scenario Vs Test Case

Test scenarios are rather vague and cover a wide range of possibilities. Testing is all about being very specific.

For a Test Scenario: Check Login Functionality there many possible test cases are:

- Test Case 1: Check results on entering valid User Id & Password
- Test Case 2: Check results on entering Invalid User ID & Password
- Test Case 3: Check response when a User ID is Empty & Login Button is pressed, and many more

BUG REPORTING

Bug reporting in software testing is a structured process of documenting and communicating issues found in software. It's a crucial part of ensuring the final product meets quality standards and functions as intended. Effective bug reports provide developers with the necessary information to understand, reproduce, and fix issues, ultimately improving software reliability and stability.

Here's a more detailed look at bug reporting in software testing:

Key Aspects of Bug Reporting:

➤ Documentation:

Bug reports are written documents that detail software defects or issues.

➤ Communication:

They serve as a way for testers to communicate problems to developers and other stakeholders.

➤ **Reproducibility:**

Good bug reports include clear steps to reproduce the issue, allowing developers to easily verify the bug.

➤ **Context:**

They should provide relevant context, such as the test case, environment, and expected vs. actual outcomes.

➤ **Severity and Priority:**

Assessing the severity and priority of the bug helps prioritize fixing efforts.

➤ **Visual Evidence:**

Screenshots, videos, or logs can be included to visually demonstrate the issue and aid in understanding.

➤ **Bug Tracking:**

Bug reporting is often integrated with bug tracking systems to manage, track, and monitor the status of reported bugs.

Elements of a Good Bug Report:

- **Clear Title:** A concise and descriptive title summarizing the issue.
- **Detailed Description:** A clear explanation of the issue, including when and how it occurred.
- **Steps to Reproduce:** Step-by-step instructions on how to recreate the bug.
- **Expected vs. Actual Results:** What was expected to happen versus what actually happened.
- **Environment Details:** Information about the environment where the bug was found (e.g., browser, operating system).
- **Visual Proof:** Screenshots, videos, or logs that demonstrate the bug.
- **Severity and Priority:** An assessment of the bug's impact and urgency.

- Additional Information: Relevant console logs, network requests, or source URLs can be included.

Benefits of Effective Bug Reporting:

- Improved Software Quality:

By identifying and addressing bugs early, the overall quality of the software is enhanced.

- Faster Debugging:

Well-written bug reports allow developers to quickly understand and fix issues.

- Streamlined Development Process:

Effective bug reporting facilitates better communication and collaboration between testers and developers.

- Enhanced User Experience:

- By fixing bugs, the user experience is improved, leading to increased user satisfaction.

BUG LIFE CYCLE IN SOFTWARE DEVELOPMENT

During the development of any software product the development teams follow the Software Development Life Cycle (SDLC) processes. But the development process is not so easy and always runs smoothly. During the development process when a product is being developed different types of defects or bugs arise with the product. So these defects are identified and resolved throughout the development process just to deliver a good quality software product at last.

WHAT IS A BUG/DEFECT?

A defect is an error or bug in an application that is created during the building or designing of software due to which software starts to show abnormal behaviors during its use. So it is one of the important responsibilities of the tester to find as many as defects possible to ensure the quality of the product is not affected and the end product is fulfilling all requirements perfectly for which it has been designed and provides the required services to the end-user. Because as much as defects will be identified and resolved then the software will behave perfectly as per expectation.

What is the Defect Life Cycle?

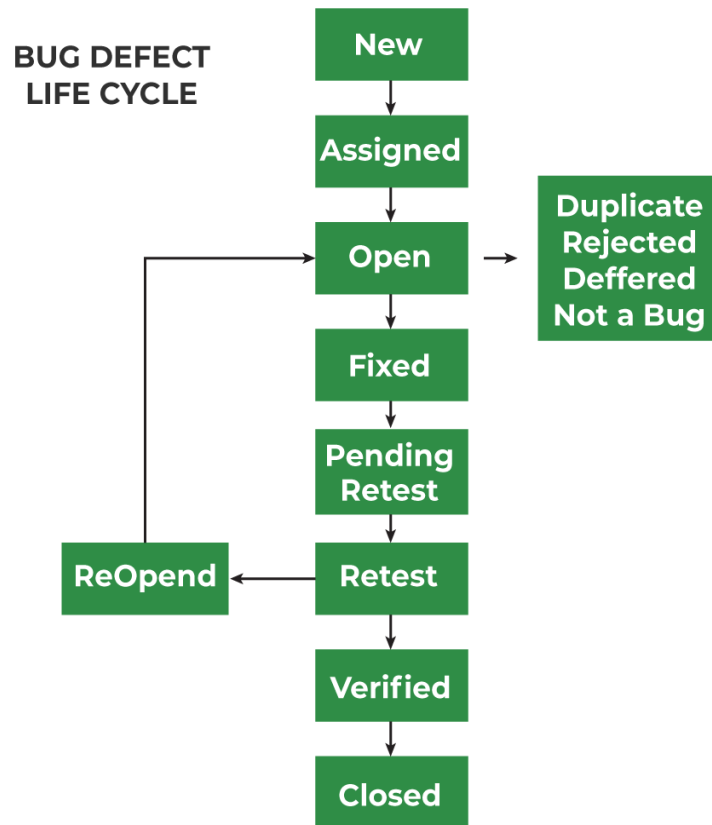
In the Software Development Process, the Defect Life Cycle is the life cycle of a defect or bug that it goes through covering a specific set of states in its entire life. Mainly bug life cycle refers to its entire state starting from a new defect detected to the closing off of that defect by the tester. Alternatively, it is also called a Bug Life Cycle.

- The journey of the Defect Cycle varies from organization to organization and also from project to project because development procedures and platforms as well as testing methods and testing tools differ depending upon organizations and projects.
- The number of states that a defect goes through also varies depending upon the different tools used and processes followed during the software testing.
- The objective of the defect lifecycle is to easily coordinate and communicate the current status of the defect and thus help to make the defect-fixing process efficient.

DEFECT STATUS

Defect status or Bug status is the current state from which the defect is currently going through.

The number of states the defect goes through varies from project to project.



1. New: When any new defect is identified by the tester, it falls in the 'New' state. It is the first state of the Bug Life Cycle. The tester provides a proper Defect document to the Development team so that the development team can refer to Defect Document and can fix the bug accordingly.

2. Assigned: Defects that are in the status of 'New' will be approved and that newly identified defect is assigned to the development team for working on the defect and to resolve that. When the defect is assigned to the developer team the status of the bug changes to the 'Assigned' state.

3. Open: In this 'Open' state the defect is being addressed by the developer team and the developer team works on the defect for fixing the bug. Based on some specific reason if the

developer team feels that the defect is not appropriate then it is transferred to either the 'Rejected' or 'Deferred' state.

4. Fixed: After necessary changes of codes or after fixing identified bug developer team marks the state as 'Fixed'.

5. Pending Retest: During the fixing of the defect is completed, the developer team passes the new code to the testing team for retesting. And the code/application is pending for retesting on the Tester side so the status is assigned as 'Pending Retest'.

6. Retest: At this stage, the tester starts work of retesting the defect to check whether the defect is fixed by the developer or not, and the status is marked as 'Retesting'.

7. Reopen: After 'Retesting' if the tester team found that the bug continues like previously even after the developer team has fixed the bug, then the status of the bug is again changed to 'Reopened'. Once again bug goes to the 'Open' state and goes through the life cycle again. This means it goes for Re-fixing by the developer team.

8. Verified: The tester re-tests the bug after it got fixed by the developer team and if the tester does not find any kind of defect/bug then the bug is fixed and the status assigned is 'Verified'.

9. Closed: It is the final state of the Defect Cycle, after fixing the defect by the developer team when testing found that the bug has been resolved and it does not persist then they mark the defect as a 'Closed' state.

PERFORMANCE TESTING

Performance testing is a type of software testing that focuses on evaluating the performance and scalability of a system or application. Performance testing aims to identify bottlenecks, measure system performance under various loads and conditions, and ensure that the system can handle the expected number of users or transactions.

Performance Testing is the process of analyzing the quality and capability of a product. It is a testing method to determine the system's performance in terms of speed, reliability, and stability under varying workloads.

LOAD TESTING

- The load testing is carried out for normal and extreme load conditions.
- Load testing is a type of performance testing that simulates a real-world load on a system or application to see how it performs under stress.
- The goal of load testing is to identify bottlenecks and determine the maximum number of users or transactions the system can handle.
- It is an important aspect of software testing as it helps ensure that the system can handle the expected usage levels and identify any potential issues before the system is deployed to production.

Load Testing Techniques

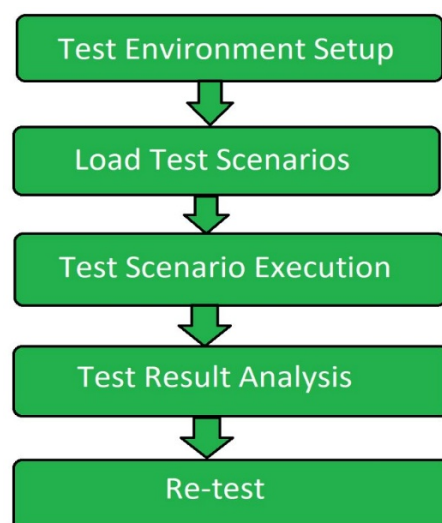
1. **Stress testing:** Testing the system's ability to handle a high load above normal usage levels
2. **Spike testing:** Testing the system's ability to handle sudden spikes in traffic
3. **Soak testing:** Testing the system's ability to handle a sustained load over a prolonged period of time
4. **Tools for Performance Testing:** Make use of specialized load testing tools like Locust, Gatling, JMeter, LoadRunner, and Apache Benchmark. These tools assist in gathering performance measurements and simulating a large number of users.
5. **Specify the Test Objectives:** Clearly state what your load test's goals are. Recognize the required response times, transaction volumes and expected user behavior.

6. **Determine Crucial Situations:** Determine the essential user scenarios that correspond to common usage patterns. A variety of actions, including user logins, searches, form submissions and other significant interactions, should be covered by these scenarios.

Objectives of Load Testing

1. **Evaluation of Scalability:** Assess the system's ability to handle growing user and transaction demands. Find the point at which the system begins to function badly.
2. **Planning for Capacity:** Describe the system's ability to accommodate anticipated future increases in the number of users, transactions and volume of data. Making well-informed decisions regarding infrastructure upgrades is made easier by this.
3. **Determine bottlenecks:** Identify and localize bottlenecks in the application or infrastructure's performance. Finding the places where the system's performance can suffer under load is part of this.
4. **Analysis of Response Time:** For crucial transactions and user interactions, track and evaluate response times. Make that the system responds to changes in load with reasonable response times.
5. **Finding Memory Leaks:** Find and fix memory leaks that may eventually cause a decline in performance. Make sure the programmer doesn't use up too many resources when it's running.

Load Testing Process



1. **Test Environment Setup:** Firstly create a dedicated test environment setup for performing the load testing. It ensures that testing would be done in a proper way.
2. **Load Test Scenario:** In second step load test scenarios are created. Then load testing transactions are determined for an application and data is prepared for each transaction.
3. **Test Scenario Execution:** Load test scenarios that were created in previous step are now executed. Different measurements and metrics are gathered to collect the information.
4. **Test Result Analysis:** Results of the testing performed is analyzed and various recommendations are made.
5. **Re-test:** If the test is failed then the test is performed again in order to get the result in correct way.

Metrics of Load Testing

Metrics are used in knowing the performance of load testing under different circumstances. It tells how accurately the load testing is working under different test cases. It is usually carried out after the preparation of load test scripts/cases.

1. Average Response Time

It tells the average time taken to respond to the request generated by the clients or customers or users. It also shows the speed of the application depending upon the time taken to respond to the all requests generated.

2. Error Rate

The Error Rate is mentioned in terms of percentage denotes the number of errors occurred during the requests to the total number of requests. These errors are usually raised when the application is no longer handling the request at the given time or for some other technical problems. It makes the application less efficient when the error rate keeps on increasing.

3. Throughput

This metric is used in knowing the range of bandwidth consumed during the load scripts or tests and it is also used in knowing the amount of data which is being used for checking the

request that flows between the user server and application main server. It is measured in kilobytes per second.

STRESS TESTING

Stress testing is defined as types of software testing that verifies the stability and reliability of the system. This test particularly determines the system's robustness and error handling under the burden of some load conditions. It tests beyond the normal operating point and analyses how the system works under extreme conditions. Stress testing is performed to ensure that the system does not crash under crunch situations. Stress testing is also known as Endurance Testing.

Characteristics of Stress Testing

1. **Identification of Risk:** Stress testing's main objective is to locate and evaluate a system's possible hazards and weaknesses.
2. **Quantitative and Qualitative Analysis:** While numerical data are crucial, it's also critical to comprehend the qualitative characteristics of the system's response and potential weak points.
3. **Variable Parameters:** Stress testing includes changing variables including interest rates, market conditions, transaction volumes and outside influences that could have an impact on the system.
4. **Cross-Functional Involvement:** Many departments within an organization must work together and participate in stress testing. This cross-functional strategy makes sure that the stress testing procedure benefits from a variety of viewpoints and specialties.
5. **Open and Honest Communication:** Stress testing necessitates open and honest communication regarding the goal, approach, and outcomes of the testing procedure.

Purpose of Stress Testing

- **Analyze the behavior of the application after failure:** The purpose of stress testing is to analyze the behavior of the application after failure and the software should display the appropriate error messages while it is under extreme conditions.

- **System recovers after failure:** Stress testing aims to make sure that there are plans for recovering the system to the working state so that the system recovers after failure.
- **Uncover Hardware issues:** Stress testing helps to uncover hardware issues and data corruption issues.
- **Uncover Security Weakness:** Stress testing helps to uncover the security vulnerabilities that may enter into the system during the constant peak load and compromise the system.
- **Ensures data integrity:** Stress testing helps to determine the application's data integrity throughout the extreme load, which means that the data should be in a dependable state even after a failure.

Stress Testing Process

The stress testing process is divided into 5 steps:

1. Planning the stress test

- **Gathering System Data:** For a web application, collect details about server specifications, database configurations, network bandwidth, and current traffic patterns.
- **Analyzing the System:** Evaluate how the web application performs during peak traffic times, such as during a sales event. Identify any existing issues like slow page loads or database timeouts.
- **Defining Stress Test Goals:** Set goals such as ensuring the application can handle a traffic spike of 10,000 users per minute without crashing and maintaining response times below 2 seconds.

2. Create Automation Scripts

- **Developing Stress Testing Scripts:** Use tools like Apache JMeter or LoadRunner to create scripts that simulate user actions such as logging in, browsing products, and making purchases.
- **Generating Test Data:** Create datasets that include various user profiles, product inventories, and transaction records to ensure the test is realistic and comprehensive.

3. Script Execution

- **Analyzing Test Data:** Review the logs to identify performance trends. Look for any significant slowdowns or failures as the load increases.

- **Identifying Bottlenecks:** If the application slows down significantly at 5,000 users, investigate the server logs to determine if the issue is related to database queries, server CPU limits, or network bandwidth.

4. Result Analysis

This phase involves analyzing stress test results and identifying the bottlenecks.

- **Running the Stress Test Scripts:** Execute the scripts in a test environment that mirrors the production setup. Gradually increase the number of simulated users to observe how the system handles the load.
- **Storing Test Results:** Use logging tools to capture performance data such as server response times, error rates, and resource utilization metrics.

5. Tweaking and Optimization

This step involves fine-tuning the system and optimizing the code with the goal meet the desired benchmarks.

- **Fine-Tuning the System:** Based on the findings, optimize database queries, increase server CPU capacity, or enhance network configurations.
- **Optimizing for Desired Benchmarks:** Retest the application to ensure it now handles 10,000 users per minute with response times below 2 seconds.

VOLUME TESTING

Volume Testing is a type of software testing which is carried out to test a software application with a certain amount of data. The amount used in volume testing could be a database size or it could also be the size of an interface file that is the subject of volume testing. While testing the application with a specific database size, database is extended to that size and after that the performance of the application is tested.

Characteristics of Volume Testing

- Performance of the software decline as passing of the time as there is huge amount of data overtime.
- Basically the test data is created by test data generator.
- Only small amount of data is tested during development phase.
- The test data need to be logically correct.

- The test data is used to assess the performance of the system.

Objectives of Volume Testing

- To recognize the problems that may be created with large amount of data.
- To check the system's performance by increasing the volume of data in the database.
- To find the point at which the stability of the system reduces.
- To identify the capacity of the system or application.
-

Volume Testing Attributes

- **System's Response Time:** During the volume testing, the response time of the system or the application is tested. It is also tested whether the system responds within the finite time or not. If the response time is large then the system is redesigned.
- **Data Loss:** During the volume testing, it is also tested that there is no data loss. If there is data loss some key information might be missing.
- **Data Storage:** During the volume testing, it is also tested that the data is stored correctly or not. If the data is not stored correctly then it is restored accordingly in proper place.
- **Data Overwriting:** In volume testing, it is tested that whether the data is overwritten without giving prior information to the developer. If it so then developer is notified.

Advantages of Volume Testing

- Volume testing is helpful in saving maintenance cost that will be spent on application maintenance.
- Volume testing is also helpful in a rapid start for scalability plans.
- Volume testing also helps in early identification of bottlenecks.
- Volume testing ensures that the system is capable of real world usage.

FAIL-OVER TESTING

Software products are tested multiple times before they are released to ensure they work as expected. However, testing before delivery doesn't guarantee that issues won't arise in the future. Sometimes, software can fail due to unexpected events like network problems or server

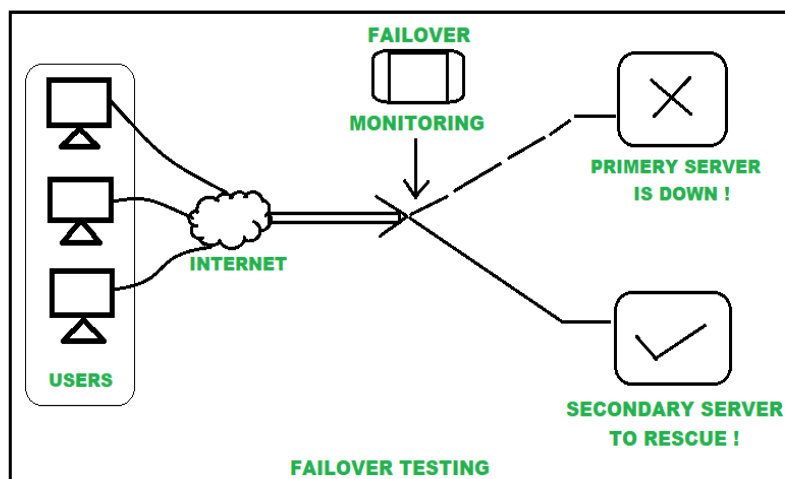
failures. Failover testing is designed to check how well the software can recover from such failures.

For example, imagine your computer shuts down unexpectedly. When you restart your browser, a pop-up asks if you want to restore all the pages you had open. Clicking “Restore” brings back all the tabs exactly as they were. This process of recovering and restoring the previous state of the application is what failover testing ensures.

There are two common configurations for failover testing: **active-active** and **active-passive standby**.

- **Active-Active:** In this setup, all servers are running and share the load equally.
- **Active-Passive Standby:** Here, one server takes the main load, and the backup server remains idle until the primary one fails.

Working of Failover testing



1. **Consider the factors:** Before performing failover testing like budget, time, team, technology, etc.
2. **Analysis on failover reasons and design solutions:** Determine probable failure situations that the system might experience. Examine the causes of failover, including software bugs, hardware malfunctions, network problems, etc. It provides fixes for any flaws or vulnerabilities found in the failover procedure.

3. **Testing failover scenarios:** It develops extensive test cases to replicate various failover scenarios. This covers both unplanned failovers (system or component failures) and scheduled failovers (maintenance). Test cases ought to address many facets of failover, such as load balancing, user impact, network rerouting, and data synchronization.
4. **Executing the test plan:** To reduce the impact on production systems, carry out the failover test plan in a controlled setting. Keep an eye on how the system behaves during failover to make sure it satisfies the recovery point and recovery time objectives (RPO and RTO, respectively).
5. **Detailed report on failover:** Keep a record of the failover testing findings, including any problems you ran across, how long it took to failover and how it affected customers or services. Assess problems according to their severity and offer suggestions for improvements.
6. **Necessary actions based on the report:** Distribute the report on the failover test to all pertinent parties, such as project managers, developers, and system administrators.

Benefits of Failover Testing

1. **Determines Vulnerabilities and Weaknesses:** Failover testing helps in locating holes and vulnerabilities in the system by simulating different failure scenarios.
2. **Verifies Redundancy Procedures:** Redundancy mechanisms are frequently incorporated into systems to improve reliability. In order to make sure that these systems such as redundant servers, databases, or network paths, work flawlessly in the event of a breakdown, failover testing verifies their efficacy.
3. **Improving the User Experience:** Consumers anticipate consistent and dependable service availability. By reducing interruptions and ensuring service availability even in the face of unanticipated circumstances, failover testing contributes to the provision of a positive user experience.
4. **Encourages Compliance:** Failover testing supports organizations in demonstrating compliance with these rules by ensuring that systems can recover within prescribed times.
5. **Encourages Continuous Improvement:** Organizations that regularly test their systems can learn from each test cycle, fix problems found and upgrade their failover procedures to meet changing business needs and technological advancements.

RECOVERY TESTING

Recovery Testing is a crucial aspect of software testing focused on ensuring that applications can recover quickly and effectively from crashes, failures, and unexpected interruptions. This type of testing verifies the software's robustness and its ability to restore operations after a malfunction. Recovery testing is also referred to as Disaster Recovery Testing.

Recovery tests for failures

A system or software should be recovery tested for failures like:

- Power supply failure
- The external server is unreachable
- Wireless network signal loss
- Physical conditions
- The external device not responding
- The external device is not responding as expected, etc.

Steps to be performed before executing a Recovery Test

1. **Recovery Analysis** – It is important to analyze the system's ability to allocate extra resources like servers or additional CPUs. This would help to better understand the recovery-related changes that can impact the working of the system. Also, each of the possible failures, their possible impact, their severity, and how to perform them should be studied.
2. **Test Plan preparation** – Designing the test cases keeping in mind the environment and results obtained in recovery analysis.
3. **Test environment preparation** – Designing the test environment according to the recovery analysis results.
4. **Maintaining Back-up** – Information related to the software, like various states of the software and database should be backed up. Also, if the data is important, then the backing up of the data at multiple locations is important.

5. **Recovery personnel Allocation** – For the recovery testing process, it is important to allocate recovery personnel who are aware and educated enough for the recovery testing being conducted.
6. **Documentation** – This step emphasizes on documenting all the steps performed before and during the recovery testing so that the system can be analyzed for its performance in case of a failure.

Example:

When a browser is working on multiple sessions, we can stimulate software failure by restarting the system. After restarting the system, we can check if it recovers from the failure and reloads all the sessions it was previously working on.

Advantages:

- Improves the quality of the system
- Risk elimination
- Improved performance
- Ensures Reliability
- Identifies Weaknesses
- Enhances User Experience
- Improves System Stability
- Supports Business Continuity
- Validates Backup and Recovery Procedures
- Reduces Costs
- Boosts Confidence

CONFIGURATION TESTING

Configuration Testing is the process of testing the system under each configuration of the supported software and hardware. Here, the different configurations of hardware and software mean the multiple operating system versions, various browsers, various supported drivers, distinct memory sizes, different hard drive types, various types of CPU, etc.

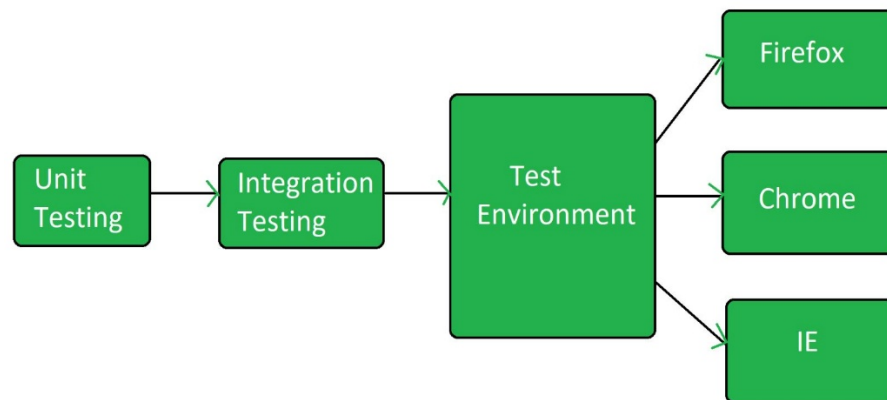
The various configurations are Win XP, Win 7 32/64 bit, Win 8 32/64 bit, Win 10, etc.

1. **Database Configuration:** Oracle, DB2, MySQL, MSSQL Server, Sybase etc.
2. **Browser Configuration:** IE 8, IE 9, FF 16.0, Chrome, Microsoft Edge etc.

Objectives of Configuration Testing:

1. **Adaptability to Different Configurations:** Check that the program's basic features work consistently and dependably in all configurations. Testing the behavior of the program with different setups and settings is part of this process.
2. **Evaluation of Stability:** Examine the software's stability under various configurations. Find and fix any configuration-specific problems that might be causing crashes, unstable systems or strange behavior.
3. **Testing the User Experience:** Assess the value and consistency of the user experience across various setups. Make that the graphical user interface (GUI) of the software adjusts to various screen sizes, resolutions and display settings.
4. **Security throughout Configurations:** To make sure that sensitive data is kept safe, test the software's security features in various setups. Determine and fix any vulnerabilities that might be configuration-specific.
5. **Compatibility of Networks:** Examine the software's behavior with various network setups. Evaluate its compatibility with various network types, speeds and latency.
6. **Data Compatibility:** Check if the programme can manage a range of data configurations, such as those from diverse sources, databases and file formats. Verify the consistency and integrity of the data across various setups.

Configuration Testing Process:

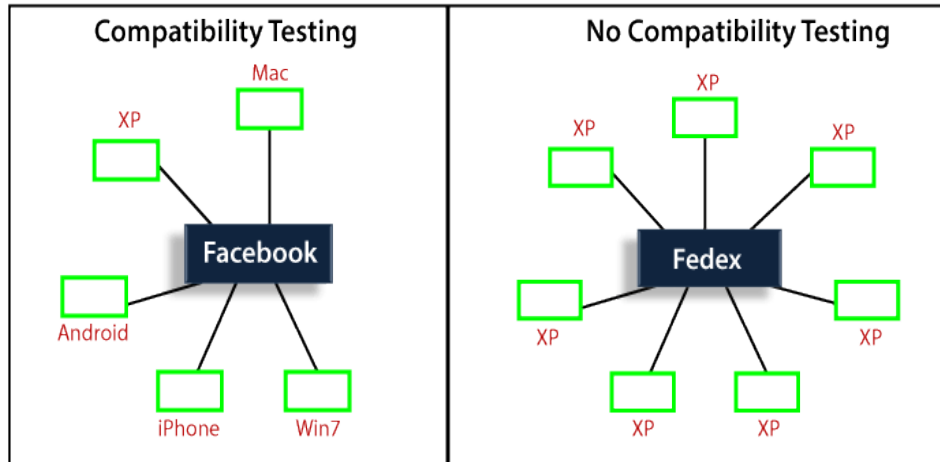


Types of Configuration Testing:

1. **Software Configuration Testing:** Software configuration testing is done over the Application Under Test with various operating system versions and various browser versions etc. It is a time-consuming testing as it takes long time to install and uninstall the various software which are to be used for testing. When the build is released, software configuration begins after passing through the unit test and integration test.
2. **Hardware Configuration Testing:** Hardware configuration testing is typically performed in labs where physical machines are used with various hardware connected to them. When a build is released, the software is installed in all the physical machines to which the hardware is attached and the test is carried out on each and every machine to confirm that the application is working fine. While doing hardware configuration test, the kind of hardware to be tested is spelled out and there are several computer hardware and peripherals which make it next to impossible to execute all the tests.

COMPATIBILITY TESTING

Checking the functionality of an application on different software, hardware platforms, network, and browsers is known as compatibility testing. Once the application is stable, we moved it to the production, it may be used or accessed by multiple users on the different platforms, and they may face some compatibility issues, to avoid these issues, we do one round of compatibility testing.



Types of Compatibility testing

1. Software
2. Hardware
3. Network
4. Mobile

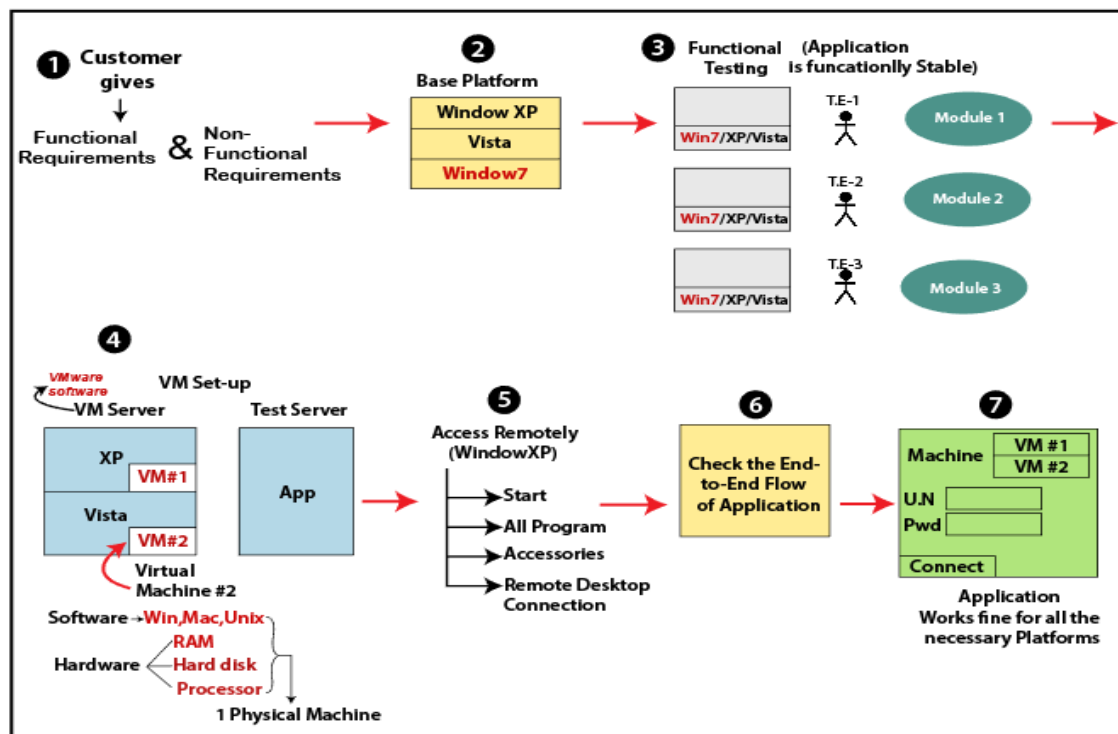
Software- Software means different operating systems (Linux, Window, and Mac) and also check the software compatibility on the various versions of the operating systems like Win98, Window 7, Window 10, Vista, Window XP, Window 8, UNIX, Ubuntu, and Mac.

Hardware - The application is compatible with different sizes such as RAM, hard disk, processor, and the graphic card, etc.

Mobile - Check that the application is compatible with mobile platforms such as iOS, Android, etc.

Network - Checking the compatibility of the software in the different network parameters such as operating speed, bandwidth, and capacity.

Compatibility testing process



1. Firstly, the customer will give the functional requirement as well as a non-functional requirement.
2. After getting the non-functional requirement, the one base platform will decide according to the most commonly used platform.

3. After that, the test engineer will start functional testing on the base platform until the application is functional stable.
4. We have to test the application on a different platform, so for this, we have a VMware software.
5. For compatibility testing, we will use VM Server where we install all necessary operating system and browser, and access that server using Remote Desktop connection.
6. We access VMware set up remotely, and after that, the test engineer will do one round of compatibility testing on the platform and check the end-to-end flow.
7. The end-to-end flow will go on until the application is stable, and applications work fine for all necessary platforms and handover to the customer.

USABILITY TESTING

Usability Testing is a significant type of software testing technique, which comes under the non-functional testing. It is primarily used in user-centered interaction design in order to check the usability or ease of using a software product. The implementation of usability testing requires an understanding of the application, as it is extensive testing.

Characteristics:

1. Easy to understand
2. Easy to access
3. Look and feel
4. Faster to Access
5. Effective Navigation
6. Good Error Handling

Features

- It is an essential type of non-functional testing technique, which comes under the black-box testing technique in software testing.
- Usability testing is performed throughout the system and acceptance testing levels.

- Generally, usability testing is implemented in the early stage of the Software Development Life Cycle (SDLC).
- The execution of usability testing offers more visibility on the prospects of the end-users.
- The usability testing makes sure that the software product meets its planned purpose.
- It also helps us to find many usability errors in the specified software product.
- Usability testing mainly tests the user-friendliness, usefulness, traceability, usability, and desirability of the final product.
- It offers direct input on how real-users use the software/application.
- The usability testing includes the systematic executions of the product's usability under a measured environment.

Usability Testing Process

The Process of usability testing is completed into few significant steps. This process will assist us in providing and creating different results for all the issues identified during the execution of testing.

In real-time, usability testing tests the application's behavior from the user's perspective even though it is a time-consuming process, providing the tester the most precise outcomes from actual testing.

And that gives us an idea of the errors/flaws in our product and helps us distinctly before installing it on the server.

The usability testing process follows a precise set of steps to help the team get the detailed and helpful response from the end-users.

Usability Testing Process



Step1: Planning

The first step of usability testing is planning, where the team makes the test plan and generates some document samples that help the testing team complete the usability testing tasks.

The objective of the usability test is governed in the planning step. Here, the aim is not to have the volunteers sit in front of our application and recording their activities, but we need to fix the crucial features and elements of the system.

We need to give the tasks to our test engineers who are familiar with these crucial features. And the usability testing method, number, and demographics of usability test engineers, test report formats are also fixed throughout the planning phase.

Step2: Team Recruiting

Once the planning phase is completed, we will move to the next step of usability testing, which is **team recruiting**.

As the name suggests itself, here, we will hire or recruit the end-user delegates and the participants or the test engineers as per the budget and density of the product.

These representatives or test engineers are prepared to sit across the test sessions and validate the correctness and usability of the product.

Primarily, the selection of these test engineers is based on the necessity of testing along with the number of persons mentioned in the test plan.

As soon as the hiring of a test engineer is achieved, the team is appointed to the particular responsibilities and jobs.

Step3: Test Execution

Once the planning and team recruiting steps have been completed successfully. We will proceed to the next step, which is **Test Execution**.

In the test execution step, the test engineers execute the usability testing and implement their assigned responsibilities. In this situation, the users' needs to test the product to find irregularity, if any, and also record them correctly.

Step4: Test Result Documentation

The **test result documentation** step includes the results based on the **test execution** step and then proceeds for further analysis.

Step5: Data Analysis

Once the test result documentation is done, we will move to the next step of the usability testing process, i.e., **Data Analysis**.

The response or the feedback is obtained from usability testing evaluation in the data analysis phase. And the outcomes are classified, and the patterns are acknowledged.

In this step, the data from usability tests are wholly evaluated to get expressive implications and help us provide actionable suggestions to enhance the overall usability of our product.

Step6: Reporting

After performing all the above steps successfully, we will finally reach the last step of the usability testing process which is named as **Reporting**.

Example

We are taking one **banking application** where we produce the application for the manager.

Now, if the end-user (manager) starts using the application in front of the test engineers

Suppose two test engineer sits at the back of the end-user while he/she is using the application and takes the report of the defect as a developer to check whether the end-user is using the application in a right way or not.

And the end-user (manager) will check the application step by step because he/she knows that the Test engineer is watching him/her.

Sometimes Test engineer has to do usability testing for the following reasons:

- There is no money to spend on usability testing.
- Do not want to outsource to another company.

SECURITY TESTING

Security Testing is a type of Software Testing that uncovers vulnerabilities in the system and determines that the data and resources of the system are protected from possible intruders. It ensures that the software system and application are free from any threats or risks that can cause a loss. Security testing of any system is focused on finding all possible loopholes and weaknesses of the system that might result in the loss of information or reputation of the organization.

Types of Security Testing

1. Vulnerability Scanning

It is a type of testing that uses automated tools to scan the system for known vulnerabilities and weaknesses. It aims to detect patterns of vulnerabilities that are commonly exploited by attackers. By performing vulnerability scans regularly, organizations can proactively address these vulnerabilities before they become security risks.

2. Security Scanning

It involves identifying weaknesses in the network or system and then providing solutions to mitigate these risks. It can be performed either manually or automatically, depending on the complexity of the system. This process helps uncover potential weak points that could be exploited by attackers, allowing for early intervention to secure the system.

3. Penetration Testing

It simulates an attack from a malicious hacker to identify vulnerabilities in the system. This type of testing helps organizations understand how an attacker might exploit weaknesses in the system. By performing penetration testing, organizations can see their system from an attacker's perspective and fix vulnerabilities before they are exploited in a real-world attack.

4. Risk Assessment

It involves analyzing the security risks that could affect the organization. Risks are categorized as low, medium, or high, and this testing suggests controls and measures to minimize those risks. Risk assessment helps prioritize actions by identifying the most critical threats and focusing efforts on addressing them first, ultimately improving the overall security posture of the system.

5. Security Auditing

These is an internal inspection of the system to identify security defects. This can involve reviewing system configurations, checking for weaknesses in the code, or conducting a line-by-line inspection of the application's source code. Security audits ensure that all security standards and protocols are being followed and identify any gaps that need to be addressed.

6. Ethical Hacking

It also known as white-hat hacking, is when security professionals are hired to simulate attacks on the system to identify vulnerabilities. Unlike malicious hacking, ethical hacking is done with the organization's consent to help improve system security. Ethical hackers use the same techniques as malicious hackers to uncover weaknesses in the system, but their goal is to fix those flaws before they can be exploited by actual attackers.

7. Posture Assessment

These will combines security scanning, ethical hacking, and risk assessments to provide an overall view of the system's security. It gives a comprehensive evaluation of the system's security by integrating multiple testing methods, ensuring that no part of the security infrastructure is overlooked. This assessment helps organizations understand their security readiness and take necessary actions to strengthen their defenses.

8. Application Security Testing

These testing focuses specifically on identifying vulnerabilities within the application itself. This includes examining the application's code, configurations, and dependencies to identify

flaws that could lead to security breaches. Regular application security testing ensures that the software does not contain any weaknesses that could be exploited by attackers.

9. Network Security Testing

In these testing targets the vulnerabilities in the network infrastructure, such as firewalls, routers, and other network devices. This testing is crucial for identifying weaknesses that could allow unauthorized access to the system. Network security testing helps ensure that the communication pathways between devices are secure and that sensitive data is protected from cyber threats.

10. Social Engineering Testing

Social Engineering Testing simulates phishing, baiting, or other manipulative techniques used to exploit human behavior to gain unauthorized access. This type of testing focuses on the human element of security, ensuring that employees are aware of potential threats and know how to protect themselves from such attacks.

Goal of Security Testing

- To identify the threats in the system and measure the potential vulnerabilities of the system.
- To help in detecting every possible security risk in the system and help developers fix security problems through coding.
- The goal of security testing is to identify vulnerabilities and potential threats in a system or application and to ensure that the system is protected against unauthorized access, data breaches, and other security-related issues.
- Identify vulnerabilities: Security testing helps identify vulnerabilities in the system, such as weak passwords, unpatched software, and misconfigured systems that could be exploited by attackers.
- Evaluate the system's ability to withstand an attack: Security testing evaluates the system's ability to withstand different types of attacks, such as network attacks, social engineering attacks, and application-level attacks.
- Ensure compliance: Security testing helps ensure that the system meets relevant security standards and regulations, such as HIPAA, PCI DSS, and SOC2.

- **Provide a comprehensive security assessment:** Security testing provides a comprehensive assessment of the system's security posture, including the identification of vulnerabilities, the evaluation of the system's ability to withstand an attack, and compliance with relevant security standards.
- **Help organizations prepare for potential security incidents:** Security testing helps organizations understand the potential risks and vulnerabilities that they face, enabling them to prepare for and respond to potential security incidents.
- **Identify and fix potential security issues before deployment to production:** Security testing helps identify and fix security issues before the system is deployed to production. This helps reduce the risk of a security incident occurring in a production environment.

Principle of Security Testing

1. **Confidentiality:** Verifies that sensitive data is only accessible to authorized users, often through encryption and access control mechanisms.
2. **Integrity:** Verifies that data remains unchanged and unaltered during storage or transmission. Hash functions and checksums are commonly used to guarantee integrity.
3. **Authentication:** Ensures that only authorized users can access the system. This involves testing password policies, multi-factor authentication (MFA), and identity verification mechanisms.
4. **Authorization:** Verifies that authenticated users can only access the resources and data they are authorized to use, through mechanisms such as role-based or attribute-based access control (RBAC and ABAC).
5. **Availability:** Ensures that the system remains functional and accessible, even under heavy traffic or during a cyberattack, such as a Distributed Denial of Service (DDoS) attack.
6. **Non-Repudiation:** Ensures that users cannot deny their actions in the system. Digital signatures, audit logs, and transaction records are commonly used to guarantee non-repudiation.
7. **Resilience:** Verifies the system's ability to recover from incidents, such as system crashes or attacks, by evaluating backup systems and response protocols.

Advantages

- Identifying vulnerabilities
- Improving system security
- Ensuring compliance
- Reducing risk
- Improving incident response

TESTING DOCUMENTATION

Testing documents are prepared at different stages. These documents are discussed as follows.

1. Before Testing:

- **SRS document** – Functional Requirements document.
- **Test Policy document** – It means the product must be tested far before release.
- **Test Strategy document** – It mentions detailed aspects of test the team, responsibility matrix, and rights/responsibilities of the test manager and test engineer.
- **Traceability Matrix document** – This is SDLC document, that is related to the requirements-gathering process. As new requirements come, they are added to this matrix. They can be traced forward and backward. These matrices help testers know the source of the requirement.

2. During Testing:

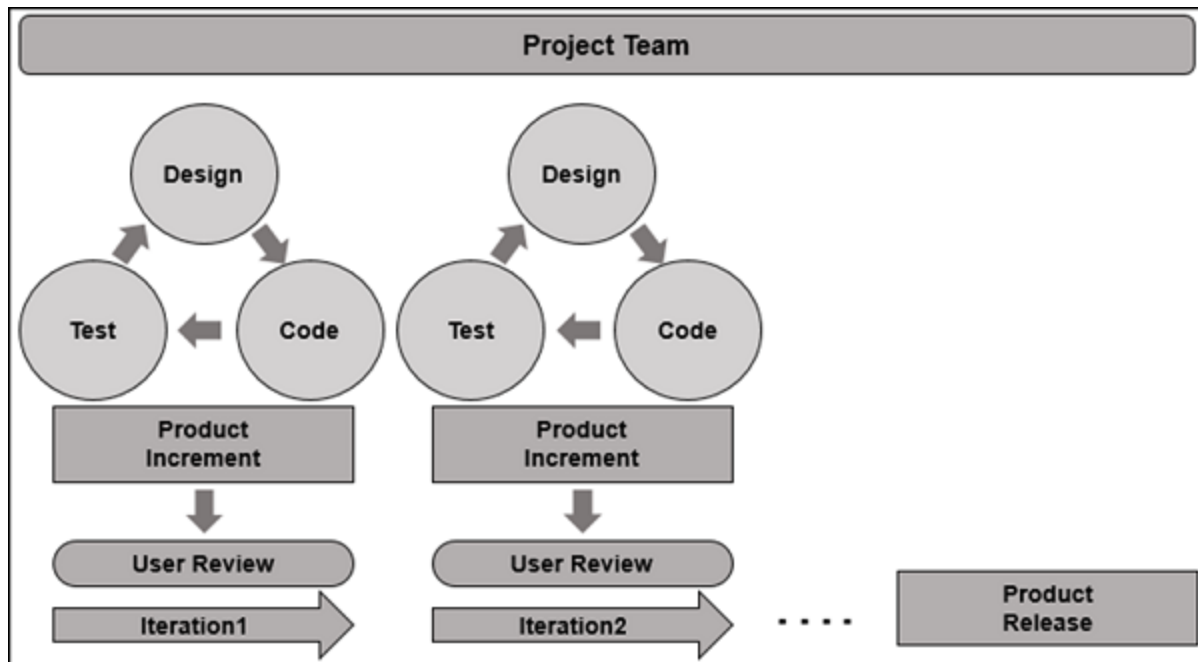
- **Test Case document** – It contains the list of to-be tests. It includes various testing like Unit test plan, Integration test plan, System test plan and Acceptance test plan.
- **Test description** – It is a detailed description of all test cases and procedures for executing them.
- **Test case report** – It contains a test case report resulting from the test.
- **Test logs** – It contains test logs for every test case report.

3. After Testing:

After testing, only the test summary remains which is a collective analysis of all test reports and logs. The software is released under the version control system if it is ready to launch. It summarizes and concludes whether the software is ready to launch.

TESTING IN THE AGILE ENVIRONMENT

- Agile is an iterative development methodology, where the entire project team participates in all the activities.
- The requirements evolve as the iterations progress, through collaboration between the customer and the self-organizing teams.
- As Coding and Testing are done interactively and incrementally, during the course of development, the end-product would be of quality and ensures customer requirements.
- Every iteration results in an integrated working product increment and is delivered for User Acceptance Testing.
- The customer feedback thus obtained would be an input to the next / subsequent Iterations.



Continuous Integration, Continuous Quality

- Continuous Integration is the key for Agile Development success. Integrate frequently, at least daily such that you are ready for a release as and when required.
- Testing in Agile becomes an essential component of all the phases of the development, ensuring continuous quality of the product.
- Constant feedback from everyone involved in the project adds to the quality of the product.
- In Agile, communication is given utmost importance and the customer requests are received as and when necessary.
- This gives the satisfaction to the customer that all the inputs are considered and working quality product is available throughout the development.

Agile Testing Methodologies

1. The testing practices are well defined for every project, whether Agile or not, to deliver quality products.
2. Traditional Testing principles are quite often used in Agile Testing. One of them is Early Testing that focuses on –

- Writing Test Cases to express the behavior of the system.
- Early Defect Prevention, detection and removal.
- Ensuring that the right test types are run at the right time and as part of the right test level.

The other commonly used Agile Testing Methodologies are –

- Test-Driven Development (TDD) – Test-Driven Development (TDD) is based on coding guided by tests.
- Acceptance Test-Driven Development (ATDD) – Acceptance Test-Driven Development (ATDD) is based on communication between the customers, developers and testers and driven by pre-defined Acceptance Criteria and Acceptance Test Cases.
- Behavior-Driven Development (BDD) – In Behavior-Driven Development (BDD) testing is based on the expected behavior of the software being developed.

TESTING THE WEB AND MOBILE APPLICATIONS:

What is a Web Application?

A web app is an application that runs on a web browser and is accessed over the internet, without the need to download it.

Example: Gmail, Google Docs

What is a Mobile Application?

A mobile app is an application designed for smartphones and tablets, usually available for download through app stores.

Example: WhatsApp, Uber

Types of Web Applications

- Static Web Applications
- Dynamic Web Applications
- E-Commerce Applications
- Single-Page Applications (SPAs)

- Progressive Web Applications (PWAs)
- Content Management Systems (CMS)
- Enterprise Web Applications
- Portal Web Applications

1. Static Web Applications

Simple web applications that primarily display fixed content and do not interact dynamically with users.

Technology Stack: HTML, CSS, and JavaScript (basic level).

Features:

- Easy to build and deploy.
- Limited interactivity.

Examples: Personal blogs, portfolio websites.

2. Dynamic Web Applications

Applications that generate content dynamically based on user interactions and data.

Technology Stack: Backend languages (for example, PHP, Python, Ruby), frameworks, and databases.

Features:

- Real-time data updates.
- Requires a backend for database interaction.

Examples: Social media platforms, e-commerce sites.

3. E-Commerce Applications

Web applications specifically designed for online shopping and transactions.

Technology Stack: Backend (for example, Node.js, Ruby on Rails), frontend frameworks (React, Angular), and payment gateways.

Features:

- Product catalogs, shopping carts, payment integration.
- Secure and scalable design.

Examples: Amazon, eBay, Shopify.

4. Single-Page Applications (SPAs)

Applications that load a single HTML page and dynamically update content without reloading the page.

Technology Stack: Frameworks like React, Angular, Vue.js.

Features:

- Fast and seamless user experience.
- Heavy reliance on JavaScript.

Examples: Gmail, Google Maps, Trello.

5. Progressive Web Applications (PWAs)

Web applications that behave like native mobile apps while being accessible through browsers.

Technology Stack: HTML, CSS, JavaScript, Service Workers.

Features:

- Offline capabilities.
- Cross-platform compatibility.
- Installation as an app-like icon on devices.

Examples: Twitter Lite, Starbucks PWA.

6. Content Management Systems (CMS)

Applications that allow users to create, edit, and manage digital content without coding knowledge.

Technology Stack: WordPress, Joomla, Drupal.

Features:

- User-friendly interfaces.
- Customizable templates.

Examples: News websites, blogs, business websites.

7. Enterprise Web Applications

Applications built to meet the needs of organizations for business processes, communication, and productivity.

Technology Stack: Backend (Java, .NET), frontend (React, Angular), and robust databases.

Features:

- Scalable and secure.
- Integration with enterprise systems.

Examples: ERP systems, CRM platforms like Salesforce.

8. Portal Web Applications

Applications that provide a gateway to access diverse services, content, or information.

Technology Stack: Backend languages and frameworks for user authentication and data management.

Features:

- Role-based access.
- Customizable dashboards.

Examples: Student portals, employee portals.

Types of Mobile Applications

1. Native Applications

These apps are built specifically for a mobile operating system (for example, Android or iOS) using platform-specific programming languages.

Technology Stack:

- **Android:** Java, Kotlin

- **iOS:** Swift, Objective-C

Advantages:

- High performance and responsiveness.
- Full access to device features (for example, camera, GPS).
- Better user experience with platform-specific UI/UX.

Examples: WhatsApp, Google Maps, and Facebook (native versions).

2. Mobile Web Applications

These apps run on mobile browsers and are not installed on devices. They use responsive design to function across different screen sizes.

Technology Stack: HTML, CSS, JavaScript, and frameworks like ReactJS, AngularJS.

Advantages:

- Cross-platform compatibility (accessible via browsers).
- No installation is required, reducing storage usage.
- Easier to update and maintain.

Examples: Twitter Lite, Google Search on mobile browsers.

3. Hybrid Applications

Hybrid apps combine elements of both native and web applications, typically wrapped in a native shell to allow access to device features.

Technology Stack: Frameworks like React Native, Flutter, Ionic, Cordova.

Advantages:

- Cross-platform development with a single codebase.
- Faster development compared to native apps.
- Can access device features using plugins.

Examples: Instagram, Uber, Evernote.

Tools or Frameworks for Web Application Testing

- ✓ BrowserStack
- ✓ Selenium
- ✓ Jira
- ✓ Cypress
- ✓ Postman
- ✓ Playwright
- ✓ Tools or Frameworks Used for Mobile App Testing
- ✓ BrowserStack
- ✓ Appium
- ✓ Espresso
- ✓ XCUITest
- ✓ Calabash

Difference between Web and Mobile Applications

Aspect	Web Applications	Mobile Applications
--------	------------------	---------------------

Platform Dependency	Runs on web browsers; platform-independent.	Platform-specific (Native) or cross-platform (Hybrid).
Installation	No installation required; accessed via URL.	Requires installation from app stores (for example, Google Play, App Store).
Internet Dependency	Requires an internet connection unless cached (for example, PWAs).	Can work offline (for native apps) depending on the design.
Development	Built using web technologies (HTML, CSS, JavaScript, etc.).	Developed using platform-specific languages (Swift, Kotlin) or frameworks like Flutter.
User Experience (UX)	Limited by browser capabilities; relies on responsive design.	Richer and more intuitive, utilizing device features like GPS and cameras.
Performance	Dependent on browser and internet speed.	Generally faster, with direct access to hardware resources.
Updates	Instantly updated on the server side.	Requires users to download updates from the app store.

Device Features	Limited access to device features (some via browser APIs).	Full access to device hardware like cameras, accelerometers, etc.
Offline Capability	Limited; mostly requires a stable internet connection.	Better offline support (for example, native apps store data locally).
Cost	Cheaper to develop and maintain as it works across platforms.	More expensive, especially for separate native apps for each OS.
Examples	Google Docs (web version), Amazon (website).	WhatsApp, Instagram, Uber.

Tools or Frameworks for Web Application Testing

Selenium

Selenium is a widely used open-source tool for automating web browser testing. It supports multiple browsers, platforms, and programming languages.

Key Features:

- Supports cross-browser testing on Chrome, Firefox, Safari, etc.
- Compatible with multiple programming languages like Java, Python, and C#.
- Provides Selenium Grid for parallel test execution.

- Flexible integration with CI/CD tools like Jenkins.

Pros:

- Highly customizable and extendable.
- Large community support and extensive documentation.
- Open-source with no licensing cost.

Cons:

- Requires advanced programming skills.
- Limited support for modern web app features like shadow DOM.
- Initial setup can be complex

What is Selenium?

Selenium is an open-source framework designed for automating web browsers. It enables users to test website functionality across different browsers, ensuring consistency and compatibility.

Key features of Selenium:

- Supports functional and cross-browser testing to verify seamless performance across multiple browsers.
- Allows test script creation in various programming languages, including Java, Python, JavaScript, Ruby, C#, and more.
- Easily integrates with testing frameworks like TestNG, JUnit, and Cucumber, enhancing flexibility and extensibility.

Importance of Testing in Selenium

Manual testing can be time-consuming and prone to human errors. Selenium Automation allows tests to be executed quickly and accurately, reducing the likelihood of human mistakes and ensuring consistent test results. Selenium allows developers and testers to automate the testing of web applications across different browsers and platforms.

1. **Language Support:** Selenium allows you to create test scripts in different languages like Ruby, Java, PHP, Perl, Python, JavaScript, and C#, among others.
2. **Browser Support:** Selenium enables you to test your website on different browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Internet Explorer (IE), etc.
3. **Scalability:** Automated testing with Selenium can easily scale to cover a wide range of test cases, scenarios, and user interactions. This scalability ensures maximum test coverage of the application's functionality.
4. **Reusable Test Scripts:** Selenium allows testers to create reusable test scripts that can be used across different test cases and projects. This reusability saves time and effort in test script creation and maintenance.
5. **Parallel Testing:** Selenium supports parallel test execution, allowing multiple tests to run concurrently. This helps reduce the overall testing time, making the development process more efficient.
6. **Documentation and Reporting:** Selenium provides detailed test execution logs and reports, making it easier to track test results and identify areas that require attention.
7. **User Experience Testing:** Selenium can simulate user interactions and behavior, allowing testers to assess the user experience and ensure that the application is intuitive and user-friendly.
8. **Continuous Integration and Continuous Deployment (CI/CD):** Selenium can be integrated into CI/CD pipelines to automate the testing of each code change. This

integration helps identify and address issues earlier in the development cycle, allowing for faster and more reliable releases.

Components of Selenium

Selenium Suite has 4 components namely:

1. Selenium IDE
2. Selenium RC
3. Selenium WebDriver
4. Selenium Grid

Selenium IDE

Selenium IDE is a Chrome and Firefox plugin. The primary use of a Selenium IDE is to record user interactions such as clicks, selections etc in the browser and plays them back as automated tests. It then generates the test script (of the automated tests) in programming languages like C#, Java, Python, and Ruby and Selenese (Selenium's own scripting language).

Selenium IDE helps in:

- Creating automated test scripts and validating them at speed
- Identifying and highlighting errors during the replay of interactions
- Cross Browser Testing

Selenium RC

Selenium RC was built to automate the testing of web applications by simulating user interactions across different browsers and platforms. It provided a way to browser automation remotely and execute test scripts written in various programming languages.

Selenium WebDriver

Selenium WebDriver is a powerful and enhanced version of Selenium RC which was developed to overcome the limitations of Selenium RC. WebDriver communicates with browsers directly with the help of browser-specific native methods, thereby completely eliminating the need of Selenium RC. WebDriver works closely with Selenium IDE and Selenium Grid resulting in reliable test execution at speed and scale.

Selenium Grid

Selenium Grid is a smart proxy server that allows QAs to run tests in parallel on multiple machines. This is done by routing commands to remote web browser instances, where one server acts as the hub. This hub routes test commands that are in JSON format to multiple registered Grid nodes.

Browsers Supported in Selenium WebDriver

Selenium supports these Browsers:

- Google Chrome
- Mozilla Firefox
- Safari
- Internet Explorer
- Microsoft Edge

Selenium WebDriver is a powerful browser automation tool designed to test web applications efficiently. Unlike Selenium RC, WebDriver directly interacts with browser elements, making it faster and more reliable. It supports multiple browsers, platforms, and programming languages, enabling comprehensive testing across different environments.

When to use Selenium WebDriver

Selenium WebDriver is primarily designed to test web applications.

- **Functional Testing:** Test the functionality of web applications by automating user interactions such as clicking buttons, filling out forms, navigating pages, and verifying expected outcomes.
- **Cross-Browser Testing:** Test web application for consistency across different browsers and browser versions (e.g., Chrome, Firefox, Edge, Safari)
- **Cross-Platform Testing:** Test the web application on different operating systems such as Windows, macOS, and Linux.
- **Regression Testing:** Test the existing functionality of the web app, whenever a new feature is introduced.
- **Data-Driven Testing:** Execute the same test script with different sets of input data to validate different scenarios.
- **Parallel Testing:** Selenium WebDriver can be integrated with tools like Selenium Grid to execute tests in parallel across multiple browsers and platforms, reducing testing time.
- **Complex User Flows:** Simulate complex user workflows or interactions that need to be tested repeatedly.
- **Integration with Continuous Integration (CI) Pipelines:** Selenium WebDriver can be seamlessly integrated into your CI/CD pipeline, enabling automated testing with every code change and ensuring code quality.
- **UI/UX Testing:** Automate user interface (UI) testing, ensuring that the visual elements and layout of your application are consistent.

- **End-to-End Testing:** Selenium WebDriver can be used for end-to-end testing, where you simulate real user interactions across different parts of the application to ensure a seamless user experience.
- **Page Object Model (POM) Implementation:** If you adopt the Page Object Model (POM) design pattern, WebDriver can be used effectively to encapsulate the interactions and elements of different web pages, leading to more maintainable and modular test scripts.
- **Complex Scenarios:** When dealing with complex scenarios such as handling alerts, pop-ups, iframes, and dynamic content, Selenium WebDriver provides the flexibility to address these challenges.
- **Performance Testing:** While not its primary use, WebDriver can be used to simulate user load and interactions for basic performance testing.

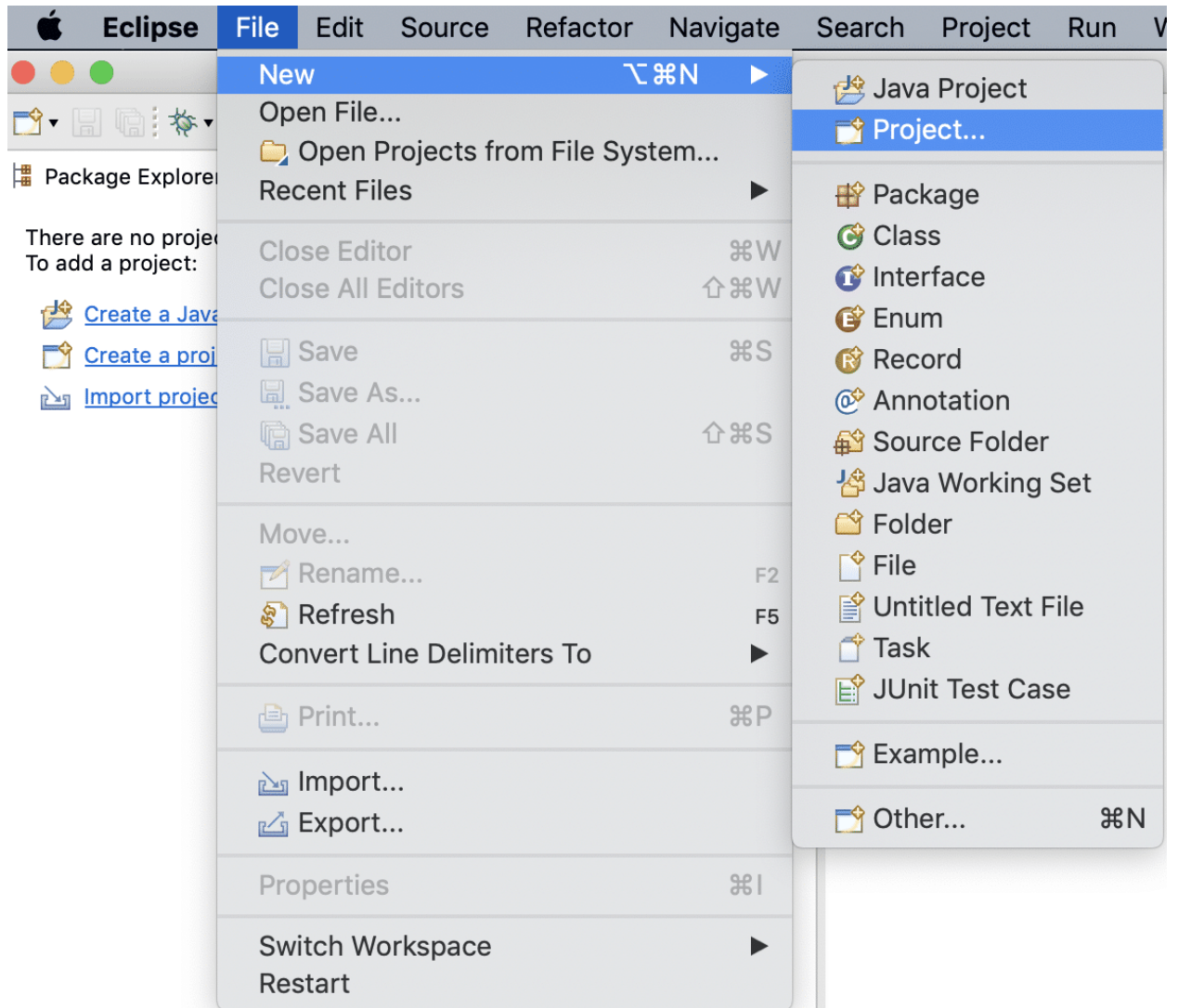
Configure Selenium WebDriver with Eclipse

Eclipse is one of the most used IDE by the java developers. In a project, all of these IDEs provide their ways to integrate and use functionalities of those libraries in your project. All the Java libraries are bundled and provided as JARS (Java Archives), which can be included in a project and then the project can invoke the needed functions of various classes bundled in those jar files. Similarly, Selenium WebDriver for Java also provides a set of jar files, which we need to include in the project to access and use the functionalities provided by Selenium WebDriver.

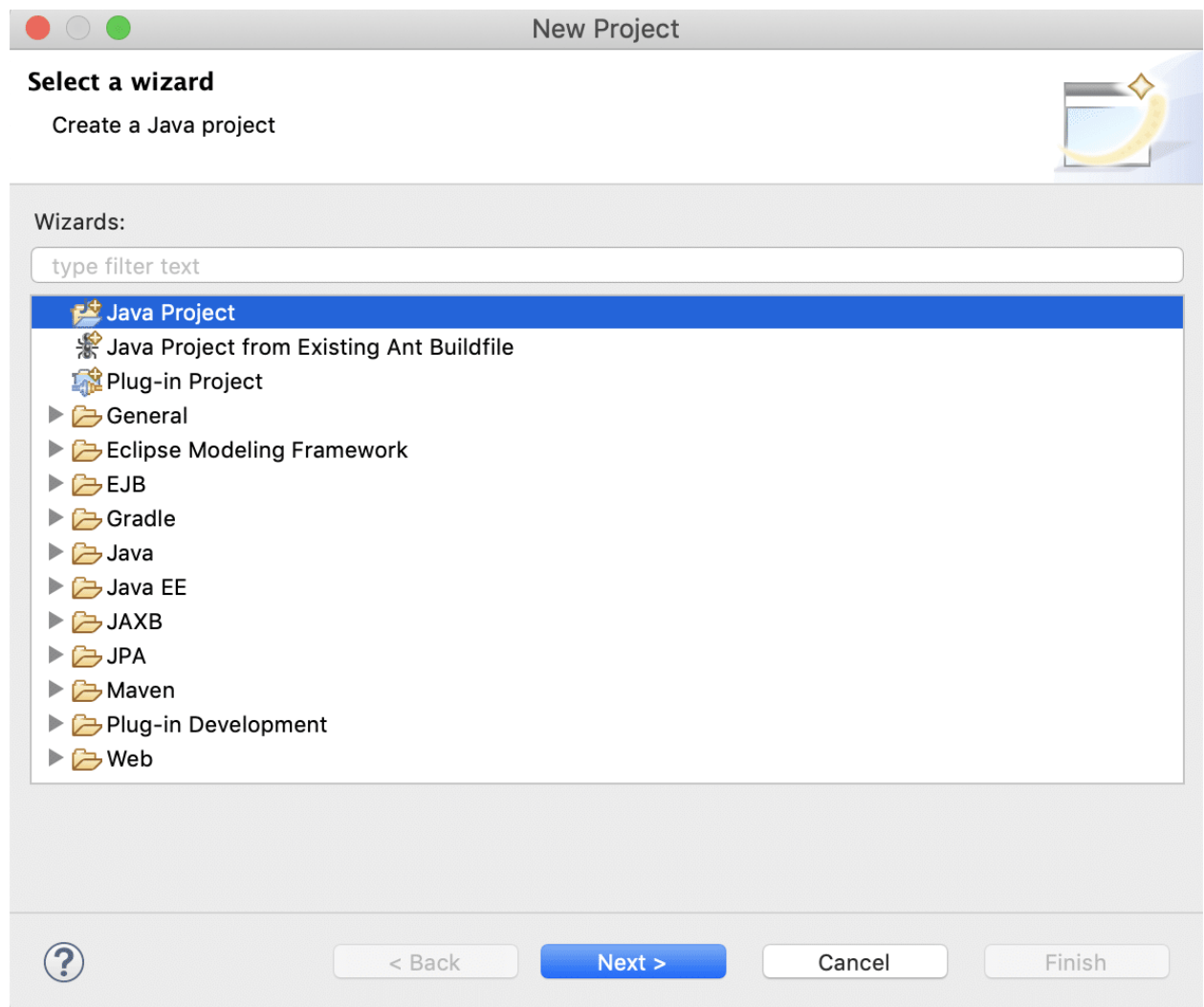
- How to launch Eclipse and create a workspace?
 - o Create a new project in Eclipse
 - o Create a new package under the project
 - o Create a new class under the package
 - o Add Selenium WebDriver Jars to the project

Create a new project in Eclipse?

1. Navigate to **File > New > Project**. Alternatively, you can directly select the **Java Project** option under the **New** menu.



2. It will open the "**New Project** " wizard. From the **New Project** dialogue box, select **Java Project** and click on **Next**.



3. On the "**Create a Java Project** " wizard, specify a name to the Java project and select other options as shown below:

New Java Project

Create a Java Project

Create a Java project in the workspace or in an external location.

Project name:

☒ Use default location

Location: [Browse...](#)

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE 'Java SE 8 [1.8.0_181]' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

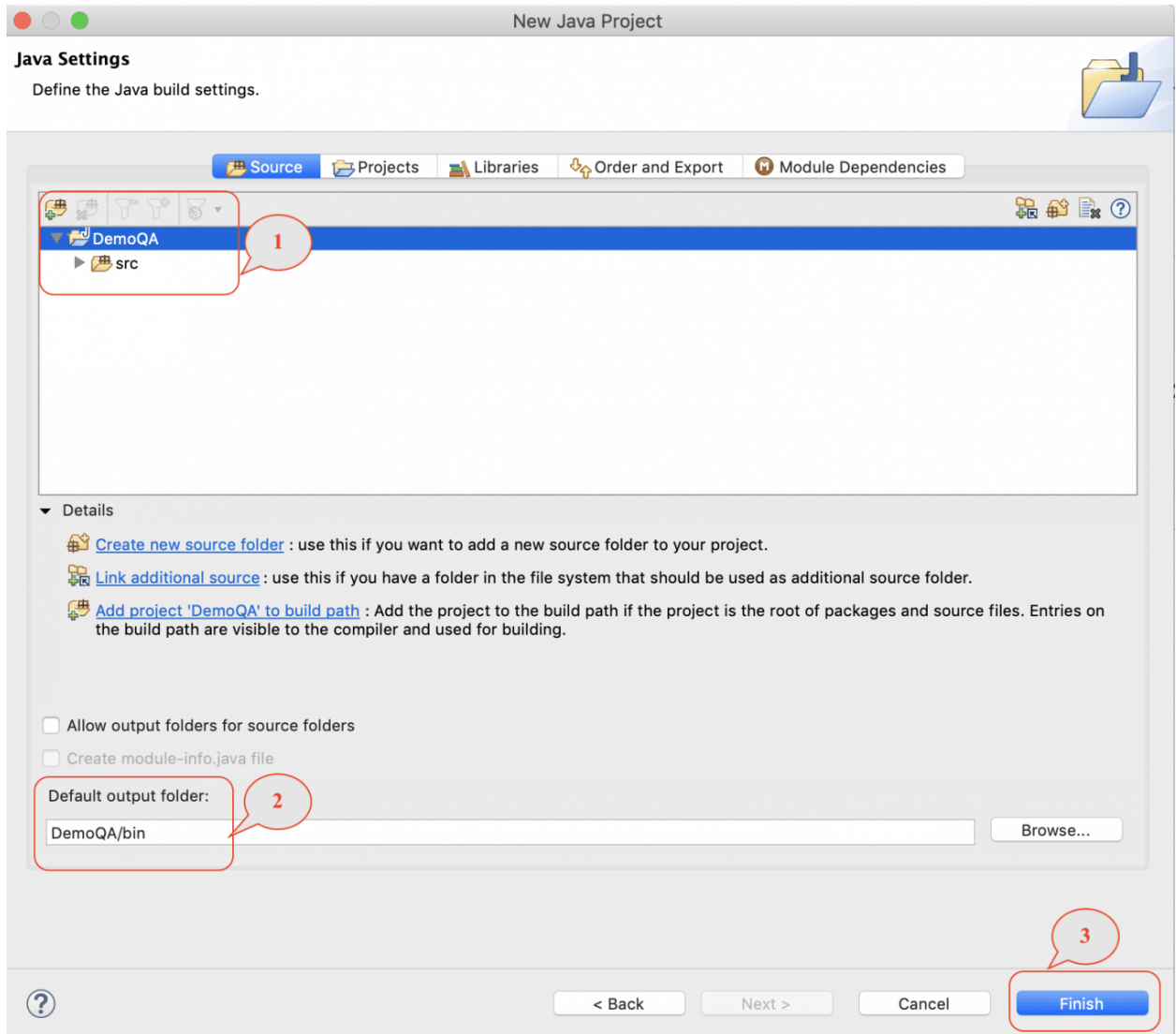
Working sets

☐ Add project to working sets [New...](#)

Working sets: [Select...](#)

[?](#) [< Back](#) [Next >](#) [Cancel](#) [Finish](#)

After completing all the configurations as mentioned above, either you can click on the "**Finish**" button to close the dialogue box or click on the "**Next**" button, as heightened by **marker 5**, which will open the dialogue box showing the folders for the project, sources, and classes as shown below:



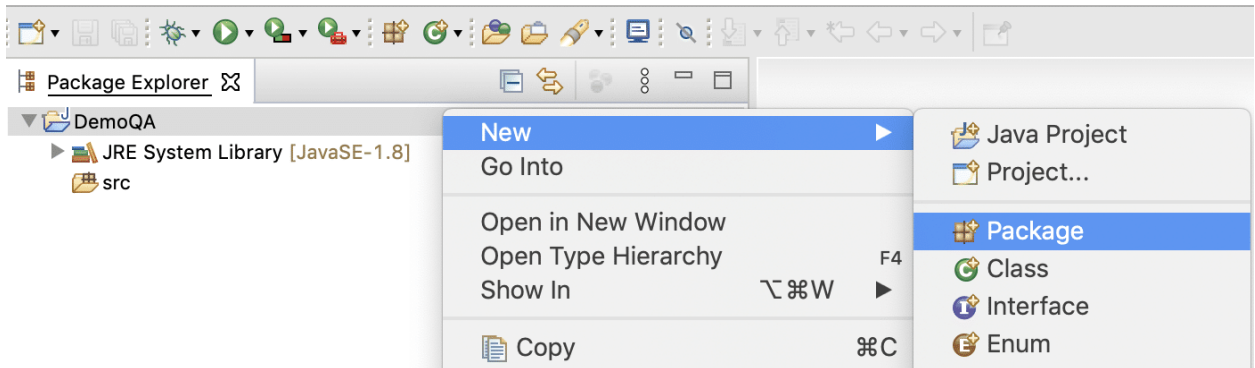
- As we can see in the above screenshot, Eclipse created a folder "**src**" (highlighted as 1). It will store all the source files and created a folder "**bin**" (highlighted as 2), which will store all the compiled class files. Click on the "**Finish**" button (highlighted as 3) to close the dialogue box.
- 4. Clicking on the "**Finish**" button will show a pop-up to select the Java perspective, which is more around aligning the different components of a Java project in the left Explorer box of Eclipse.

Once we finish creating a Java project in Eclipse, the next step is to create a **package** under the Project.

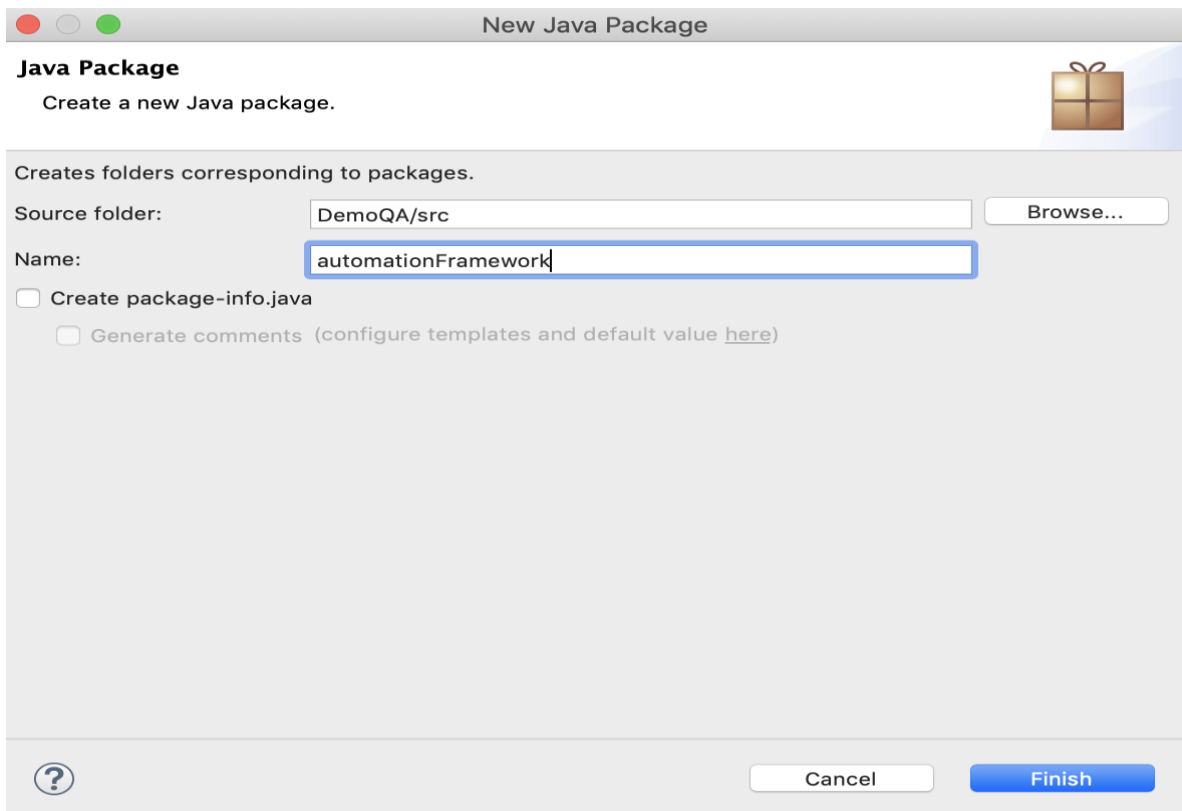
Create a new package under the project

To create a new package under the newly created project "**DemoQA**", follow the steps as mentioned below:

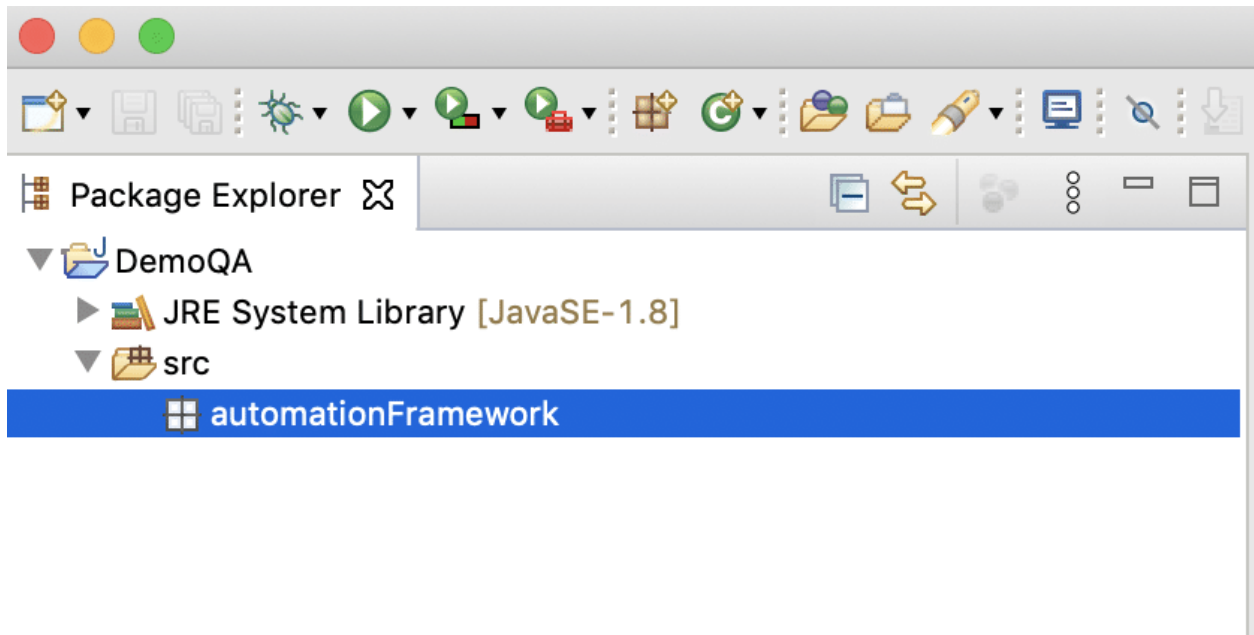
1. Right-click on your project name '*DemoQA*' and navigate to **New > Package**.



2. Give a name to your package, say, 'automationFramework ' and click on the **Finish** button.



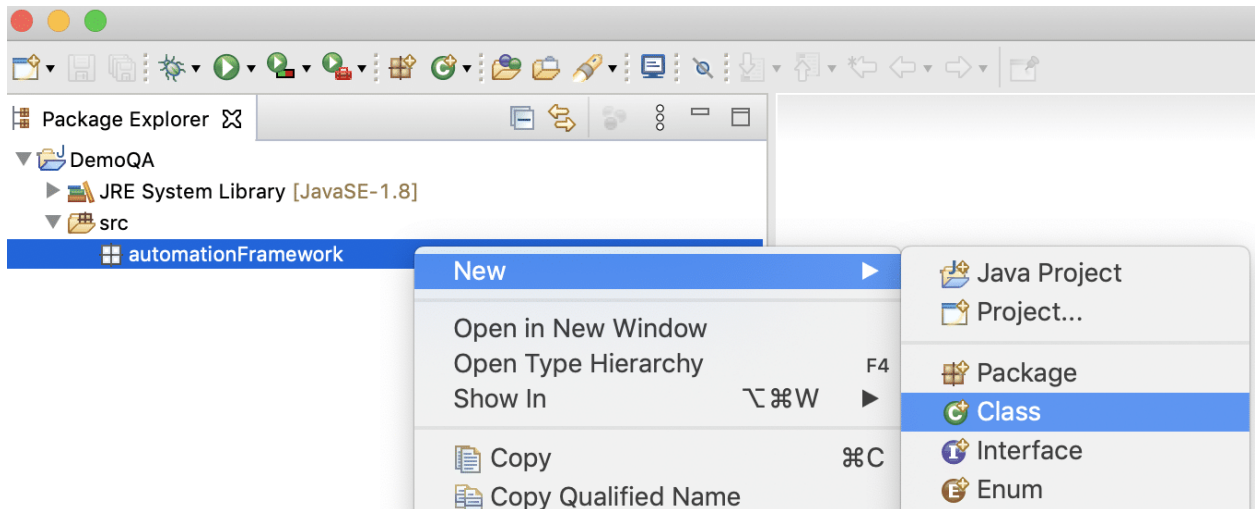
3. Once the package creation is successful, it will reflect under the **src** folder of the project in the **Eclipse Project Explorer** windows, as shown below:



Once you finish creating a package under the project, the next step is to create a **Java class** under the package.

Create a new class under the package

1. Right-click on the package 'automationFramework ' and navigate to **New > Class**.



2. Once you click on the "**Class**" option, You will see a **New Java Class** wizard as shown below:

New Java Class

Java Class

Create a new Java class.

Source folder: DemoQA/src 1 Browse...

Package: automationFramework Browse...

☐ Enclosing type: Browse...

Name: FirstTestCase 2 3

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: java.lang.Object 4 Browse...

Interfaces: 5 Add... Remove

Which method stubs would you like to create?

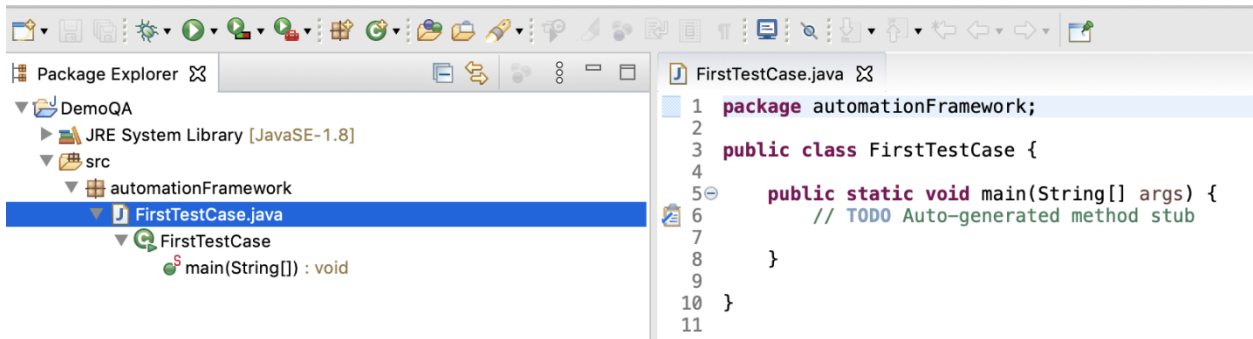
- ☒ public static void main(String[] args)
- ☐ Constructors from superclass
- ☐ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

? Cancel 6 Finish

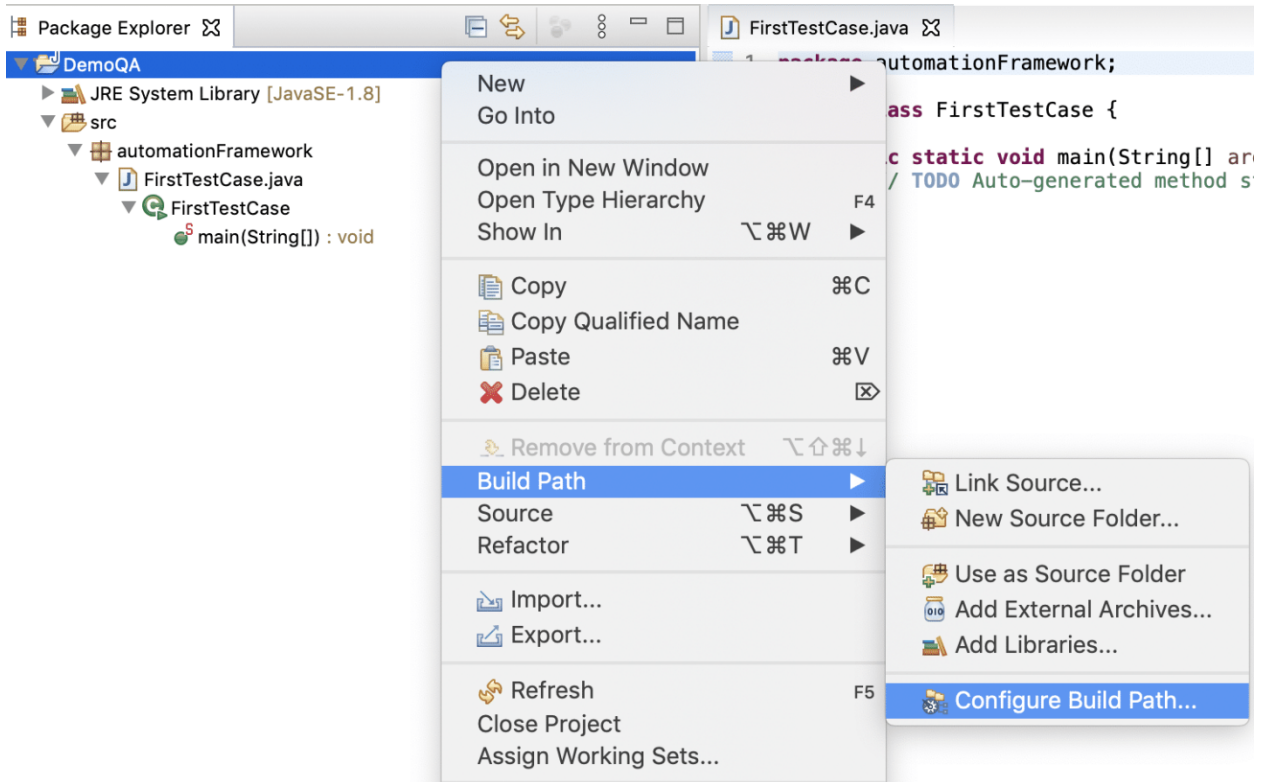
3. Once we create the Java class successfully. It will reflect under the "**Project Explorer**" window of Eclipse as shown below:



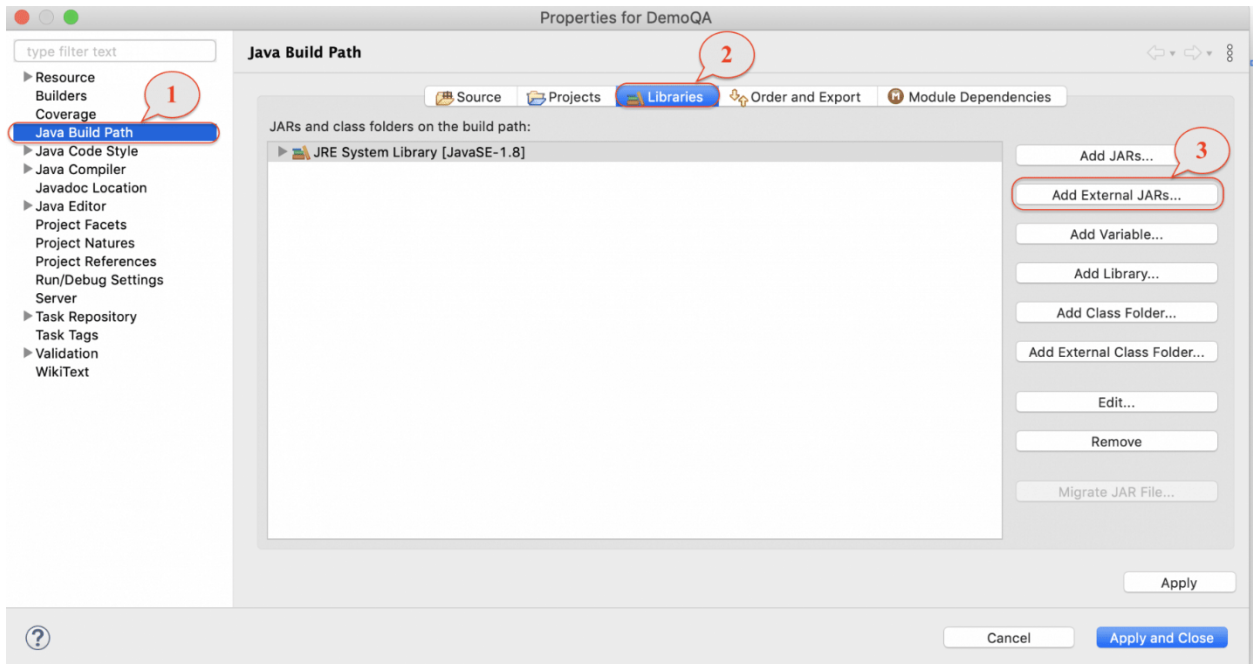
How to add Selenium WebDriver Jars to the project?

To include any of the Java libraries in the Eclipse projects, the same need to have in the ***Build Path of Eclipse**. Following the same, the Selenium WebDriver libraries also need to be included in the "**Build Path**" of **Eclipse**. As we have already downloaded the Selenium WebDriver, We can include those **Selenium WebDriver Jar files** into the **Eclipse project** by following the steps as mentioned below:

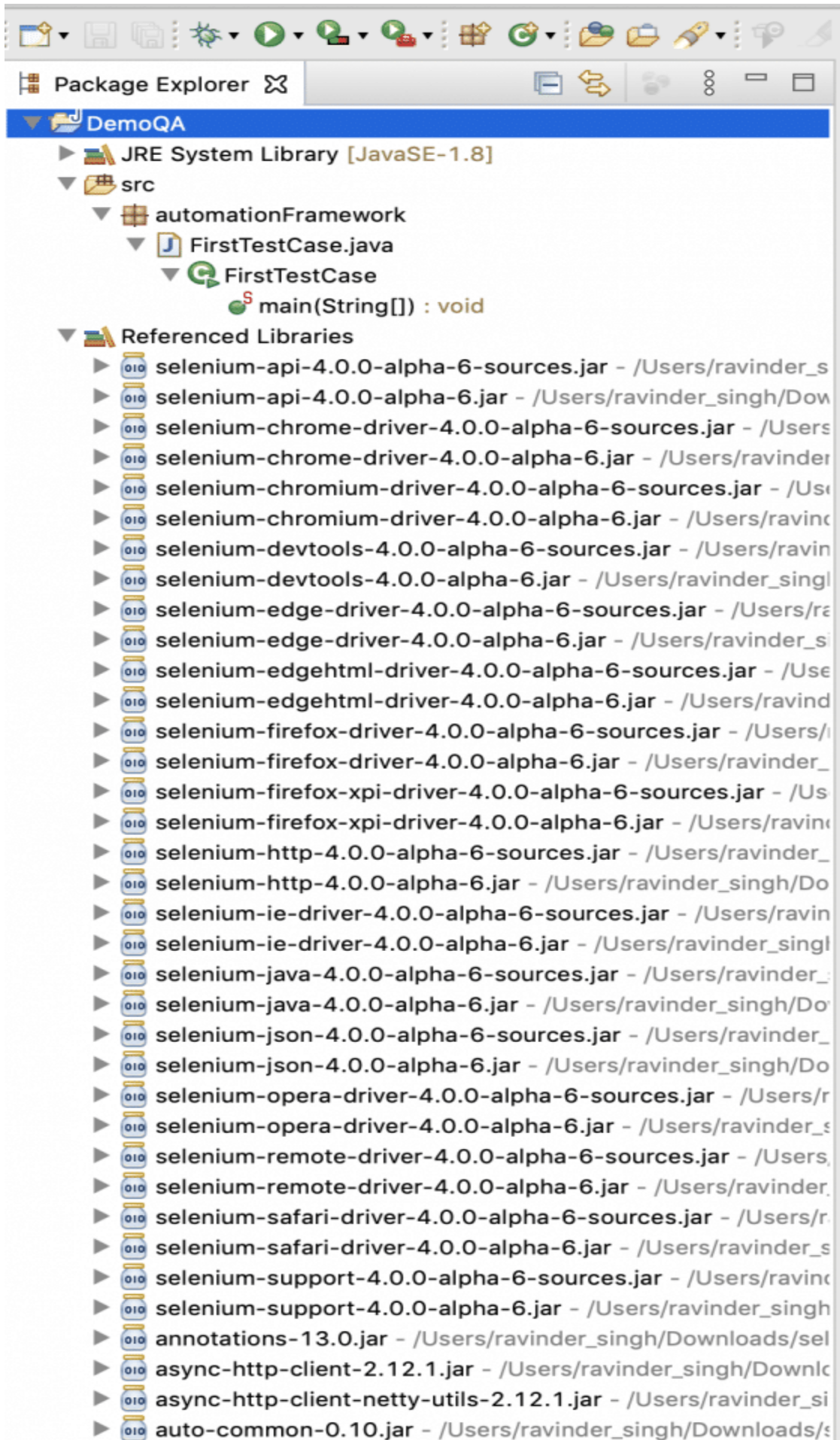
1. Firstly, either Right-click on the project '*DemoQA*' > **Build Path** > **Configure Build Path**



2. Secondly, as "**Selenium WebDriver**" is an external library for the project, so we need to add the "**Selenium WebDriver**" dependencies as "**External JARs**".



- First, select the "**Java Build Path**" (as shown by marker 1) in the left panel, if it is not auto-selected.
 - After that, click on the "**Libraries**" tab, as shown by **marker 2**.
 - Finally, click on the "**Add External JARs**" button, as shown by **marker 3**.
3. Thirdly, browse to the folder where you have extracted all the **Selenium WebDriver** jar files.
 4. Lastly, once you finish adding all the "**JAR**" files, click on the button "**Apply and Close**". It will show all the **JAR** files in the "**Project Explorer** " window under the "**Referenced Libraries** " folder as shown below:



The configuration of the **Selenium WebDriver jars** *is now complete*. Conclusively, you are all set to write your **first test script using Selenium WebDriver in Eclipse**.

TestNG

TestNG is a testing framework inspired from JUnit and NUnit but introducing some new functionality that makes it more powerful and easier to use. It is an open-source automated testing framework; where NG of TestNG means Next Generation. TestNG is similar to JUnit but it is much more powerful than JUnit. It is designed to be better than JUnit, especially when testing integrated classes.

TestNG eliminates most of the limitations of the older framework and gives the developer the ability to write more flexible and powerful tests with help of easy annotations, grouping, sequencing & parametrizing.

Benefits of TestNG

1. It gives the ability to produce HTML Reports of execution
2. Annotations made testers life easy
3. Test cases can be Grouped & Prioritized more easily
4. Parallel testing is possible
5. Generates Logs
6. Data Parameterization is possible

What is a Selenium WebElement?

A **WebElement** in Selenium represents an HTML element on a web page, allowing interaction (click, send keys, retrieve text) through methods like **click()**, **sendKeys()**, and **getText()**. WebElements provide methods to interact with those elements, retrieve information, and verify element properties.

The WebElement interface in Selenium enables interaction with both visible and hidden elements on a web page. WebElement command returns either null/void or the element found.

Syntax of Selenium WebElement

<start tag> content </end tag>

- HTML elements can contain other elements. Every HTML document carries such HTML elements.
- WebElement Selenium WebDriver methods apply to almost all DOM elements on a web page.
- Each WebElement is represented in [Selenium](#) via the WebElement interface – which is used by Selenium to interact with visible and invisible elements on the web page.

Example of a WebElement in Selenium command:

```
WebElement element = driver.findElement(By.id("UserName"));
```

List of Selenium WebElement Commands

1. `sendKeys()` – Type text into input fields.
2. `isDisplayed()` – Check if the element is visible on the page
3. `isSelected()` – Check if the element is selected
4. `submit()` – Submits a form or triggers a submit action on a form element
5. `isEnabled()` – Check if the element is enabled
6. `getLocation()` – Returns values of the x and y coordinates of the WebElement relative to the top-left corner of the viewport.
7. `clear()` – Clear text from an input field
8. `getText()` – Get the visible text inside the WebElement.
9. `getTagName()` – Returns the tag name of the WebElement as a string. The tag name is the type of the HTML element.
10. `getCssValue()` – Get the value of a specified CSS property of a WebElement.
11. `getAttribute()` – Get the value of a specific attribute of the element
12. `click()` – Click on the WebElement
13. `getSize()` – Returns values of height and width of the WebElement.

What is testng.xml file and its purpose?

TestNG is a popular testing framework in Java that provides several features for writing test cases. One of the key components of TestNG is the `testng.xml` file, which allows for easy configuration and control over test execution.

The `testng.xml` file is a configuration file used by the [TestNG](#) testing framework in Java. It allows you to define and organize your test suite, specify which test classes to run, and configure various aspects of your test execution. This XML file provides a way to manage complex test scenarios and control the test execution in a more structured way.

It allows for:

- Grouping test cases into test suites and test groups.
- Specifying the order of execution.
- Controlling parallel execution and setting thread count.
- Filtering which test methods to run.

What is the Purpose of the testng.xml File?

1. **Centralized Configuration:** Instead of hardcoding configurations into individual test classes, the `testng.xml` file allows you to configure your entire test suite in a single place. This makes it easier to adjust test settings, change the execution order, or add/remove tests without modifying the test code itself.
2. **Modularity:** The file lets you create multiple test groups, suites, and sub-suites, which enables modular testing. This is useful for organizing tests based on functionality, teams, or release cycles.
3. **Control Over Execution:** You can use the `testng.xml` to set the execution order of test cases, manage parallel execution, and even specify conditions under which certain tests should or shouldn't run (e.g., based on environment, priority, or dependency).
4. **Filter and Fine-tune Tests:** By specifying only certain methods or classes to run, you can selectively execute parts of your test suite. This is beneficial when you're testing specific features or modules.
5. **Ease of Maintenance:** The file promotes better maintenance of your test suite, as you can modify test configurations without altering the test scripts. This separation of concerns makes scaling and updating test suites more manageable.

Code Example

Step 1. Running Tests with testng.xml

```
package com.example.tests;

import org.testng.annotations.Test;

public class TestClass1 {
    @Test
    public void testMethod1() {
        System.out.println("TestClass1 - testMethod1 executed");
    }
}

public class TestClass2 {
    @Test
    public void testMethod2() {
        System.out.println("TestClass2 - testMethod2 executed");
    }
}
```

TEST REPORTING

Test reporting is a process in software testing that involves gathering, analyzing, and presenting essential test results and statistics to stakeholders. Additionally, a Test Report is a detailed document that contains a summary of the test, the process involved and the final test results. A Test Report is a summary of all the tests that were done, what was intended to be achieved with them, and what were the testing results once the tests were complete. It aims to summarize the results of the entire testing process formally and shares a comprehensive view of the testing status overall.

Importance of Test Reporting

Test reporting is crucial for software development from an ROI perspective. It helps:

- Maintain Cost-effectiveness
- Ensure release readiness
- Improve User Churn Rate
- Better Visibility and Control

Maintain Cost-Effectiveness

A good Test Report helps identify critical bugs very early in the cycle. It helps in the Root Cause Analysis, empowering the team with better debugging capabilities. This saves a lot of time and cost as the testers do not need to manually run the tests and report issues. Detecting a bug faster is always crucial as it can **save up to 85% of the cost** that would occur if they are caught later in the cycle.

Ensure Release Readiness

Getting a clear sense of release readiness with test coverage assessment, defect root cause analysis, and requirements validation by Test Report helps deliver high-quality software. Test Report validates if the requirements are successfully met. It aids in decision-making by presenting the current status of the software under test. It helps identify defects and issues, classifying them based on severity, which makes it easier to decide the release readiness.

Improving User Churn Rate

Software industry is highly competitive, and users expect a seamless bug-free user experience to continue using an application.

Test reports have detailed information about the number of tests executed, test scenarios, no of the tests that were passed/failed, known defects, if any and much more. These details provide valuable insights about the overall product health.

Test Report contributes to **continuous improvement** in the testing process. Teams can analyze the report to identify areas for process enhancement and optimization. This can ensure a bug-free software release for the users, improving the overall user churn rate with a better experience.

Better Visibility and Control

Test Report allows project stakeholders to track progress, identify bottlenecks, and make informed decisions, especially for larger projects. It enables project managers to have better control over the testing activities.

Test reports provide the necessary information and valuable insights to stakeholders about the product quality, help them keep track of the product and decide whether it is ready for release. It also helps them in measuring whether it is aligning with the business requirements.

Types of Test Reports

- **Test Summary Report:** It provides a high-level overview of the testing activities conducted during a specific phase or cycle of the project. It includes metrics such as the number of test cases executed, passed, failed, and any outstanding defects. The summary report helps stakeholders understand the overall status of the testing effort.
- **Test Execution Report (TER):** It provides detailed information about the execution of test cases, including the test case ID, description, status (pass/fail), and any comments or observations from the tester. TER allows stakeholders to track the progress of testing and identify areas that may require further attention.
- **Defect Report/Bug Report:** It documents any defects or issues identified during testing. It typically includes information such as the defect ID, description, severity, priority, steps to reproduce, and status. The defect report helps prioritize and manage the resolution of issues found during testing.
- **Traceability Matrix:** A traceability matrix maps requirements to test cases, ensuring that each requirement has been adequately tested. It helps verify that all project requirements have been addressed and provides a clear understanding of the test coverage.

- **Performance Test Report:** This report presents the results of performance testing, including metrics such as response time, throughput, resource utilization, and scalability.
- **Security Test Report:** For applications handling sensitive data or operating in security-critical environments, a security test report documents the findings of security testing activities. It identifies vulnerabilities, potential threats, and recommendations for improving the security posture of the software.
- **Regression Test Report:** When conducting regression testing, a report may be generated to summarize the impact of changes on existing functionality. It includes information about the test cases executed, any regressions detected, and the overall stability of the system after changes.
- **Compliance Test Report:** In regulated industries such as healthcare or finance, compliance test reports ensure that software meets relevant regulatory requirements and standards. These reports document adherence to specific guidelines and may be required for certification or approval.
- **User Acceptance Test (UAT) Report:** This report documents the results of user acceptance testing, where end-users validate whether the software meets their requirements and expectations. It includes feedback from users, any issues encountered, and the overall readiness of the software for deployment.

Components of Test Report

1. **Test Objective** includes the purpose of the testing and clear requirements from the overall testing cycle.
2. **Areas Covered** include the overall regions and the functionalities of the product that were covered in testing. It need not capture every test scenario in detail but should cover all the areas at a high level.
3. **Areas Not Covered** capture the product areas that were not covered in testing. Any areas not tested are clearly highlighted, setting expectations accordingly. It also specifies the reasons for the areas not covered and the risks involved.

4. **Testing Approach** indicates the systematic method used to plan, design, execute, and evaluate tests based on objectives, constraints, and requirements.
5. **Defect Report** captures details about the defects identified, its root cause analysis, and management. Though it usually gets captured already during the testing process. However, including it in the test summary report gives additional information and insights about the defects.
6. **Platform Details** include details of the platform and environment used during testing. This helps in troubleshooting if any defects are identified. Also, helps in analyzing coverage, risks, and robustness of the testing.
7. **Overall Summary** includes commentary on the application's overall status under test. It informs the stakeholders about the critical issues discovered and its status (open/closed). It helps understand the overall readiness of shipping the product to the customer.

Characteristics of a Good Test Report

Some of the core characteristics of a good test report are:

- It captures the purpose of the document clearly.
- It gives a high-level overview of the product's functionality that is being tested.
- It captures the test scope, highlighting:
 - Modules in scope
 - Modules out of scope
 - Module that was in scope but couldn't be tested due to any underlying constraint or dependency.
- It captures different testing metrics for all kinds of tests that were performed, majorly
 - No. of test cases planned
 - No. of test cases executed
 - No. of test cases passed
 - No. of test cases failed
- It includes the different types of Testing that were performed, such as Unit Testing, Smoke Testing, Sanity Testing, Regression Testing, and Performance Testing
- It specifies the Test Environment, Tools and Frameworks used for testing.

- It clearly defines the Exit Criteria, which, once successfully met, the Testing team gives a go-ahead for the application to Go Live.